國立臺灣海洋大學資訊工程學系 專題報告

自動化羽球賽事分析系統

從單打轉播影片產出逐拍資料

作者

01257001　宋辰星　01257001@mail.ntou.edu.tw
01257019　劉俊麟　01257019@mail.ntou.edu.tw
01257134　陳宥嘉　01257134@mail.ntou.edu.tw
01268135　洪于宸　01268135@mail.ntou.edu.tw

指導教授：張欽圳　博士

中　華　民　國　一　百　一　十　五　年　　月　　日

Department of Computer Science and Engineering, National Taiwan Ocean University
Project Report

Automated Badminton Match Analysis from Broadcast Video

Stroke-Level Data for Singles Matches

Author

01257001	Sung Chen Hsing	01257001@mail.ntou.edu.tw
01257019	Liu Chun Lin	01257019@mail.ntou.edu.tw
01257134	Chen Yu Chia	01257134@mail.ntou.edu.tw
01268135	Hung Yu Chen	01268135@mail.ntou.edu.tw

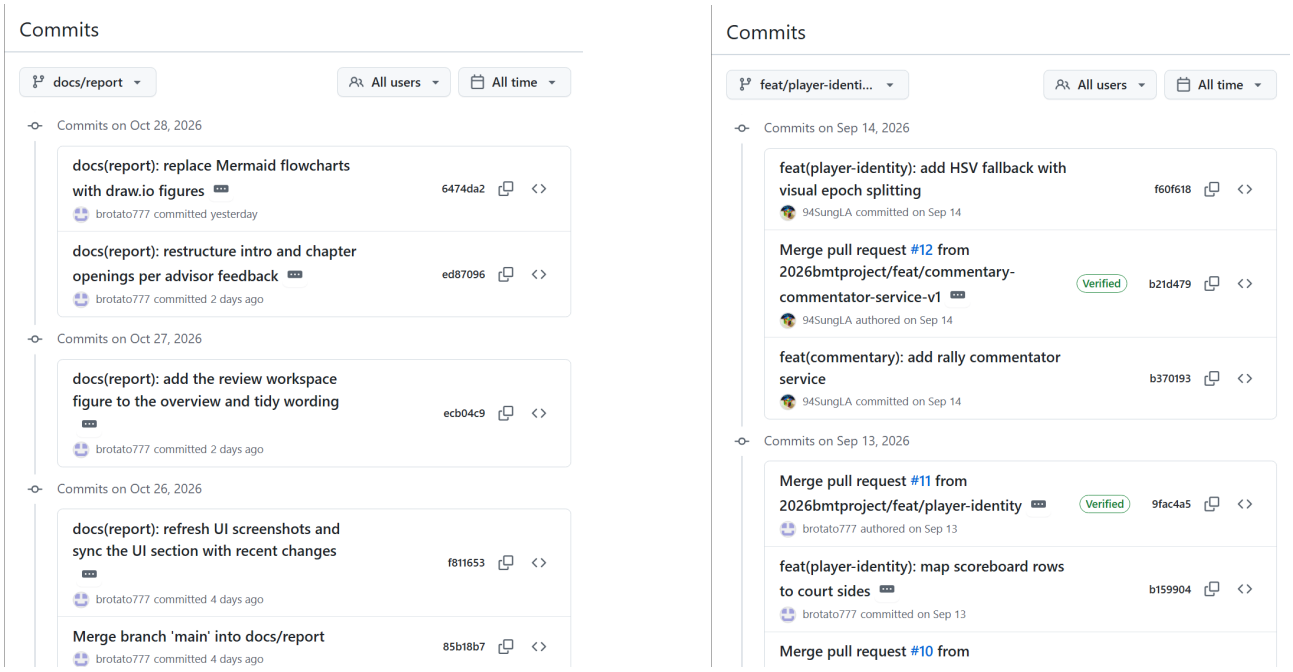
Advisor：Dr. Chin-Chun Chang

2026/ /

專題分工及貢獻度說明

編號	姓名	主要工作內容	專題貢獻度
1	劉俊麟	系統架構，賽事影片分段、比分辨識、場地偵測、球員骨架追蹤、球體追蹤、擊球時刻偵測、球種分類、球員身分對應，實驗資料集、各模組評分與端到端測試，前端介面設計與重構，報告主體撰寫與 LaTeX 排版、海報、競賽影片	67%
2	宋辰星	專題構想，歡呼偵測與標註工具、精彩片段排序、賽評生成，球員身分對應視覺備援，回看資料匯出、本機分析任務服務、Windows 桌面版與場地校正介面，賽評與音訊實驗，報告音訊章節與系統架構圖	27%
3	陳宥嘉	文獻調查，ShuttleSet 資料整理，規則式精彩片段評分原型，前端介面初版，競品分析	4%
4	洪于宸	歡呼資料標註，報告第一、二章初稿與競品文獻回顧，行政事務	2%

專案管理系統截圖



GitHub 工作紀錄

Trello 工作紀錄

目錄

1 簡介 **2**

1.1 研究動機 2

1.2 專題目標 2

1.3 系統概觀 3

1.4 主要貢獻 4

1.5 效能評估結果 5

1.6 報告組織 5

2 文獻探討 **6**

2.1 系統層級的比較 6

2.2 關鍵子任務的既有方法 7

2.2.1 回合切割 7

2.2.2 球場偵測 7

2.2.3 球體追蹤 8

2.2.4 擊球偵測 8

2.2.5 球種辨識 8

2.2.6 比分辨識 8

2.3 研究缺口與本系統定位 9

3 系統設計 **10**

3.1 系統架構 10

3.1.1 管線約定 11

3.2 影片前處理 12

3.2.1 賽事影片分段 12

3.2.2 比分辨識 18

3.3 場景與人體理解 20

3.3.1 場地偵測 20

3.3.2 球員骨架追蹤 30

3.4 球體分析 32

3.4.1 球體追蹤與軌跡重建 32

3.4.2 擊球時刻偵測 35

3.4.3 球種分類 41

3.5 音訊與精彩度分析 43

3.5.1 歡呼音訊量測 43

3.5.2 精彩片段排序 45

3.6 賽事理解 46

3.6.1 球員身分對應 46

3.6.2 賽評生成 47

3.7 使用者介面 51

3.7.1 概述 51

3.7.2 回看工作區 52

3.7.3 分析任務與場地校正 55

3.7.4 實作概要 56

4 實驗結果 **58**

4.1 資料集與重現方式 58

4.2 各模組結果 59

4.2.1 賽事影片分段 59

4.2.2 場地偵測 60

4.2.3 擊球時刻偵測 62

4.2.4 球種分類 64

4.2.5 歡呼偵測與精彩度分析 66

4.3 端到端結果 68

4.3.1 處理時間 69

4.3.2 時間分布 69

4.3.3 限制 69

4.3.4 應用於延遲直播的可能性 69

5 討論與結論 **71**

5.1 系統優點與限制 71

5.2 結論 72

人工智慧的使用 **73**

開發過程中的生成式 AI 73

確保正確性的做法 73

參考文獻 **75**

圖目錄

1.1 系統概觀 3

1.2 回看介面 4

3.1 系統架構 11

3.2 賽事影片分段處理流程 13

3.3 部分片段 MAD 14

3.4 取對數 14

3.5 固定機位比賽片段與其他片段的 MAD 分佈 15

3.6 三類 Otsu 的兩個切點 15

3.7 跨片段兩兩 MAD 矩陣（依平均差異排序）. 16

3.8 跨片段平均差異的分佈與門檻 17

3.9 邊界尖峰偵測與修正 17

3.10 比分辨識處理流程 18

3.11 overlay 只出現在部分影格時，主群集平均的還原結果 19

3.12 轉錄取代挑選，VLM 完整抄寫每隊數字，程式取最右一個為當前比分 20

3.13 場地偵測處理流程 21

3.14 四張取樣影格與合成圖 22

3.15 全畫面掃描到的線段，與落在球場遮罩內的部分 22

3.16 取樣區塊、參考色與門檻的判定 23
3.17 形態學清理、最大連通區域與空洞填補各階段的遮罩變化 23
3.18 場邊看板與球場同色時，紋理版本剔除色度版本誤納的區域 24
3.19 局部亮度突起與場線像素的萃取結果 24
3.20 霍夫線段與兩族的角度粗分 25
3.21 消失點 RANSAC 三次抽樣的候選點與其支持度 25
3.22 同族線段合併的前後對比 25
3.23 合法性檢查 26
3.24 場線像素與其距離轉換圖 27
3.25 網格枚舉的候選與其吻合度評分 27
3.26 ICP 第一輪與最終輪的網格與反投影殘差 28
3.27 投影線的鄰近像素與其離線距離直方圖 29
3.28 遠端底線由內側峰改取最外側峰的前後對比 29
3.29 候選池的吻合度評分與勝出者的投影 29
3.30 球員骨架追蹤處理流程 30
3.31 由上而下，先裁切放大再估骨架 31
3.32 腳點反投影至球場座標 31
3.33 整場腳點落點，固定不動者被判為靜態錨點 32
3.34 球體追蹤與軌跡重建處理流程 33
3.35 從熱圖以 largest-blob 讀出基準軌跡 33
3.36 InpaintNet 補回球員遮擋造成的缺幀 34
3.37 低門檻多候選與 Viterbi 選出的最佳路徑 34
3.38 擊球時刻偵測處理流程 35

3.39 逐幀滑動窗口產生的球種、信心與擊球者證據 36

3.40 四種互補的擊球候選訊號 37

3.41 各訊號在完整回合中的候選分布與合併結果 38

3.42 三道候選篩選規則及其保留／刪除結果 39

3.43 由擊球序列與連續球種證據反推漏拍位置 40

3.44 由輔助軌跡與發球錨點救回漏拍 40

3.45 利用回合有效邊界清除開頭誤偵與尾段落地球 41

3.46 結合比分與發球證據判定計分板死時間 41

3.47 球種分類處理流程 42

3.48 依相鄰擊球時刻建立 BST 的 100 幀輸入窗口 42

3.49 BST 的 25 類預測 43

3.50 歡呼音訊量測處理流程 43

3.51 片段音訊的讀取範圍與切窗 44

3.52 精彩片段排序處理流程 46

3.53 球員身分對應處理流程 47

3.54 賽評生成處理流程 48

3.55 使用者介面操作流程 51

3.56 比賽庫，每場比賽標示分析狀態與是否可回看 51

3.57 分析設定，每個分析項目標示已完成、未執行或已過期 52

3.58 回看工作區：① 播放器與疊圖 ② 回合分析欄 ③ 時間軸 52

3.59 疊圖開啟前後：(a) 原始轉播畫面 (b) 開啟全部圖層 53

3.60 回合分析欄：(a) 擊球序列與精華排名 (b) 產生賽評後 (c) 擊球點分頁 54

3.61 時間軸的四種模式（全場尺度）：(a) 片段 (b) 擊球 (c) 比分 (d) 歡呼 55

3.62 擊球模式在兩種尺度下的呈現：(a) 全場 (b) 單回合 55

3.63 分析計畫預覽 56

3.64 任務進度，顯示整體與各階段的進度及詳細記錄 56

3.65 場地校正，拖曳四個外角點對齊球場邊線 57

3.66 使用者介面實作架構 57

表目錄

1.1 各模組的代表指標 5

2.1 既有整合系統與開源專案在各子任務的做法 7

2.2 本系統在各子任務採用的路線 9

3.1 管線各階段的相依與產物 12

3.2 $\log_{10}(\mathrm{MAD})$ 的分佈與組成 14

3.3 賽評生成各項資訊的來源與處理方式 48

3.4 賽評生成的實際輸出範例 50

4.1 資料集切分 58

4.2 賽事影片分段的偵測準確率（%） 59

4.3 賽事影片分段的邊界誤差（影格） 60

4.4 賽事影片分段的失配成因 60

4.5 場地偵測的角點定位誤差（像素） 61

4.6 場地偵測的命中率 61

4.7 場地偵測的未命中成因 62

4.8 擊球時刻偵測的準確率（%，容差 ±5 影格） 63

4.9 不同容差下的擊球時刻偵測準確率（%，全部 14 場） 63

4.10 擊球時刻偵測的時間誤差（影格） 63

4.11 擊球時刻偵測的失配成因（測試集） 64

4.12 球種分類的準確率（%） 64

4.13 各球種的分類表現（BST 沒看過的 3 場，%） 65

4.14 以系統偵測或真值擊球幀進行球種分類的比較（%） 65

4.15 球種分類的錯誤成因（BST 沒看過的 3 場） 65

4.16 歡呼偵測的標註資料 66

4.17 歡呼偵測的逐場交叉驗證結果（LOMO） 66

4.18 歡呼分類器的校準診斷 67

4.19 不同歡呼特徵路線的比較（LOMO） 67

4.20 歡呼強度候選方法與人工配對比較的一致性 67

4.21 片段層級聚合方法的比較 68

4.22 端到端處理時間 69

4.23 各階段佔總處理時間的比例 69

摘要

本專題開發一套羽球單打轉播影片的全自動分析系統，將一場完整的轉播影片轉換為逐拍資料。分析管線依序進行賽事影片分段、比分辨識、場地偵測、球員骨架追蹤、球體追蹤、擊球時刻偵測與球種分類，各階段以統一的資料契約串接，可單獨替換與重跑，單機一張消費級 GPU 即可執行。在管線之上，系統以大型語言模型生成賽評。另依觀眾歡呼為比賽片段排出精彩程度。使用者可在回看介面對照影片檢視所有結果。以 16 支公開的國際賽事轉播建立評估資料。在未參與開發的測試比賽上，影片分段 F1 為 98.66%，擊球時刻偵測 F1 為 93.14%。目前的主要瓶頸在於球種分類模型本身與擊球時刻的漏偵測，且系統尚無法察覺自身輸出的錯誤。

專案連結

GitHub 原始碼

github.com/2026bmtproject/badminton-analysis-system

系統介紹影片

youtube.com/watch?v=Ep4Ka1XEgFk

Chapter 1

簡介

1.1 研究動機

羽球國際賽事的完整轉播影片大量公開於網路，記錄了選手每回合攻防的過程。教練與選手想從中歸納對手的慣用球路，研究者想統計戰術或訓練預測模型，真正需要的都不是影片本身，而是逐拍的資料，像是每回合有幾拍、每拍是什麼球種等等。影片並不直接提供這些資訊，目前仍須由人標記。ShuttleSet [1] 等羽球資料集即以人工標記建立，自動定位擊球影格的研究 [2] 也以減少這類人工作業為出發點。

在嘗試將既有方法套用於完整轉播影片時，我們發現學術上的單項方法大多預設輸入已經整理好，例如球種分類需要已知的擊球時刻與球場範圍。這些前提在資料集中由人工標記提供，面對一支未經處理的轉播影片時卻無從取得。要讓所有分析在真實影片上運作，就必須將它所依賴的上游步驟自動化。

因此，本專題欲補足從「原始轉播影片」到「逐拍資料」之間的落差，使後續的戰術分析、賽事理解與資料驅動應用，能建立在可由真實轉播影片取得的資料之上。

1.2 專題目標

本專題旨在建立一套可批次執行的全自動分析管線，將一場完整、未經人工標記的羽球單打轉播影片，轉換為可供檢索、統計與後續應用的資料，並以賽評生成與精彩片段排序兩項延伸應用，示範管線產出的用途。

- **輸入**：一支完整、未經剪輯的羽球單打轉播影片，比賽中的畫面以固定俯拍的主機位為主，即 BWF 國際賽事的常見轉播形式。

- **輸出**：整場比賽的逐拍資料，包括每個主機位比賽片段在影片中的起訖時間與比分，片段內每一拍的擊球影格、擊球者與球種，以及雙方球員的骨架、球場範圍與羽球軌跡。可選擇另外執行賽評生成與精彩片段排序。
- **運作方式**：賽後批次處理，單機一張消費級 GPU 即可執行。除比分辨識與賽評需呼叫外部 API 外，其餘皆在本機完成，影片本身不需上傳。

非本專題目標：雙打、即時轉播分析，以及取代 Hawk-Eye 等需專用攝影設備的電子判決系統。

1.3 系統概觀

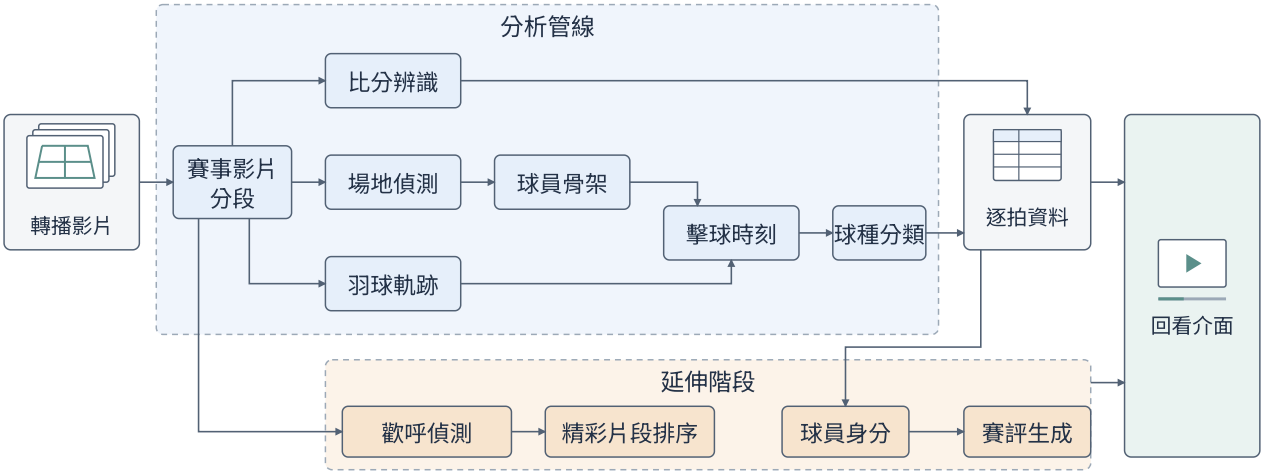


圖 1.1: 系統概觀：分析管線將轉播影片轉為逐拍資料，延伸階段與回看介面建立在其產出之上。

系統由三部分組成（圖 1.1）：把轉播影片轉成逐拍資料的分析管線、建立在管線產出之上的延伸階段，以及對照影片檢視結果的回看介面。分析管線依序處理以下步驟：

1. **界定分析範圍**：轉播影片中夾雜重播、特寫等畫面，賽事影片分段先找出主機位的比賽片段，後續所有分析都只在這些片段內進行。比分辨識再讀出每個片段計分板上的比分，讓片段能對應回比賽中的比分。
2. **定位球場、球員與羽球**：場地偵測定位球場範圍，建立畫面與球場平面之間的對應。球員骨架追蹤據此從裁判、線審與觀眾之中挑出場上的兩名球員。球體追蹤則逐影格定位羽球，重建羽球軌跡。這三項是後續擊球分析共同需要的輸入。
3. **找出每一拍並分類**：擊球時刻偵測結合羽球軌跡與球員骨架，找出每一次擊球的影格。球種分類再依每一拍前後的骨架、軌跡與球場位置，判斷球種。

管線之外另有四個延伸階段。球員身分對應依比分與發球者，把上下方對應到具體選手。賽評生成再以逐拍結果與球員身分為事實依據，由大型語言模型撰寫賽評，並逐句檢查是否有資料支持。歡呼偵測量測每個片段結束前後的觀眾歡呼，精彩片段排序據此為片段排出精彩程度。

回看介面（圖 1.2）除了對照影片檢視所有結果，也能直接發起分析任務。各模組的設計與實作見第三章。

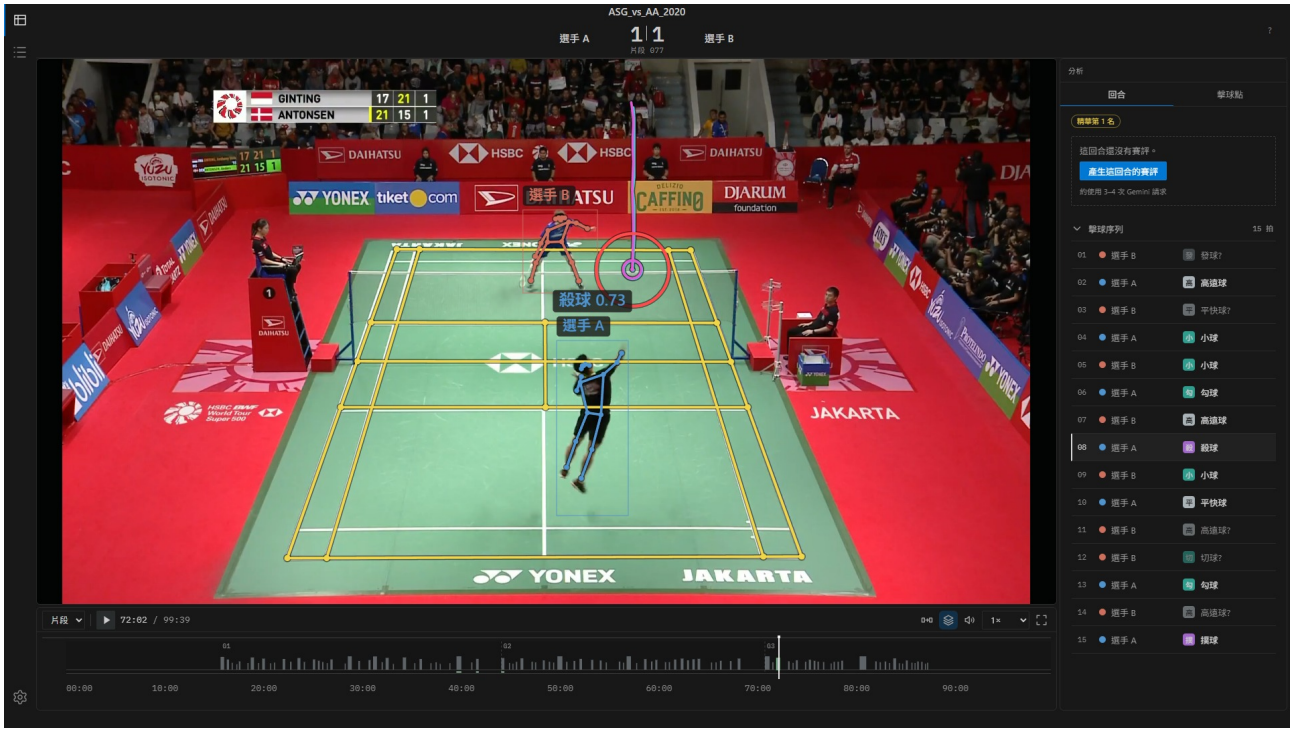


圖 1.2: 回看介面

1.4 主要貢獻

上述各模組多有既有方法可以參考，但如 1.1 節所述，這些方法預設輸入已經整理好，比賽片段起訖、球場範圍與擊球時刻都由人工提供。與既有的整合系統及開源專案相比，本系統的主要貢獻有以下四點。

1. **把既有方法所依賴的上游自動化：**比賽片段起訖、球場範圍與擊球時刻在資料集中原由人工提供，本系統分別以影格差異訊號分段、由內部線條反推範圍的場地偵測，以及融合軌跡、骨架與球種證據的擊球時刻偵測取得，不需針對每支影片人工標記或調整參數。TrackNetV3 [3]、BST [4] 等既有模型因此能直接在未經處理的轉播影片上運作。

2. **模組化、可局部重跑的管線：**各模組仍在持續改進，因此每個階段都能單獨替換與重跑。階段之間只透過統一的資料契約溝通，執行器以上游產物的內容雜湊判斷結果是否過期，中間結果以片段為單位快取。調整其中一個模組時，只需重算受影響的部分。

3. **公開、可重現的評估：**實驗使用 16 支 YouTube 公開的單打完整轉播，以影片為單位切分為開發集與測試集。我們公開每支影片的雜湊與影格指紋，讓他人能確認自己下載的影片與我們標記時所用的一致。真值、評分程式與逐場結果也一併公開。

4. **應用示範：以比賽事實約束生成式 AI 的賽評：**為展示逐拍資料的下游價值，我們在管線之上建立賽評生成，重點在於避免大型語言模型的幻覺。每一拍的時刻、擊球者與球種都取自管線結果，模型只負責措辭，產生的賽評再經審查比對分析結果，沒有資料支持的句子改以只陳述事實的模板句取代。

1.5 效能評估結果

表 1.1 列出各模組的代表指標，完整實驗與失敗案例分析見第四章。

表 1.1: 各模組的代表指標

模組	評估資料	主要指標	其他指標
賽事影片分段	測試集 9 場、670 個片段	F1 98.66%（IoU $\geq$ 0.9）	邊界誤差中位數 0 影格
場地偵測	測試集 9 場、36 個外角	命中率 72.22%	誤差中位數 2.79 像素，6 場四角全中
擊球時刻偵測	測試集 8 場、6,990 拍	F1 93.14%（$\pm$5 影格）	Precision 95.49%、Recall 90.90%
歡呼偵測	開發用 4 場逐場留一，外部 1 場	Macro F1 82.70%	ROC-AUC 0.96
處理時間	5 場、片長 332 分（片段 98 分）	片段總長的 2.71 倍	扣除 API 等待後 1.96 倍（RTX 3060 Ti）

分段與擊球時刻偵測的 F1 皆超過 93%，已可直接供下游使用。最弱的一環是場地偵測，未命中的角幾乎都源於底線旁的廣告字樣，且系統目前無法自行察覺。16 支影片中有 15 支來自 BWF 官方轉播，跨轉播單位的泛化尚未驗證。歡呼偵測僅以 4 場開發比賽與 1 場外部驗證評估。處理時間未計入音訊、身分對應與賽評等延伸階段。

1.6 報告組織

本報告其餘章節安排如下。第二章回顧既有的整合系統與各子任務的方法，歸納研究缺口與本系統的定位。第三章先說明管線架構與階段間的約定，再依處理順序介紹各模組的設計，包括影片前處理、場景與人體理解、球體分析、音訊與精彩度分析、賽事理解，最後介紹使用者介面。第四章說明資料集與重現方式，報告各模組與端到端的實驗結果。第五章討論系統的優點與限制，並總結本專題與未來工作。最後另以「人工智慧的使用」說明開發過程中生成式 AI 的使用方式。

Chapter 2

文獻探討

本章回答一個問題：既有整合系統能否把一支未經處理的轉播影片，自動轉換為逐拍資料？如果不能，卡在哪些環節？回顧範圍限於以單一轉播視角影片為輸入的方法。我們先在系統層級比較既有的整合系統與開源專案各自自動化了哪些環節，再逐一檢視這些環節的既有方法與前提，最後收斂出研究缺口與本系統的定位。

2.1 系統層級的比較

要從轉播影片取得逐拍資料，最直接的方式是使用現成的整合系統。表 2.1 依本系統面對的子任務，比較五個學術整合系統與三個 GitHub 開源專案在各環節的做法。開源專案的內容以其 README 與程式碼的描述為準。

沒有一個系統能不經人工，從原始轉播一路產出逐拍資料。回合範圍常由資料集或使用者提供 [5], [6], [7]，或需由使用者選定參考畫面、模板 [8], [9]。能自動切割回合的系統，則以人工標記的資料訓練分類器 [2], [10]。比分方面，沒有系統直接讀取計分板，最多只推算回合勝方 [5], [10]。

三個開源專案都沒有提供任何準確率等量化評估，結果是否可靠無從判斷 [7], [9], [11]。

文字報告方面，RallyLens 以 Gemini 生成回合分析報告，並讓數字全由程式計算、模型只負責詮釋，以避免數字幻覺 [7]。

表 2.1: 既有整合系統與開源專案在各子任務的做法

系統	輸入	回合切割	球場偵測	擊球偵測	球種辨識	比分	量化評估
學術整合系統							
Ghosh 等人 [10]	完整比賽	HOG + SVM	未說明	無	TCN 逐幀動作分段	由發球者推算勝方	有
Chien 與 Yu [2]	完整比賽	CNN 分類拍攝角度	Court R-CNN	姿態 Transformer	無	無	有
Bosi [8]	完整比賽	與使用者選定的參考畫面比對	使用者點選四角	軌跡幾何＋音訊	無	列為目標但未實現	有
MonoTrack [6]	回合片段	無	幾何式	HitNet	無	無	有
Guo 等人 [5]	回合片段	無	Hough 幾何式	擊球者切換＋羽球角度	手工特徵＋邏輯迴歸	只判回合勝方	有
GitHub 開源專案							
RallyLens [7]	使用者截取的片段	無	Hough 自動，可人工點選	手腕與羽球距離規則	無	無	無
Badminton-VisionAI [11]	完整比賽	README 稱自動，未說明方法	預設讀取固定關鍵點檔	未說明	Roboflow 雲端模型	無	無
RallyLive [9]	完整比賽或直播串流	與使用者選定的球場模板比對	比對白／黃場線模型，可人工修正	實驗功能	無	無	無

2.2 關鍵子任務的既有方法

2.2.1 回合切割

既有方法多利用「回合進行時，轉播固定以俯拍主機位拍攝」的慣例，把回合切割化約為逐幀判斷是否為比賽畫面。一類是監督式分類，Ghosh 等人以 HOG 特徵搭配 SVM 分類，再以 Kalman 濾波平滑 [10]。Chien 與 Yu 以 CNN 分類拍攝角度，單幀準確率達 99.76%，換算成回合後 F1 卻只有 89.27%，作者歸因於轉播鏡頭形式多變 [2]。另一類是與參考畫面比對，Bosi 計算每幀與使用者選定之參考畫面的 RGB 差與 PSNR，以經驗門檻判斷 [8]。

2.2.2 球場偵測

學習式方法以標記資料訓練場地關鍵點偵測器，例如以 Court R-CNN 輸出單打場地的 6 個關鍵點 [2]，或以 YOLO 架構偵測 14 個場地地標 [12]。幾何式方法利用場線結構與已知尺寸，不需訓練。AI-Powered Badminton Coach 以 Hough 直線取得候選線，再做透視校正 [5]。MonoTrack 把標準球場模型對到候選線上，並指出原演算法寫死的角度門檻使約四分之一的影片失敗，改良後成功率由 73.9% 提升至 85.5% [6]。Bosi 的系統則由使用者人工點出球場四角 [8]。

2.2.3 球體追蹤

羽球小、快且常被遮擋，既有方法分為熱圖式與偵測框式兩條路線。熱圖式以 TrackNet 家族為代表，輸入連續數幀、輸出標示球位置的熱圖 [13]。TrackNetV2 被多個整合系統直接採用 [5], [8]，TrackNetV3 再加入背景估計與 inpainting 軌跡修補 [3]。偵測框式如 Singha 等人修改 YOLOv8，移除大物件偵測頭以強化小物件偵測 [12]。此外，多數系統另以 ARIMA、內插等後處理修正軌跡 [5], [8]。

2.2.4 擊球偵測

既有方法依所用訊號可分為四種思路：

1. **軌跡幾何：**在球的 Y 座標找峰值、X 座標找極值 [8]，開源專案 SoloShuttlePose [14] 亦屬此類。殺球前後球的運動方向連貫，此法容易漏抓 [8]。
2. **球員姿態：**以 Transformer 由雙方骨架序列預測羽球飛行方向，方向轉換處即為擊球幀。容差由 ±15 收緊到 ±5 幀時，F1 由 79.55% 降至 63.92% [2]。
3. **多源學習式：**MonoTrack 的 HitNet 以 GRU 讀取球場角點、姿態與羽球位置，逐幀判斷是否擊球，再以動態規劃施加「最短間隔、擊球者交替」約束，F1 由 0.866 提升至 0.946 [6]。AI-Powered Badminton Coach 先判斷擊球者並以其切換處分段，再依羽球角度變化定位擊球幀 [5]。
4. **音訊：**以 LPC 殘差偵測擊球聲，但球評、觀眾等背景噪音使召回率不理想 [8]。

2.2.5 球種辨識

既有方法的差異主要在輸入型態，AI-Powered Badminton Coach 以 131 維手工特徵搭配邏輯迴歸 [5]。Singha 等人以 SlowFast 與 TimeSformer 直接處理原始影像 [12]。BST 則以球員骨架、羽球軌跡與場上位置為輸入 [4]。BST 須先知道擊球時刻才能切出每一拍的片段，並依賴姿態估計、羽球追蹤與場線偵測三個既有模型的輸出 [4]。

2.2.6 比分辨識

羽球文獻中鮮少直接辨識計分板，Ghosh 等人以下一回合的發球者推算上一回合的勝方，比分只在建立資料集時由人工標記 [10]。AI-Powered Badminton Coach 只判斷回合勝方、不讀比分 [5]。Bosi 雖把比分列為目標，最終並未實現 [8]。網球的 SmartTennisTV 自動定位計分板後以 OCR 辨識，再以計分規則修正錯誤，但指出 Tesseract 需依賽事人工設定前處理 [15]。

2.3 研究缺口與本系統定位

綜合前兩節，既有工作仍有兩項缺口。第一，沒有系統能從原始轉播影片一路自動產出逐拍資料。回合範圍、球場位置等上游前提多由資料集或使用者提供，也尚無系統直接讀取計分板，連 HitNet、BST 等較強的模型也假設這些輸入已經就緒。第二，已經自動化的環節多需以逐幀或場地標記訓練，換賽事、換轉播形式就可能失準。開源專案則普遍沒有量化評估，難以判斷結果是否可靠。

本系統定位為一條批次執行的分析管線。上游步驟不需另外訓練模型，也不需針對每支影片人工標記或調整參數，讓既有的成熟模型能直接用在未經處理的轉播影片上，並以公開的真值與評分程式評估各模組。在此基礎上再產生精彩片段排序與自然語言賽評。各子任務的路線如表 2.2。

表 2.2: 本系統在各子任務採用的路線

子任務	本系統的路線	詳見
回合切割	影格差異訊號，門檻由影片自身分布估計	3.2.1 節
比分辨識	VLM 轉錄計分板	3.2.2 節
球場偵測	幾何式，不預設球場顏色，由內部場線反推球場範圍	3.3.1 節
球體追蹤	預訓練 TrackNetV3，加上多候選 Viterbi 後處理	3.4.1 節
擊球偵測	融合軌跡、骨架與球種模型，以結構約束補漏拍	3.4.2 節
球種分類	預訓練 BST	3.4.3 節
歡呼偵測	YAMNet 特徵辨識觀眾歡呼，作為精彩度訊號	3.5.1 節

Chapter 3

系統設計

本章說明系統各部分的設計與實作。3.1 節先介紹整體架構，包括各階段的相依關係、產物，以及讓各階段能單獨開發與重跑的管線約定。接著依功能分五節介紹各模組：

- **影片前處理**：賽事影片分段與比分辨識，負責界定分析範圍。
- **場景與人體理解**：場地偵測與球員骨架追蹤。
- **球體分析**：球體追蹤與軌跡重建、擊球時刻偵測與球種分類。
- **音訊與精彩度分析**：歡呼音訊量測與精彩片段排序。
- **賽事理解**：球員身分對應、賽評生成。

最後一節介紹回看介面。

各模組大致可以獨立閱讀，開頭的概述依序說明該模組要解決的問題與目標、設計重點與處理流程總覽，之後再逐步說明各步驟的細節。各模組的實驗結果集中於第四章。

3.1 系統架構

管線共有十一個階段（圖 3.1），每個階段讀取上游的產物，輸出一個 JSON 產物。除分段直接讀取原始影片外，其餘階段都以 `segments.json` 界定分析範圍。各階段的相依與產物如表 3.1。

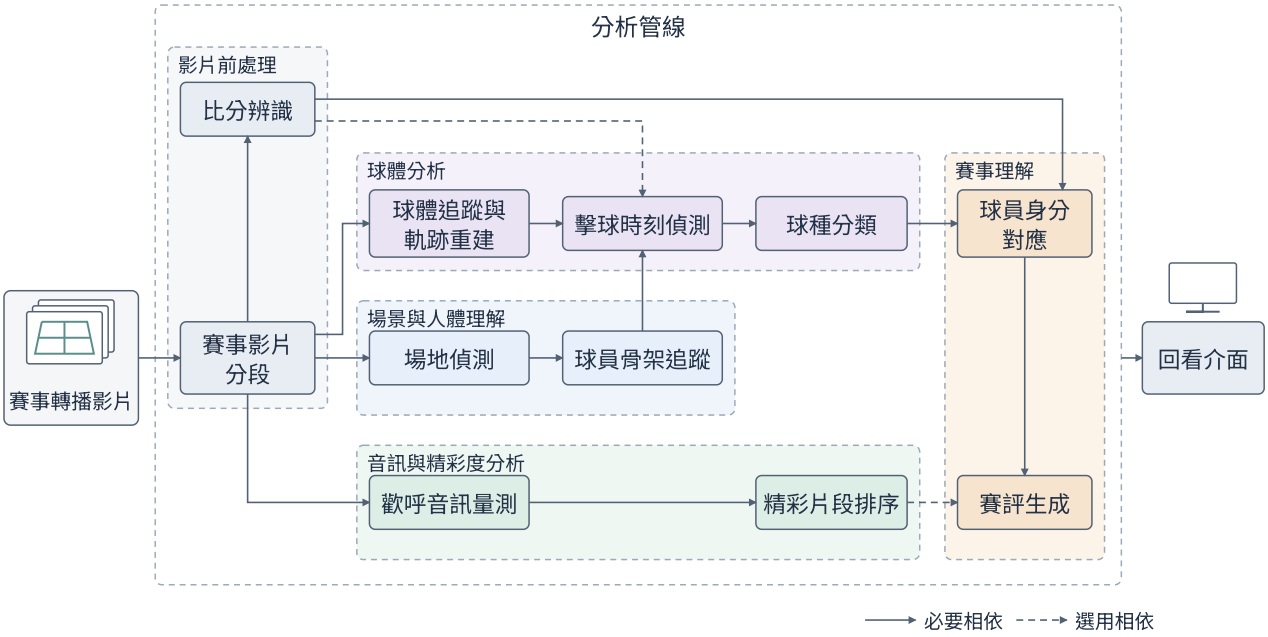


圖 3.1: 系統架構

3.1.1 管線約定

以下四個約定讓各階段能各自開發、單獨執行與重跑。

1. **單一產物、統一格式：**每個階段只寫一個 JSON 產物，外層固定是一個物件、內含一組記錄陣列，記錄欄位由程式中的資料類別統一定義。因為形狀一致，讀寫共用同一組函式，新增階段只需定義資料類別。

2. **以完整比賽時間對齊：**所有產物都以原始影片的時間為準，視覺結果記錄絕對影格編號，音訊窗口記錄在整場比賽中的起訖秒數，片段層級的結果則以片段在 `segments.json` 中的索引對應。

3. **產物與快取分離：**每場比賽的 `cache/` 放昂貴但可重建的中間結果（降解析度影片、TrackNet 熱圖、人體偵測結果、逐幀球種機率）。快取以內容命名並以片段為單位存放，因此中斷可續跑，重新分段時也只有內容改變的片段需要重算。

4. **依內容判斷是否重跑：**使用者可只選取部分階段，執行器依相依關係排序後逐一執行，已完成的階段預設跳過。透過比對上游產物的內容雜湊判斷是否跳過。任一階段失敗即停止整條管線。

表 3.1: 管線各階段的相依與產物

階段	相依階段	產物	每筆記錄的內容
賽事影片分段	—	segments.json	一個主機位比賽片段的起訖影格
比分辨識	分段	scores.json	一個片段的比分、發球方與局數；片段內含多個回合時，另記各回合比分與切點
場地偵測	分段	court.json	球場四個外角，以及球場平面與影像之間的單應矩陣
球體追蹤與軌跡重建	分段	shuttle.json	一幀中羽球的座標與可見性，兩種軌跡各一份
球員骨架追蹤	分段、場地偵測	pose.json	一幀中兩名球員的 17 個關節點與人體框
擊球時刻偵測	球體追蹤、骨架追蹤；選用：比分辨識	events.json	一次擊球的影格編號
球種分類	擊球時刻偵測、骨架追蹤、球體追蹤、場地偵測	strokes.json	一次擊球的球種（8 類）、擊球者（畫面上方或下方）與信心
歡呼音訊量測	分段	audio_signals.json	一個片段的歡呼信心與強度，以及逐窗口的歡呼分數
精彩片段排序	歡呼音訊量測	highlights.json	一個片段的精彩度分數
球員身分對應	球種分類、比分辨識；選用：骨架追蹤	identity.json	兩次換邊之間，上下方球員各對應計分板上的哪位選手，以及判定依據
賽評生成	分段、擊球時刻偵測、球種分類、比分辨識、身分對應；選用：精彩片段排序、骨架追蹤、場地偵測、球體追蹤	commentary.json	一個片段的回合賽評與逐拍賽評

3.2 影片前處理

3.2.1 賽事影片分段

概述

問題與目標：一支羽球轉播影片中，適合進行球員、羽球軌跡分析的畫面，主要來自固定俯拍的機位。若系統直接分析整部影片，非固定機位比賽畫面會產生大量錯誤資料，也浪費運算資源。本模組從轉播影片中找出固定機位比賽畫面，作為後續模組的輸入。

設計重點：本模組不需額外訓練模型、標註資料，也不需針對每支影片逐片人工調整參數，依據影片的統計結構自適應於不同影片。核心設計是把「固定機位比賽畫面主要只有球員移動」這個先驗，轉化為影格差異訊號。固定機位比賽片段的相鄰影格差異小、運鏡與鏡頭切換差異大、靜止的計分板差異極小，因此只用一個廉價的差異指標（MAD）就能把固定機位比賽片段與其他畫面在數值上分開。另一個核心設計是利用「同一機位在不同片段間具

有高度相似性」，排除計分板、重播及其他靜止畫面。在 9 場未參與開發的比賽上，片段偵測 F1 達 98.66%（見 4.2.1 節）。

處理流程總覽：如圖 3.2。

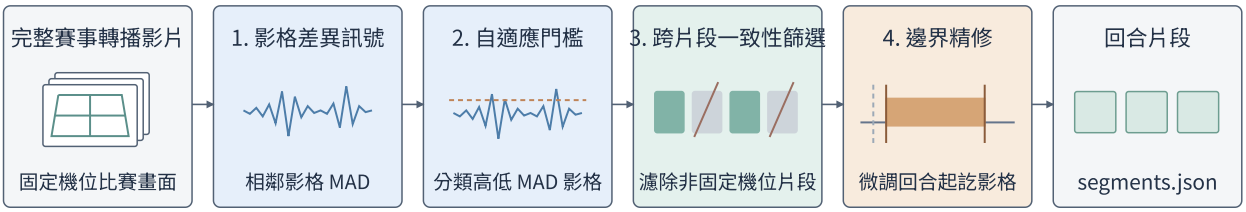


圖 3.2: 賽事影片分段處理流程

影格差異訊號

固定機位比賽片段畫面中只有兩名球員與球在動，兩影格間的差異小。其餘片段則落在兩個極端，鏡頭移動與切換使差異極大，靜止的計分板與定格畫面則差異極小。本模組以影格與影格之間的差異作為核心訊號。差異以 MAD（平均絕對差，Mean Absolute Difference）衡量：

$$\mathrm{MAD} = \frac{1}{MN}\sum_{x=1}^{M}\sum_{y=1}^{N}|I_1(x,y) - I_2(x,y)| \tag{3.1}$$

其中 M、N 為影像寬高，$I(x,y)$ 為座標 (x,y) 的灰階值。選用 MAD 的理由是成本低廉、對離群值不敏感、且動態範圍足夠。計算前先將影片縮放至 480p 以減少計算量。MAD 不隨解析度改變，相對高低仍然成立。

取樣採每隔三個影格一次，並與前一個取樣影格計算 MAD。間隔取樣一方面縮短處理時間，一方面避免特寫片段和固定機位比賽片段混淆。特寫中球員動作緩慢、背景固定，MAD 偏低且接近固定機位比賽片段，間隔取樣把緩慢位移放大、拉高其 MAD。固定機位比賽片段中球員佔比小、動作劇烈，間隔取樣影響有限。兩類片段因此在 MAD 上分開。

圖 3.3 中綠色區塊為手動標記的固定機位比賽片段，虛線為系統算出的門檻，可見綠色區塊的 MAD 通常低於門檻。

自適應門檻

本階段統計全片的 MAD 分佈，設定一個門檻把影格分成低 MAD（固定機位比賽畫面、靜止畫面）與高 MAD（運鏡、鏡頭切換）。不同影片的分佈不同，門檻須隨影片自動調整。

分析前先對 MAD 取以 10 為底的對數。鏡頭切換到完全不同畫面時每個像素都不一樣，會產生極大 MAD，使分佈右側出現長尾。取對數可壓縮長尾，使高低兩側尺度相當。實作取 $\log_{10}(\max(\mathrm{MAD}, 1))$，因為 MAD 可能為 0，需要地板值 1。

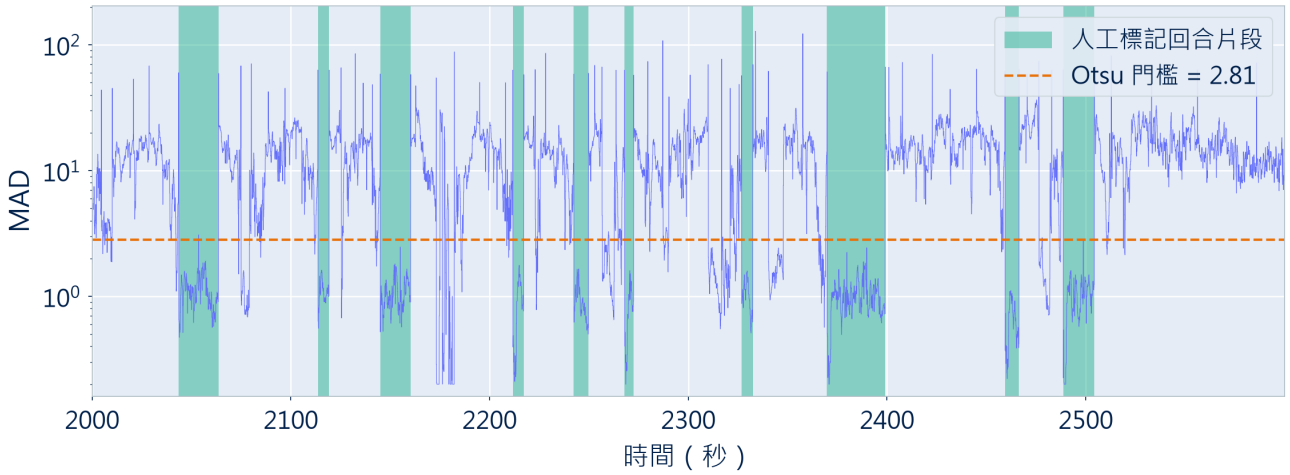


圖 3.3: 部分片段 MAD

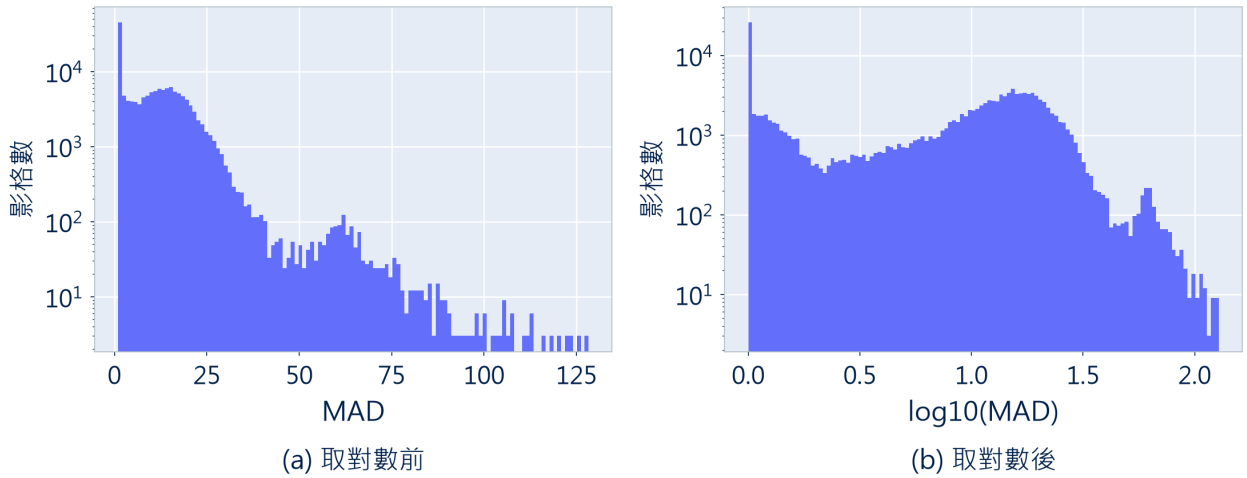


圖 3.4: 取對數

乾淨片段偵測：若輸入本身無鏡頭切換，重尾程度會遠低於多鏡頭影片，據此判定乾淨固定機位片段，跳過後續步驟直接輸出。

門檻方法：設門檻的問題與影像二值化相似，二值化統計像素亮度並切成兩類，本階段統計影格 MAD 並切成兩類，故沿用二值化的門檻計算方法。本系統採 Otsu 法，它只要求兩類均值不同即可分離，不限制分佈形狀、不需調參，正好適用於本階段低 MAD 側有尖峰、高 MAD 側是不對稱長尾的情形。

表 3.2: $\log_{10}(\mathrm{MAD})$ 的分佈與組成

MAD 區間	佔全片比例	固定機位比賽比例	畫面性質
0–0.5	6.8%	28%	近乎靜止
0.5–2	23%	75%	固定機位比賽畫面
2–2.75	2.3%	5%	谷底，門檻落點
2.75–30	65%	約 0%	重播、運鏡
30 以上	2.8%	7%	快速轉場

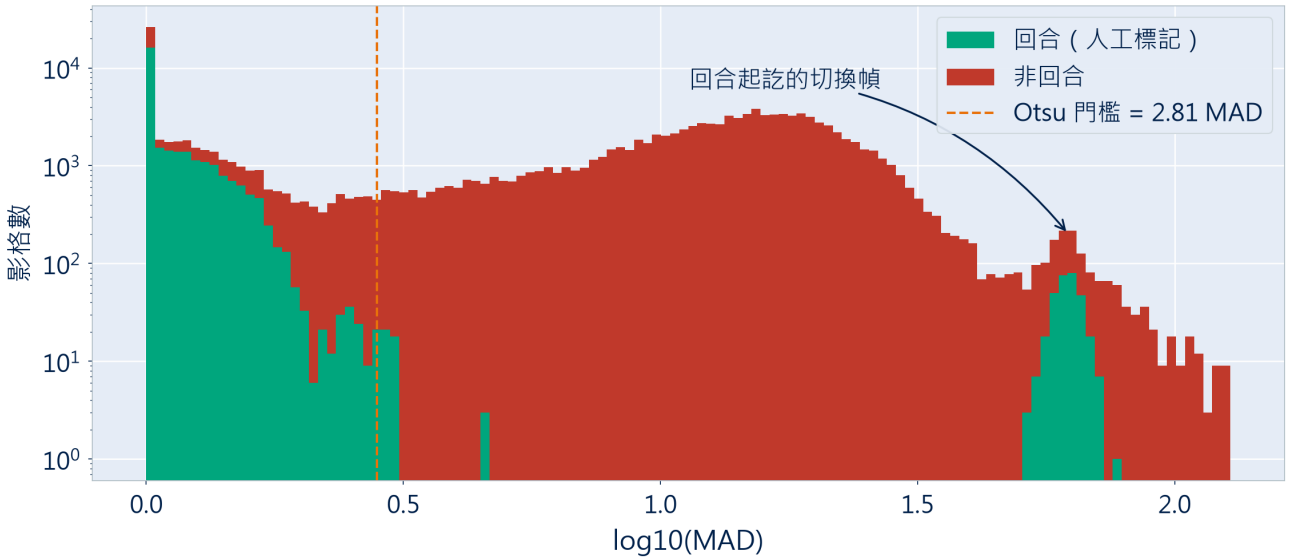


圖 3.5: 固定機位比賽片段與其他片段的 MAD 分佈

三類切分：直接做兩類 Otsu 會產生系統性偏差，高 MAD 側佔全片 68% 且橫跨 2.75 至 128，類內變異遠大於低 MAD 側，會把切點從谷底推進高 MAD 團內部，讓大量非固定機位比賽影格進入下一步。改用三類切分，上切點（10.80）吸收長尾壓力，下切點（2.81）回到谷底，取下切點為門檻（圖 3.6）。三類 Otsu 的效果等價於先移除長尾再做兩類 Otsu。

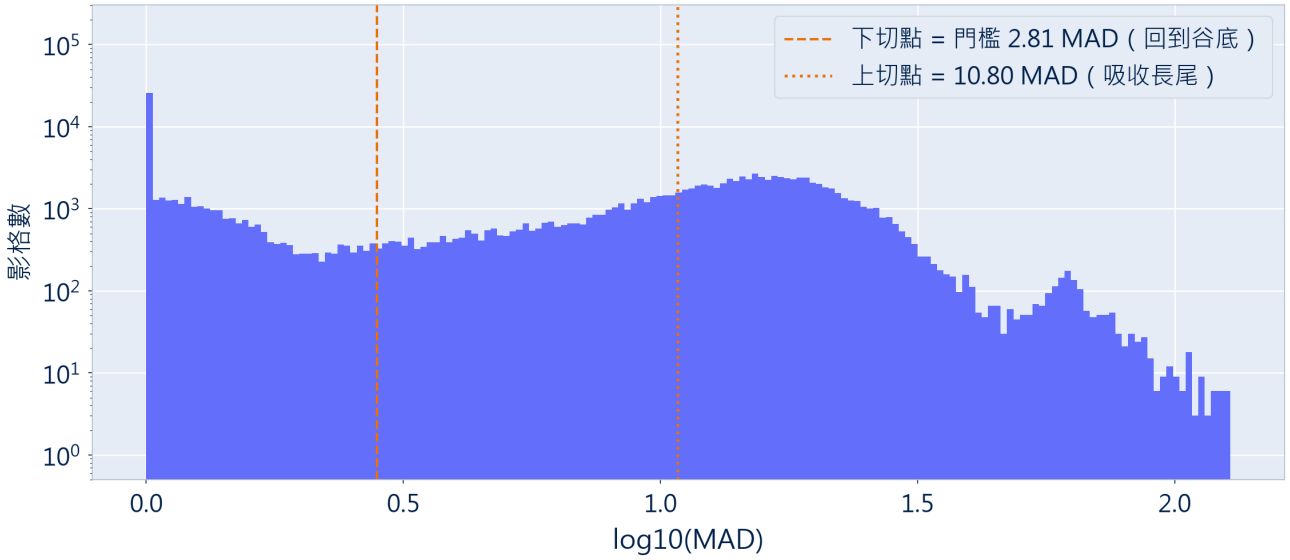


圖 3.6: 三類 Otsu 的兩個切點

跨片段一致性篩選

將連續低 MAD 影格串接成片段，此時保留下來的片段中仍混有計分板、定格等靜止片段。關鍵觀察是，此時剩餘的固定機位比賽片段數量多，且全部來自同一個固定機位、彼此在畫面上高度相似。其他片段則各自不同，與任何片段都不像。因此可以透過片段之間相互比較，判斷一個片段是否屬於佔多數的固定機位比賽片段。

設共有 N 個片段，片段 i 取其中央兩張影格，各自縮放至 128×72 並展開為特徵向量 $v_i^{(1)}, v_i^{(2)}$。兩片段的差異 $d(i,j)$ 取四組跨影格 MAD 的平均，跨片段平均差異 $\bar{d}(i)$ 則是片段 i 對其餘 $N-1$ 段的平均：

$$d(i,j) = \frac{1}{4}\sum_{a=1}^{2}\sum_{b=1}^{2} \mathrm{MAD}\left(v_i^{(a)},\, v_j^{(b)}\right), \qquad \bar{d}(i) = \frac{1}{N-1}\sum_{j \neq i} d(i,\, j) \tag{3.2}$$

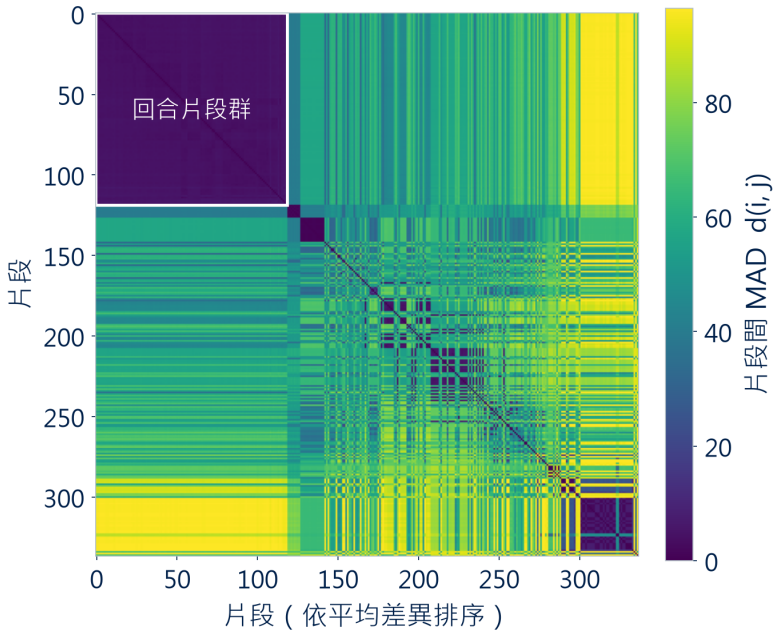


圖 3.7: 跨片段兩兩 MAD 矩陣（依平均差異排序）

因為所有固定機位比賽片段高度相似，它的 $\bar{d}(i)$ 幾乎只由少數幾個非固定機位比賽片段貢獻，數值因而偏低。又因固定機位比賽片段彼此相像，算出的 $\bar{d}(i)$ 也幾乎相等，於是它們在直方圖最左側聚成一個緊密的群。其他片段畫面各異，$\bar{d}(i)$ 偏高且分散。圖 3.7 將片段依 $\bar{d}(i)$ 排序，左上角的紫色密集區即這群固定機位比賽片段。

實測固定機位比賽片段間 $\bar{d}(i)$ 的極差在 5% 以內，最小的非固定機位比賽片段比全體最小值大 13%，門檻取全體最小值的 1.07 倍（圖 3.8）。此門檻成立的前提是固定機位比賽片段屬於最緊密的群，若影片含大量彼此相似的畫面，會形成新的低 $\bar{d}(i)$ 群使門檻失效。

通過篩選的片段中偶爾仍殘留鷹眼判定、官方重播等電腦合成畫面，其構圖與固定機位比賽畫面接近。因此對剩餘片段再算一次跨片段平均差異，採較嚴格的門檻，捨棄超過中位數三倍者。

邊界精修

此階段把被少數高 MAD 影格（分數卡片動畫）切碎的片段接回。同時第一階段的跳幀取樣使邊界存在系統性偏差，需還原到影格層級。

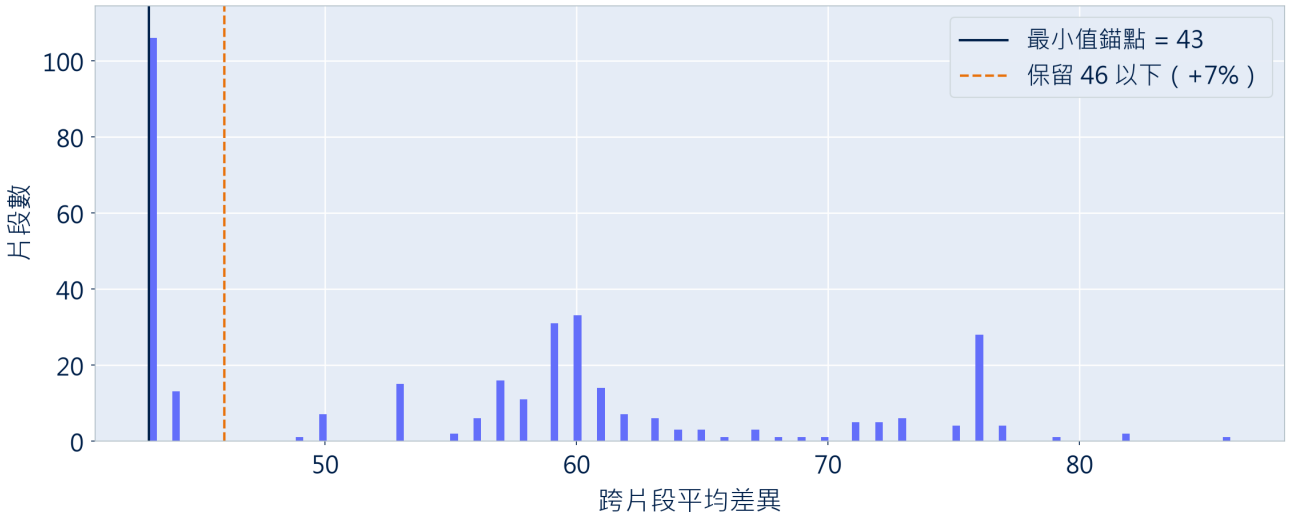


圖 3.8: 跨片段平均差異的分佈與門檻

做兩次片段連接：第一次只看距離，間距小於 5 張影格的相鄰片段直接接合。第二次針對 12 張影格以內的間隙逐影格重新掃描，僅當間隙內最大相鄰 MAD 低於 25（即不存在尖峰）時才接合。

做邊界對齊：在每個邊界附近逐影格計算相鄰 MAD，尋找同時滿足絕對值門檻與區域中位數倍率的尖峰，此尖峰對應剪接點，起點設為切入尖峰所在的影格，終點設為切出尖峰的前一張影格。

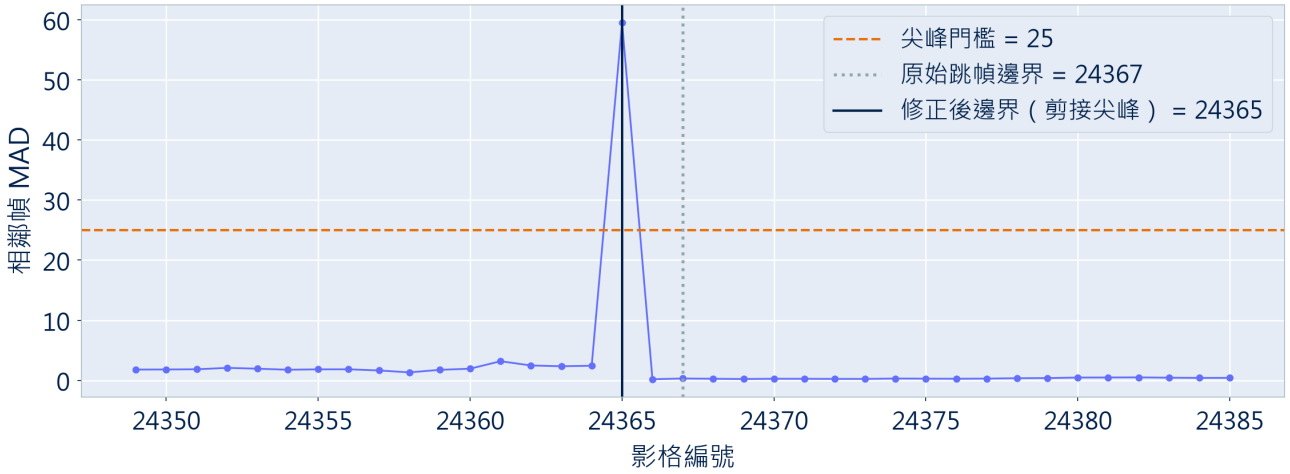


圖 3.9: 邊界尖峰偵測與修正

圖 3.9 中灰色點線為原始跳幀取樣邊界，深藍實線為修正後的邊界，橘色虛線為尖峰門檻。

3.2.2 比分辨識

概述

問題與目標：賽事影片分段模組切出的片段中，同一回合可能因中間插入重播、特寫而被切成數段，一個片段也可能因為沒有鏡頭切換而包含多個回合。本模組為每個片段讀出畫面上的計分板比分，判斷片段所屬的回合，該比分同時供下游精彩片段分析使用。

設計重點：讀計分板天生受三個困難制約：泛化性（各賽事的計分板設計、位置不固定，且須避開干擾文字）、可讀性（計分板小又模糊）、多局制（三局制計分板易讀錯局）。本模組不需訓練模型或逐賽事調參，以「把難題拆成模型擅長的小任務、再用程式補上模型不擅長的部分」為核心，有三點做法：

1. **使用視覺語言模型（VLM）：**目前使用傳統 OCR 的開源專案都需要手動標記計分板的位置。VLM 直接理解畫面結構，依視覺上下文自行分辨比分。不用為每種計分板寫過濾規則或手動標記，提升跨賽事的泛化能力。
2. **多幀合成：**利用逐像素合成，強化計分板、糊化背景和球員。在 overlay 只出現於三成影格時仍能完整還原。
3. **提示詞設計：**轉錄取代挑選，令模型由左到右完整抄寫計分板上所有數字（模型擅長），把當前局比分的挑選（模型易錯）移到程式端，讓較便宜的模型也能快速、正確的完成任務。

處理流程總覽：如圖 3.10。

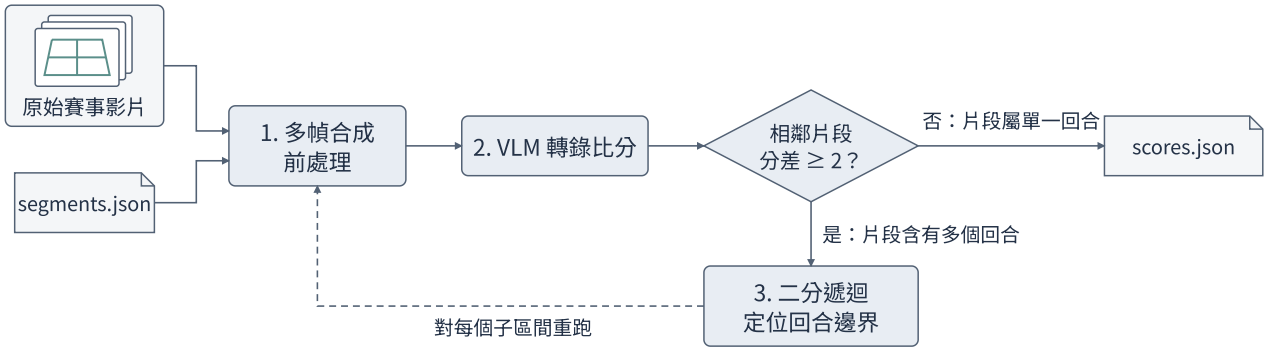


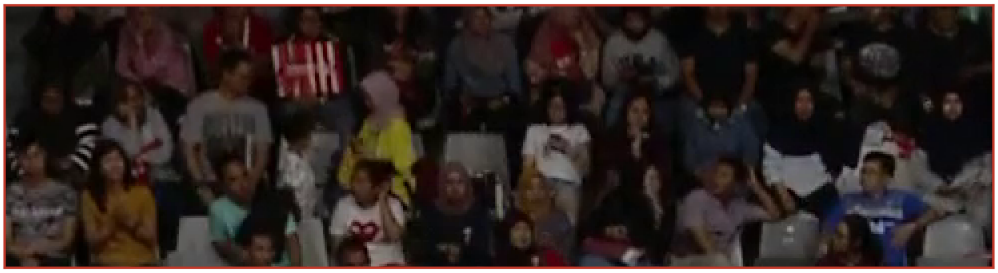
圖 3.10: 比分辨識處理流程

多幀合成前處理

本步驟為每個片段產生一張合成圖，讓 VLM 讀一張圖就拿到比分。轉播的計分板 overlay 只在片段的部分影格出現，若逐張送 VLM 去找，每個片段都得呼叫多次。一個片段通常同

屬一回合，比分不變、計分板靜態，因此把片段取樣的多張影格逐像素合成，間歇出現的計分板被疊加強化、穩定呈現在合成圖上，一個片段只需一次 API 呼叫。不斷移動的球員與觀眾被平均糊化，順帶為 VLM 濾去干擾。

合成採 `dominant_cluster`（主群集平均）：對每個像素的亮度值分箱，只平均落入最多值的那一箱。overlay 出現時亮度幾乎固定，會全擠進同一箱。背景每幀不同而分散各箱。因此最密集的一簇即為計分板（圖 3.11）。這個方法不預設目標信號在時間上佔多數，即使轉播 overlay 只間歇出現仍能保留。



(a) 抽幀：沒有計分板（37 張中有 10 張如此）

(b) 抽幀：overlay 與背景都清晰

(c) 主群集平均：overlay 保留，背景糊化

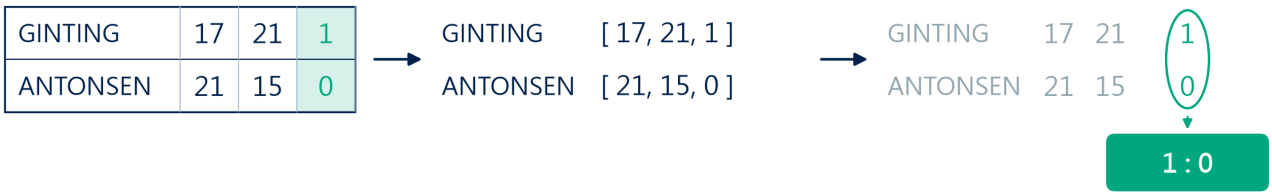
(d) 取樣的 37 張影格，其中 27 張有 overlay（綠為有、紅為無）

圖 3.11: overlay 只出現在部分影格時，主群集平均的還原結果

VLM 轉錄比分

本步驟從合成圖讀出雙方當前比分。多局制計分板每隊並列多局數字，要模型直接判斷「哪個是當前局」容易錯，完整抄寫數字對模型來說不需要推理，比較簡單。因此只要求模型逐隊由左到右抄下所有數字，挑選交給程式，固定取每隊陣列最右一個為當前比分，從根源消除模型讀錯局的可能（圖 3.12）。

實作上使用 Gemini 2.5 Flash，將 temperature 設為 0，並且關閉 thinking，減少結果變化的同時避免模型進行不必要的推理，多餘的推理浪費 token、時間又容易出錯。



(a) 計分板　(b) VLM 完整轉錄　(c) 程式取每隊最右

圖 3.12: 轉錄取代挑選，VLM 完整抄寫每隊數字，程式取最右一個為當前比分

還原被合併的回合

分段以鏡頭切換為訊號，兩回合間若沒有切換就會被併進同一片段，此時計分板在片段中途變動，單張合成只讀得到其中一個回合。本步驟負責找出並拆開這類片段。

判斷靠比分本身，正常一回合雙方總分只加一分，因此若某片段雙方比分總和一次增加兩分以上，就代表中間有回合沒被單獨切出，標記為合併片段。

標記後以二分搜尋定出回合交界，在候選區間中點重新合成、辨識比分，與兩端比較，往比分仍有跳變的一側收斂，逼近比分改變的影格。由於一個片段可能併入不只兩回合，每定出一個切點就對兩側子區間各自遞迴、繼續二分，直到每個子區間都只差一回合。

3.3 場景與人體理解

3.3.1 場地偵測

概述

問題與目標：後續所有球員與球體分析需要以球場座標來解讀位置與距離。本模組從固定俯拍的轉播畫面中定出球場，建立影像平面與球場平面投影的座標基準，供後續步驟使用。

設計重點：本模組不需訓練模型、不需標註，核心是把球場偵測化約為「在畫面中找四個外角」的問題，球場是平面、尺寸由競賽規則規定，定出四角即可解出單應性，其餘 12 條線與 16 個交點全由它投影得出。本模組的初始版本參考了一份以 OpenCV 實作的開源羽球場地偵測程式 [16]，現行版本在其基礎上重新設計，將顏色寫死的處理改為顏色無關的表面分割，並以消失點過濾與網格枚舉取代原本 $O(n^4)$ 的四邊線窮舉。在此基礎上有三點做法：

1. **零先驗的顏色自適應：**球場顏色因場館而異，系統不預設任何色彩範圍，參考色與門檻全從畫面即時估計。相較多數開源方案需固定顏色範圍或手動標記，本法無需逐場調參。

2. **多來源並聯、統一吻合度評分裁決：**不押注單一路徑，兩種遮罩、三種外角來源並行推導，最後由同一個「場線像素吻合度」的評分選出最佳者。遮罩斷裂這類錯誤不可逆，並聯確保唯一的路不會走死。

3. **以最可靠的內部線反推最不可靠的外角：**球場外緣線透視壓縮最重、最常被遮斷，最難完整偵測。網格枚舉改由遮擋少的內部發球線與中線一起投票決定四角，把證據取用的方向倒轉過來。

在 9 場未參與開發的比賽上，角點定位誤差中位數為 2.79 像素（見 4.2.2 節）。

處理流程總覽：如圖 3.13。

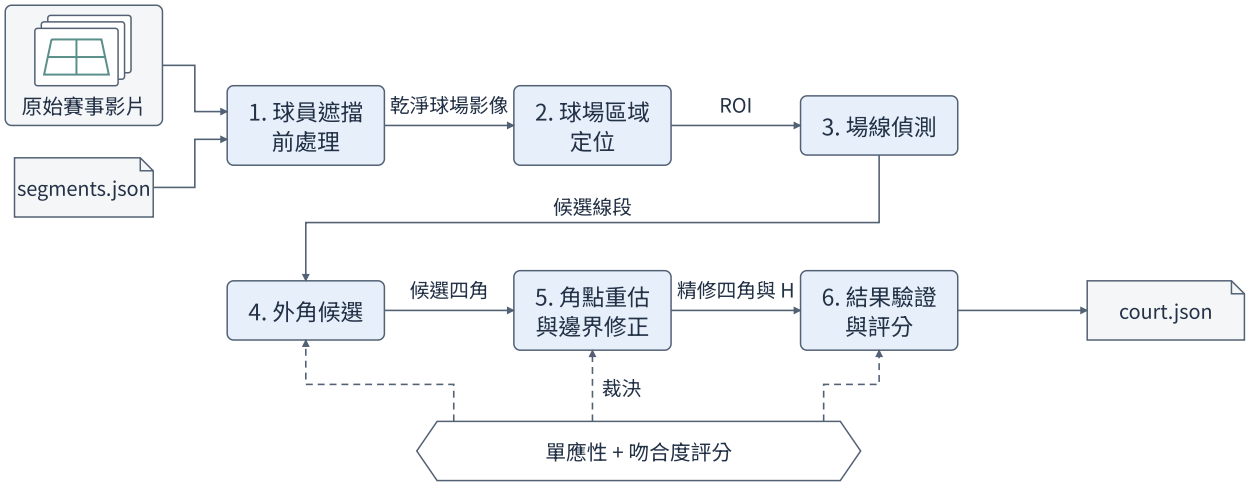
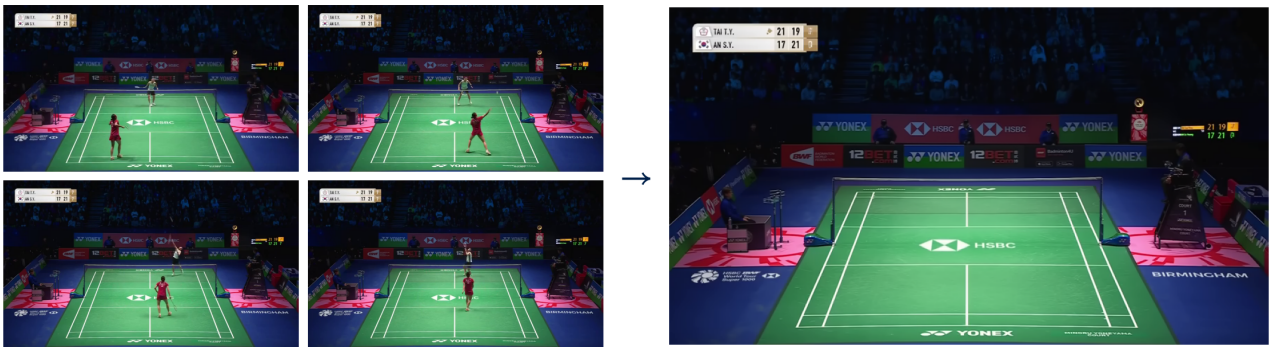


圖 3.13: 場地偵測處理流程

球員遮擋前處理

後續定位球場的主要訊號是顏色與場線，但轉播畫面中球員持續移動遮住場線與色塊，單張影格會使線條斷裂。本步驟要產生一張不含球員與短時雜訊的乾淨球場影像。機位固定時，球場在每張影格的同一座標都呈現相同顏色，唯有球員經過的少數影格例外，故對每個座標取多影格像素值的中位數，佔多數的球場顏色留下、屬於少數的球員與雜訊被濾除（圖 3.14）：

$$M(x, y) = \text{median}\{I_1(x, y), \ldots, I_K(x, y)\} \tag{3.3}$$

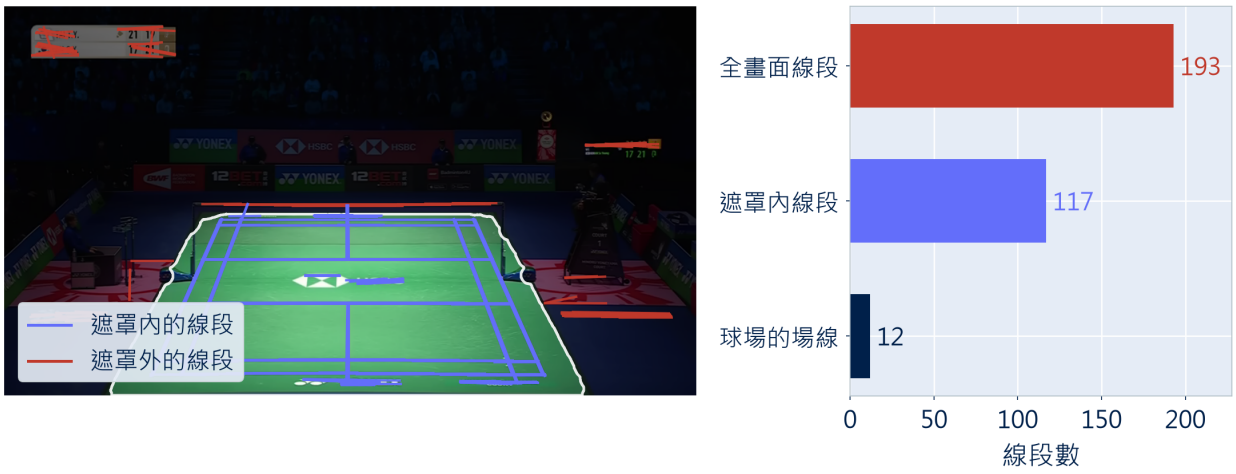


(a) 同一批回合中的四張抽樣影格　　(b) 360 張影格的逐像素中位數

圖 3.14: 四張取樣影格與合成圖

球場區域定位

本步驟要圈出球場在畫面中的大概範圍，好把範圍外的線段擋在後續場線偵測之外。實測對整張畫面掃描會得到近 200 條線段，真正屬於球場的只有 12 條（圖 3.15）。



(a) 全畫面掃描到的線段（白線為球場遮罩）　　(b) 三種範圍內的線段數

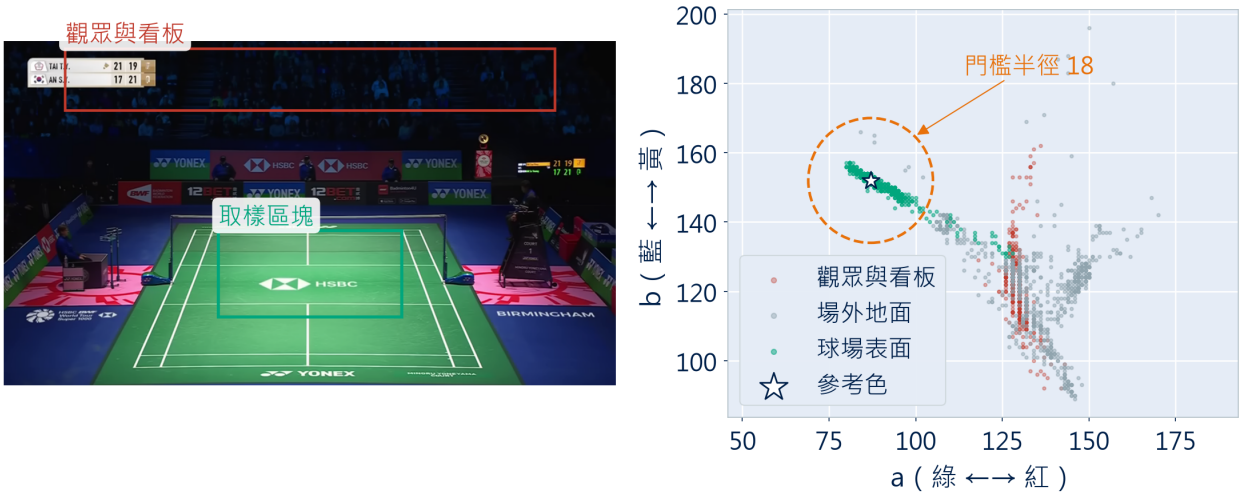
圖 3.15: 全畫面掃描到的線段，與落在球場遮罩內的部分

分離球場與周邊地板的明顯特徵是顏色，但球場顏色本身未知。做法是在畫面中央偏下取一塊由機位假設保證落在場內的區域 S，取其像素在 Lab 色度（a、b）通道的中位數作為球場參考色 $\mathbf{c}$，任一像素的色度與 $\mathbf{c}$ 的距離小於門檻 τ_c 即判為球場：

$$\mathbf{c} = \left(\operatorname{median}_S a,\ \operatorname{median}_S b\right), \qquad \tau_c = \max\left(18,\ 3 \cdot \tfrac{\sigma_a + \sigma_b}{2}\right) \tag{3.4}$$

$$C(x,y) = \left[\, \| \left(a(x,y),\, b(x,y)\right) - \mathbf{c} \| < \tau_c \,\right] \tag{3.5}$$

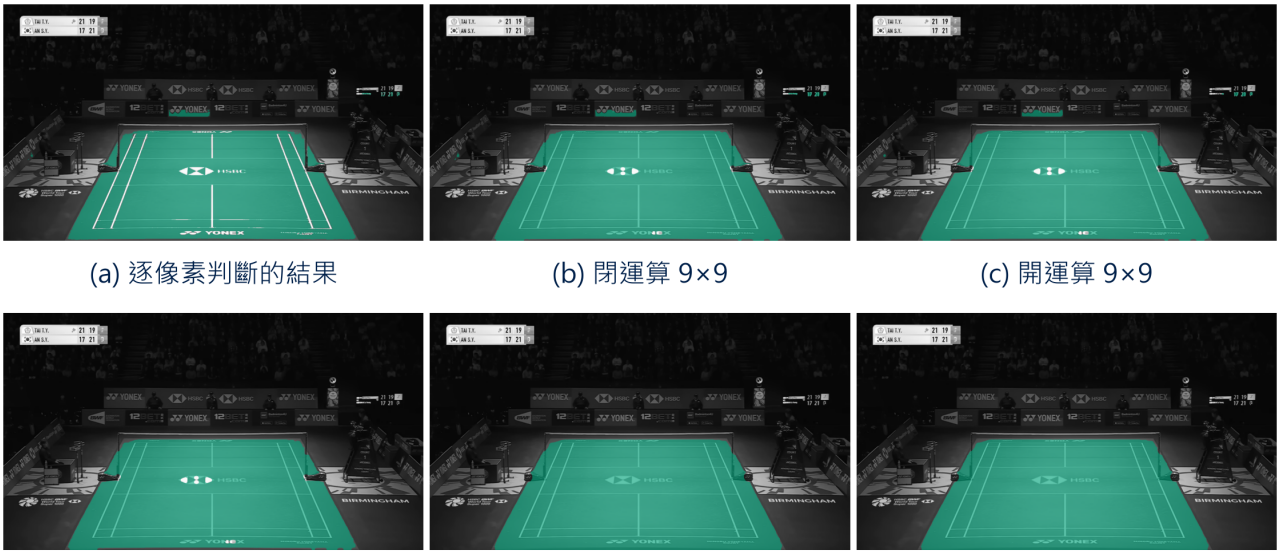
其中 σ_a, σ_b 為 S 在兩色度通道的標準差。捨棄亮度通道使光照與陰影不進入判斷。用中位數以避免取樣區塊內的像素影響門檻。門檻取 3σ 使其隨取樣塊自身的顏色離散程度放寬，下限 18 則用以容忍過於平滑的場地（圖 3.16）。



(a) 取樣區塊與對照的觀眾區域　　(b) 各區域像素在 ab 色度平面上的位置

圖 3.16: 取樣區塊、參考色與門檻的判定

上述門檻得到的是逐像素獨立的判斷，還不是一塊完整區域。白色場線把場地切成數塊，雜訊與反光留下零星破洞。模組以形態學運算補洞去雜點、取面積最大的連通區域、再自畫面左上角向內填充背景，把內部空洞併回球場（圖 3.17）。



(a) 逐像素判斷的結果　(b) 閉運算 9×9　(c) 開運算 9×9

(d) 最大連通區域　(e) 閉運算 25×25　(f) 自左上角填充背景、空洞併回

圖 3.17: 形態學清理、最大連通區域與空洞填補各階段的遮罩變化

本步驟實際產生的不只一張遮罩。除上述色度版本外，另備一個紋理版本，在色度條件之外再加一道局部亮度標準差門檻，只留下平滑區域，用以濾掉與場地同色卻花紋雜亂的地面贊助商圖樣（圖 3.18）。另有一個低色度版本，在整體畫面色度過低、色度判準失效時改以亮度相似度取代色度。這些版本不互相取代，而是一併往下送，構成後續多假設並行的二至三組遮罩。

(a) 場邊看板、牆面與球場同為紅色　(b) 只用色彩準則的遮罩　(c) 加上紋理閘門：紅色為被剔除者

圖 3.18: 場邊看板與球場同色時，紋理版本剔除色度版本誤納的區域

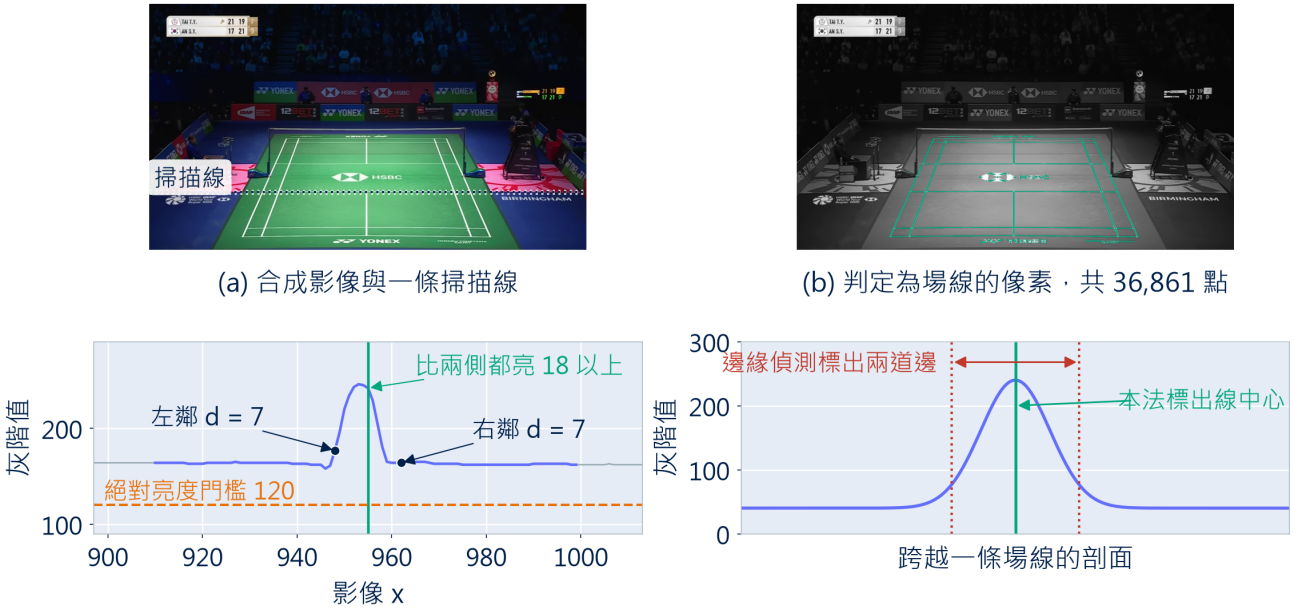
場線偵測

本步驟要在前一步的遮罩範圍內找出球場線，過程分四段，取出屬於線的像素、把像素組織成線段、濾除不屬於球場的線段、合併接近的線段。

取線像素不靠色彩，轉播素材的曝光與白平衡各異、場線的絕對色值並不固定。使用局部亮度突起，一個像素若比水平（或垂直）方向兩側都亮出門檻 τ，即屬近垂直（或近水平）的線像素，P_h、P_v 取聯集，比較距離 d 須大於線寬（圖 3.19）：

$$P_h(x,y) = \big[G(x,y) - G(x-d,y) > \tau\big] \wedge \big[G(x,y) - G(x+d,y) > \tau\big] \tag{3.6}$$

$$P_v(x,y) = \big[G(x,y) - G(x,y-d) > \tau\big] \wedge \big[G(x,y) - G(x,y+d) > \tau\big] \tag{3.7}$$

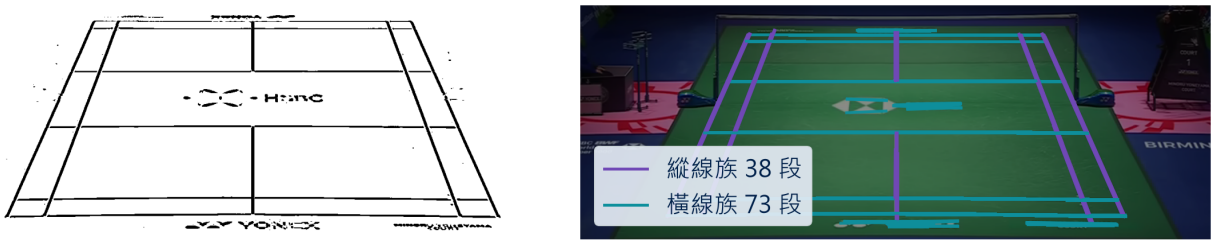


(a) 合成影像與一條掃描線　(b) 判定為場線的像素，共 36,861 點

(c) 掃描線上的亮度剖面　(d) 一條有寬度的白線，兩種取法

圖 3.19: 局部亮度突起與場線像素的萃取結果

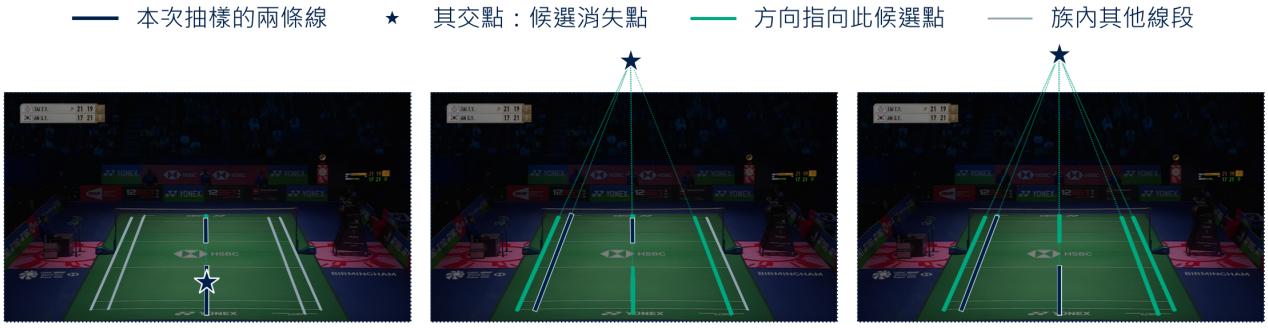
上一步的輸出是散落的像素，以機率霍夫轉換（PPHT，cv2.HoughLinesP）把共線者組織成線段（圖 3.20）。場線常因球員殘影、地面反光與遮罩侵蝕而斷開，霍夫轉換靠投票決定共線性、不依賴像素連通，能跨越這些缺口。



(a) 輸入：散落的場線像素　　(b) 抽出的 111 條線段，依傾角粗分兩族

圖 3.20: 霍夫線段與兩族的角度粗分

粗分後仍混有地面贊助商圖樣等非球場線段，以透視幾何篩選。球場的 12 條線分屬兩組平行線，在畫面上各自通過一個消失點。找消失點是離群比例可能很高的穩健估計問題，採 RANSAC，隨機取兩條線段、以交點為候選消失點，統計族內有多少線段方向指向它（以線段長度加權），重複數百次取支持度最高者（圖 3.21）。



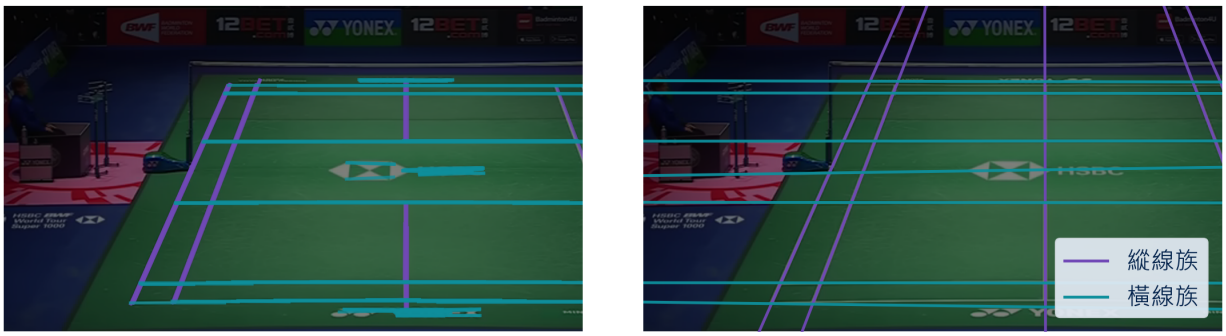
(a) 支持度低的候選
同意 7 條 · 支持度 949 px

(b) 支持度中等的候選
同意 29 條 · 支持度 9,941 px

(c) 支持度最高者，鎖定
同意 38 條 · 支持度 13,988 px

圖 3.21: 消失點 RANSAC 三次抽樣的候選點與其支持度

球網遮擋常把一條球場線拆成方向略有出入的數段，同一條物理線會以不同位置重複參與運算，故先合併。同族線段方向相近、只在垂直於該方向的位置上分開，取一條參考線把每條線段化為一維位置，位置相近者歸為一簇，每簇以最小平方擬合出一條代表線。數百條線段因此收斂成十餘條物理線，既去除重複，也把後續窮舉的組合數壓到可行範圍（圖 3.22）。



(a) 合併前：此局部有 84 條破碎線段　　(b) 合併後：全畫面收斂為 5 條縱線、7 條橫線

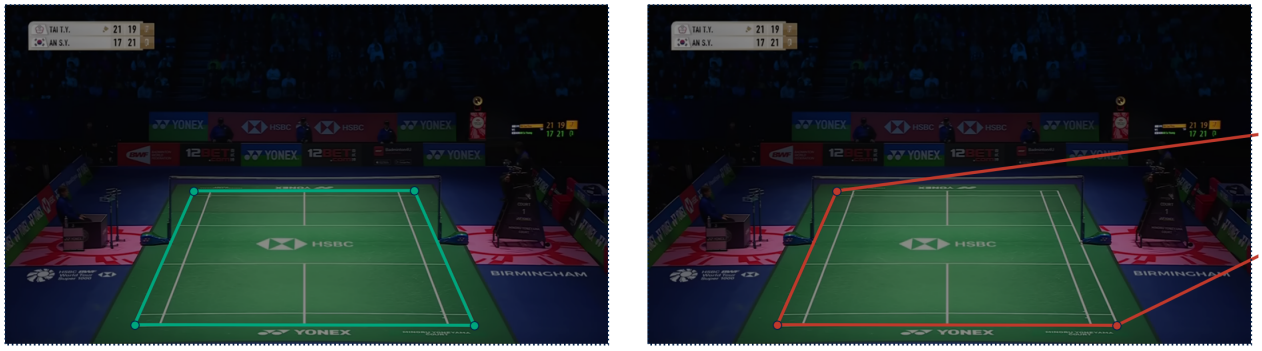
圖 3.22: 同族線段合併的前後對比

單應性與吻合度評分

本步驟解決「如何由四角解出單應性、如何量化一組四角的好壞」的問題。

羽球場的尺寸與 12 條場線位置皆為已知常數，因此可預先建立球場模型座標：5 條縱線對應固定的 x 值，7 條橫線對應固定的 y 值。將模型四角與影像中偵測到的四角建立對應後，即可用 DLT 解出單應性 H，再由 H 將模型中的 12 條場線與 16 個交點一併投影至影像。

DLT 對趨近退化的輸入不會報錯，仍會得出病態的 H，故對每個 H 加設合法性檢查：四角須為有限值、不超出畫面、不退化成細條、近端底線不短於遠端（圖 3.23）。此檢查貫穿後續所有產生 H 的階段。



(a) 通過（虛線為畫面邊界）　(b) 過度超出畫面

(c) 退化成細條　(d) 遠端反而較長

圖 3.23: 合法性檢查

吻合度評分衡量 H 推出的網格與畫面上實際偵測到的場線像素有多吻合。單條線的覆蓋率定義為：沿線等距取若干點、投影到影像，落在最近場線像素容差（4 像素）內的比例（圖 3.24）。總評分取「貼合度」與「廣度」兩項相乘：

$$S = \underbrace{\frac{1}{12}\sum_{k=1}^{12} \mathrm{cov}_k}_{\text{貼合度}} \times \underbrace{\frac{1}{12}\left|\{\, k : \mathrm{cov}_k > 0.3 \,\}\right|}_{\text{廣度}} \tag{3.8}$$

此評分是整個場地偵測判斷好壞的統一依據，後續的候選選擇與邊界修正都以它裁決。

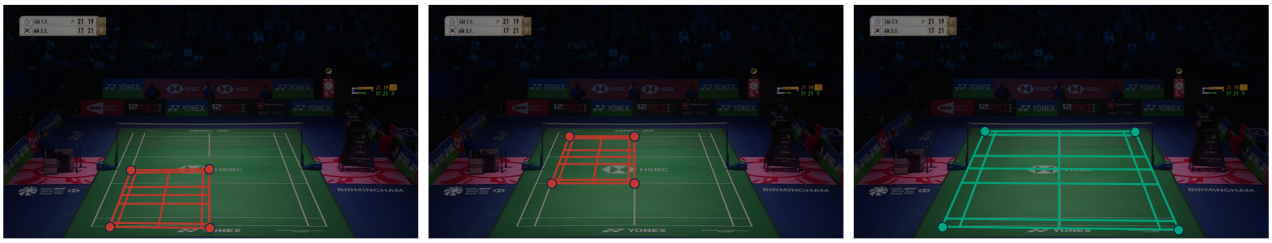
(a) 場線像素　(b) 距離轉換：每個座標到最近場線像素的距離

圖 3.24: 場線像素與其距離轉換圖

外角候選

四角須由偵測到的線或遮罩推導，任一種推導都有其失效條件，故本模組並行三種來源、各給一組候選，三者的失效條件互不重疊。

網格枚舉（主要方法）：從候選縱線任取兩條作左右邊界、橫線任取兩條作上下邊界，四線相交得一組四角，枚舉所有 $\binom{10}{2} \times \binom{12}{2} = 2970$ 組合，取 12 線網格吻合度最高者（圖 3.25）。穩健的關鍵在於以最可靠的內部線反推最不可靠的外角，等於讓發球線與中線一起投票決定四角。



(a) S = 0.065　(b) S = 0.180　(c) S = 0.516

圖 3.25: 網格枚舉的候選與其吻合度評分

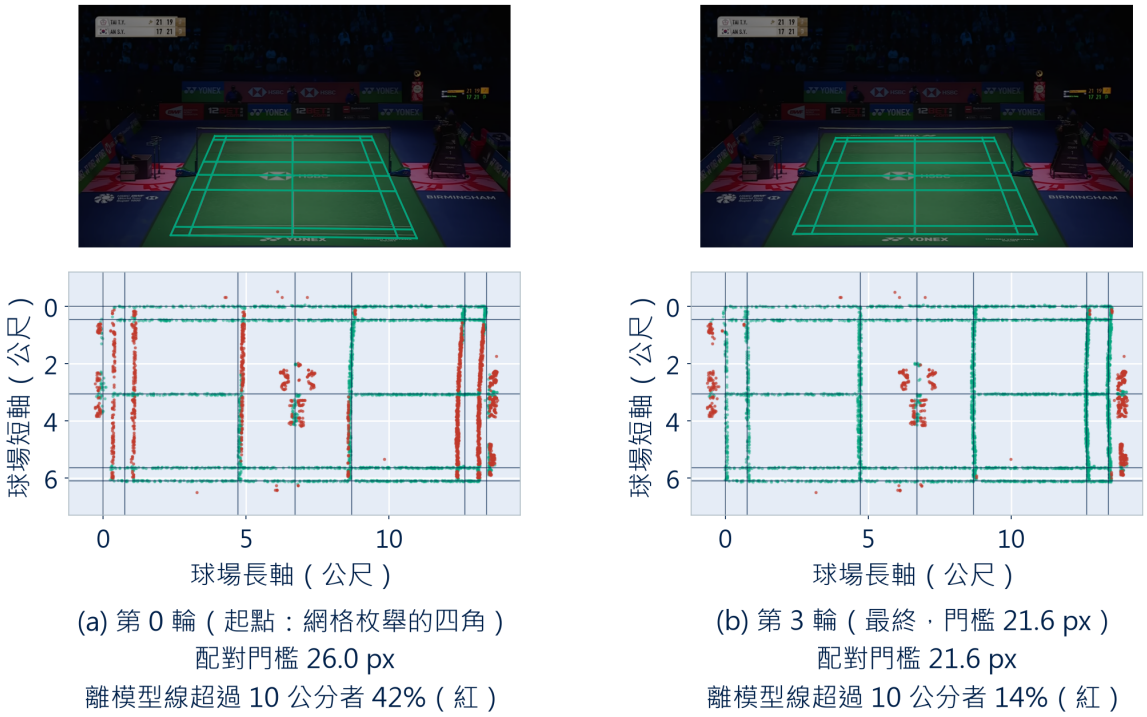
極端線相交：在兩族中各取最外側的兩條線相交得四角，最直接，但要求那四條極端線全部正確，而外緣線最難偵測，故僅作第二來源。它的失效條件與網格枚舉不同，能在枚舉搜尋空間無正解時補位。

遮罩凸包：改用遮罩形狀，取面積最大的外輪廓、求凸包並簡化為多邊形，再由頂點座標的極值取出四角。它不需任何場線，但遮罩邊界經形態學處理與空洞填補、未必等於球場外緣，精度最低，僅在前兩者都產生不出合法 H 時作為備援。

角點重估與邊界修正

前一步的四角只由少數代表線相交而得，影像中大量場線像素尚未參與，本步驟用上全部線像素把結果調準，分兩階段。

以 ICP 重估單應性：難點在建立線像素與球場模型的對應，影像中場線因透視而方向各異，難以直接配對。利用前一步的近似 H，把每個線像素反投影回球場座標，那裡的縱線是 x 為定值、橫線是 y 為定值的直線，量出該像素離哪條最近即建立對應。再用全部點對重解 H、以新 H 更新對應，反覆逼近。點對數以千計，採 RANSAC 版本把配錯到相鄰線的像素當離群值排除，對應的距離門檻隨迭代由寬到窄（圖 3.26）。



(a) 第 0 輪（起點：網格枚舉的四角）
配對門檻 26.0 px
離模型線超過 10 公分者 42%（紅）

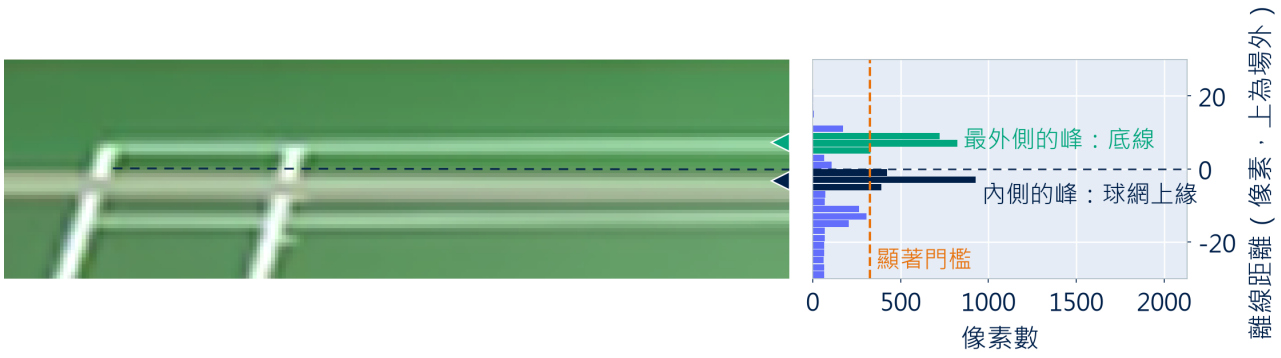
(b) 第 3 輪（最終，門檻 21.6 px）
配對門檻 21.6 px
離模型線超過 10 公分者 14%（紅）

圖 3.26: ICP 第一輪與最終輪的網格與反投影殘差

邊界修正：透視壓縮下球場遠端的內、外邊界在畫面上靠得很近，全域解無從分辨，外邊界可能整條鎖到內側線上。本步驟只動四條外邊界，把某條投影線附近的像素依「離線多遠」投到一維直方圖，每條平行線形成一個峰，二維的找線因此降為一維的找顯著峰（圖 3.27）。遠端刻意偏取最外側的峰重新擬合邊界，因為兩種誤差不對稱，外邊界略往場外時場內物體的相對位置仍成立、下游影響有限，一旦鎖進內側則截斷場內內容、不可逆。搜尋帶寬由寬到窄逐級縮小。這是帶方向的推測，故結果不直接採用，而是回到吻合度評分重算，只有分數未明顯變差才取代原解（圖 3.28）。

結果驗證與評分

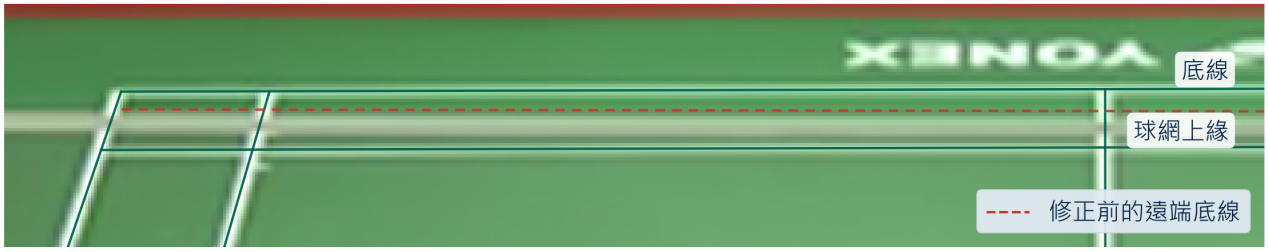
前述步驟以多假設並行：球場區域定位產生二至三組遮罩，場線偵測再對每組遮罩各給三種外角候選，故候選池大小為遮罩組數乘以三，每個候選都帶著它所依據的線像素圖。本步驟把這一池候選收斂成唯一答案，每個候選經角點重估與邊界修正後，在自己的線像素圖上算出吻合度評分，全部比較之下取最高者，其四角與單應性即為模組的最終輸出（圖 3.29）。



(a) 遠端左角：底線的投影線（虛線）與其附近的平行線　(b) 離線距離直方圖：一個峰即一條平行線

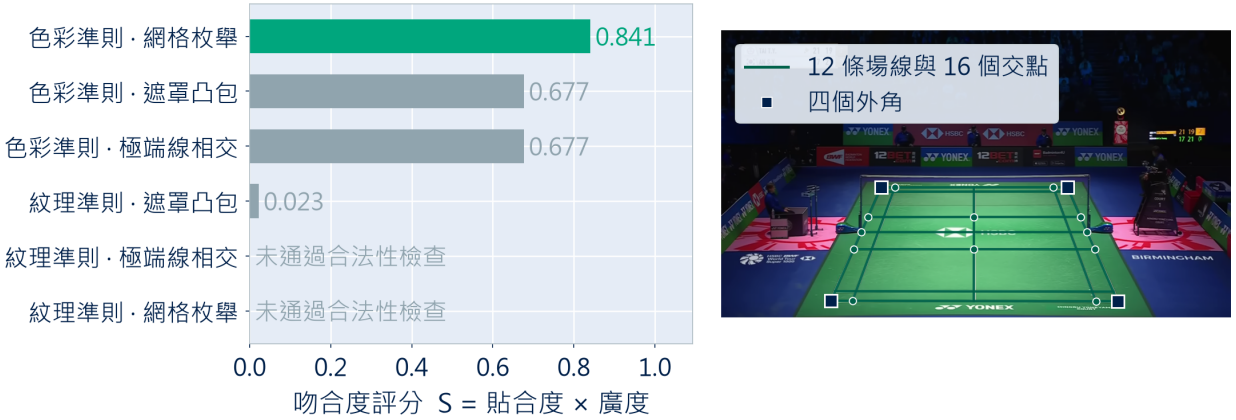
圖 3.27: 投影線的鄰近像素與其離線距離直方圖

(a) 修正前：遠端底線鎖在內側的球網上緣（S = 0.659）



(b) 修正後：改取最外側的峰（S = 0.804）

圖 3.28: 遠端底線由內側峰改取最外側峰的前後對比



(a) 候選池　(b) 勝出者（色彩準則 · 網格枚舉 · S = 0.841）的投影

圖 3.29: 候選池的吻合度評分與勝出者的投影

3.3.2 球員骨架追蹤

概述

問題與目標：球種分類需要球員的肢體姿態，本模組為每個片段還原場上兩名球員逐幀的骨架，作為下游球種分類的輸入。難點一是廣角轉播下球員在畫面裡常只有一兩百像素高，手肘、手腕等末端關節不易估準。難點二是畫面中除兩名球員外還有裁判、線審、球僮與觀眾，每個人都有骨架可抽，必須靠球場幾何從中認出真正的球員。

設計重點：本模組的核心不在姿態模型本身，而在如何從畫面中的眾多人體骨架穩定選出真正上場的兩名球員。利用球場幾何與跨幀行為自適應完成，主要有兩點做法：

1. **球場座標選人：**系統把每個候選人的腳點反投影到正規化球場座標，依所在半場判定遠、近端身分，再綜合場內位置與透視校正後的身高挑選球員。
2. **長短期時序資訊：**系統先從整場候選人的落點分布找出長時間不動的裁判、線審等靜態錨點並排除，再以短期位置記憶維持同一球員的連續身分。

處理流程總覽：如圖 3.30。

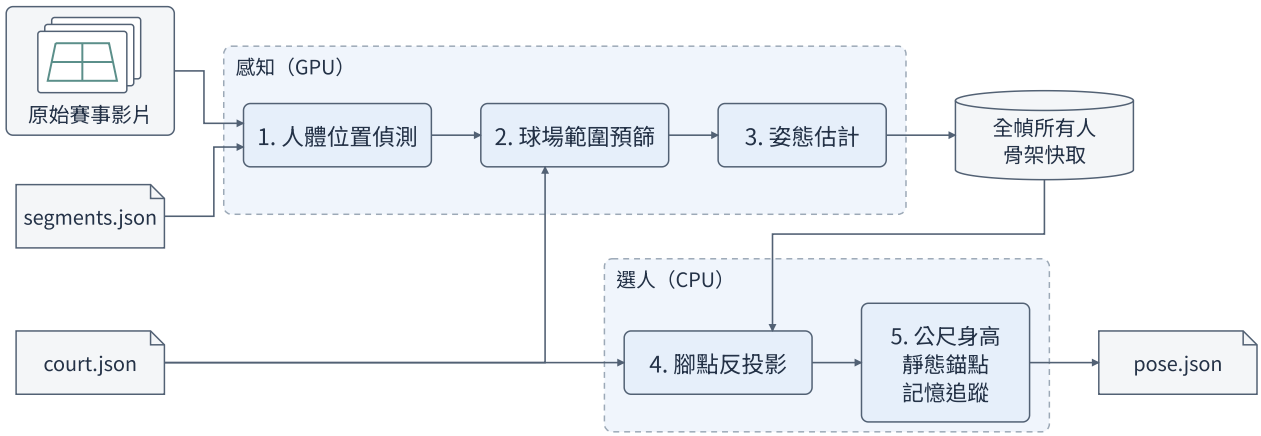
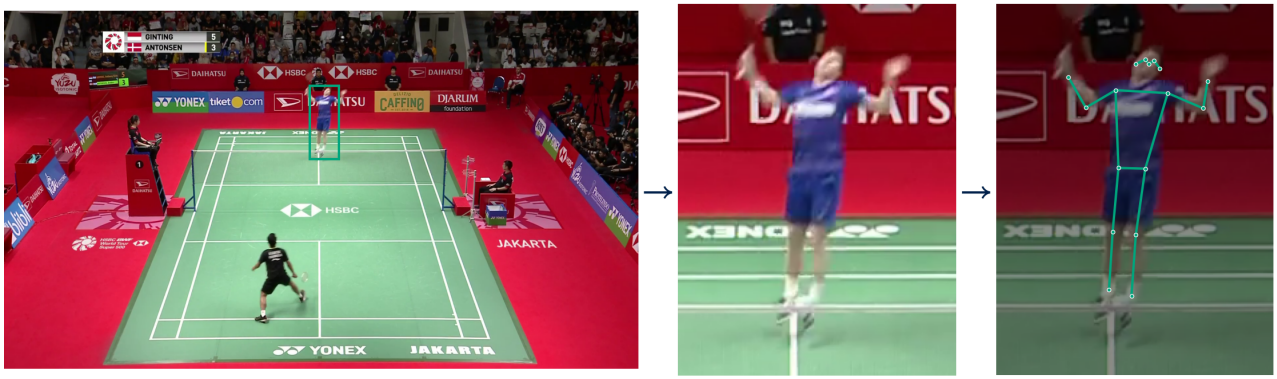


圖 3.30: 球員骨架追蹤處理流程

感知：人體偵測與姿態估計

第一階段為片段的每一幀還原畫面上所有人的骨架並快取，是整個模組的 GPU 時間瓶頸。先用人體偵測器（YOLOX）框出所有人，再採由上而下路線逐人裁切、放大回模型輸入尺寸後在滿解析度下估骨架（RTMPose），讓很小的球員仍能得到準確的關節點（圖 3.31）。由於逐人估骨架的成本隨人數線性成長，而每幀偵到的十幾二十人裡真正相關的只有兩到四人，估骨架前先借單應性把每個人的腳點反投影回球場座標，只讓落在放寬球場範圍內者進入姿態模型。



(a) 原始影格：人體框高 220 px　(b) 裁切並縮放為模型輸入 192 × 256　(c) 在特寫上估出的 COCO-17 骨架

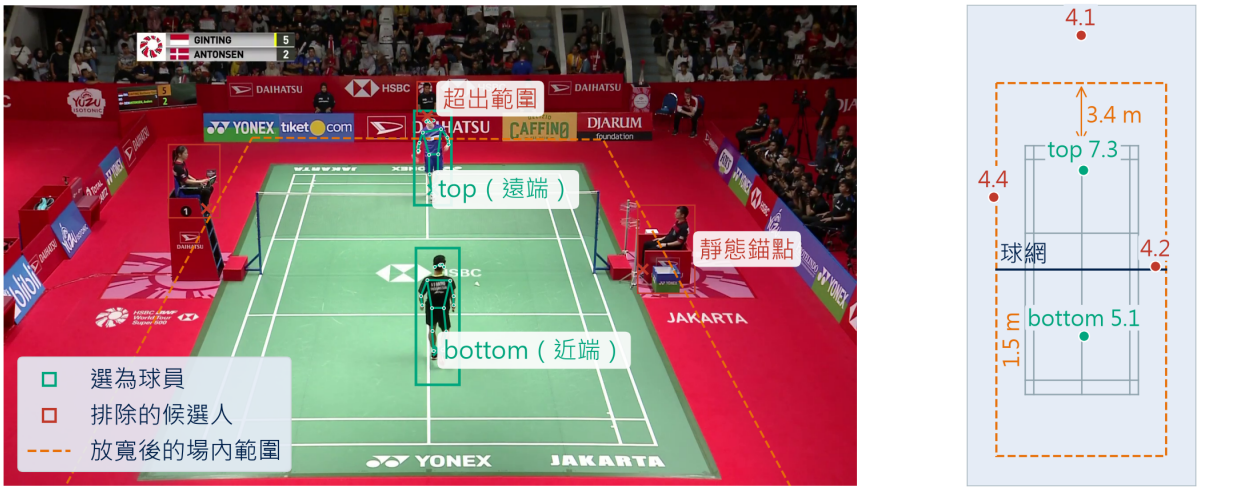
圖 3.31: 由上而下，先裁切放大再估骨架

選人：腳點與球網分半

第二階段在快取好的骨架上以純幾何認出 top、bottom 兩名球員。把每個人化約為一個雙踝中點，經單應性反投影落到正規化球場座標。每一半內通過球場邊界的候選人可能不只一個，因此需要以下的排名與排除準則挑出球員。

公尺身高排名：不能直接選最高的人，像素高度會被透視扭曲。因此先借單應性量出畫面上每點「一公尺對應多少像素」，把候選人放到同一真實尺度上比身高。

非對稱邊界放寬：單應性只在地平面上成立，球員起跳殺球時腳踝離地，反投影會把腳點推到底線之外。為此把場內範圍在底線方向（縱向）放寬、邊線方向（橫向）收緊（圖 3.32）。

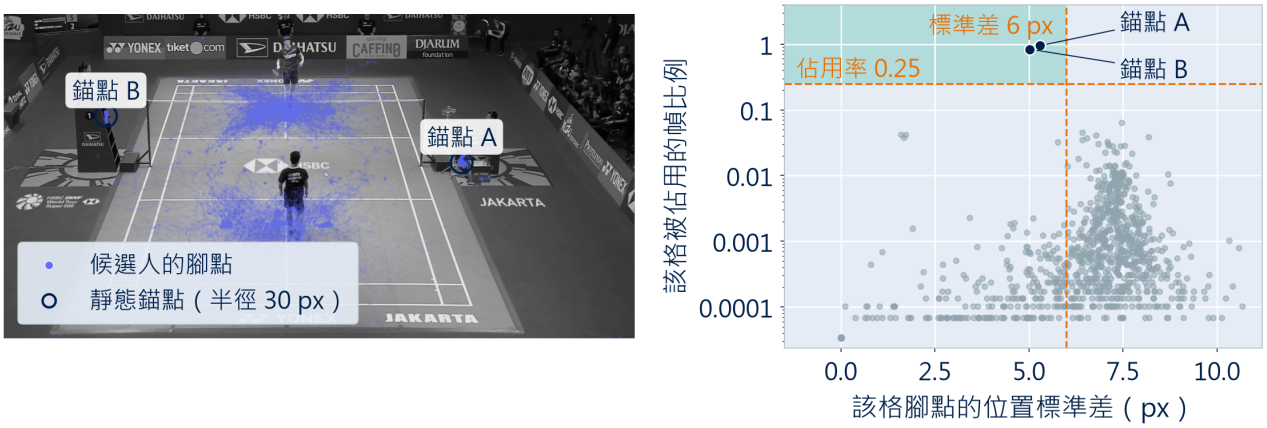


(a) 偵測到的候選人與腳點（×）　(b) 腳點反投影後的球場座標（數字為球場公尺身高）

圖 3.32: 腳點反投影至球場座標

靜態錨點排除干擾：尺寸與邊界仍擋不住一種情形，站在底線後、身高又與遠端球員相當的裁判，一旦真球員該幀被漏偵便可能頂替上來。關鍵在於是否移動，官員整場腳點落在

同一處，球員則掃過整個球場。統計整場每個候選人的腳點落點，把在很高比例的幀裡被占用且幾乎不動的格子標為靜態錨點，之後腳點離錨點太近的候選人一律拒絕（圖 3.33）。



(a) 整場的腳點落點　　(b) 每一格的佔用率與位置標準差

圖 3.33: 整場腳點落點，固定不動者被判為靜態錨點

有記憶的連續追蹤：逐幀獨立選人仍會偶爾跳錯人，故改用帶短期記憶的追蹤器，有記憶時選離「上一幀所在位置」最近的人。記憶過舊即作廢，下一幀重新起始時無前一位置可依靠，便退回用公尺身高挑最高者。連續性成為依據，把漏偵時誤選裁判的風險壓到最低。

3.4 球體分析

3.4.1 球體追蹤與軌跡重建

概述

問題與目標：本模組為每個固定機位比賽片段重建羽球在畫面中的逐幀位置，作為下游擊球時刻偵測與球種分類的基礎。難點在於目標小、速度快，且球常被球員、球拍與球網遮擋。

設計重點：本模組沿用預訓練的 TrackNetV3，並利用時間連續性與運動約束，改善單幀偵測的誤判與缺失，主要有兩點做法：

1. **多候選軌跡搜尋：**以低門檻保留每幀多個候選，再用 Viterbi 找出整段最連貫的路徑，使微弱但連續的真球能排除高信心的孤立誤偵。
2. **互補雙軌跡：**同時輸出覆蓋率較高的 InpaintNet 軌跡與較保守的 Viterbi 軌跡，供下游依任務分工使用。

處理流程總覽：如圖 3.34。

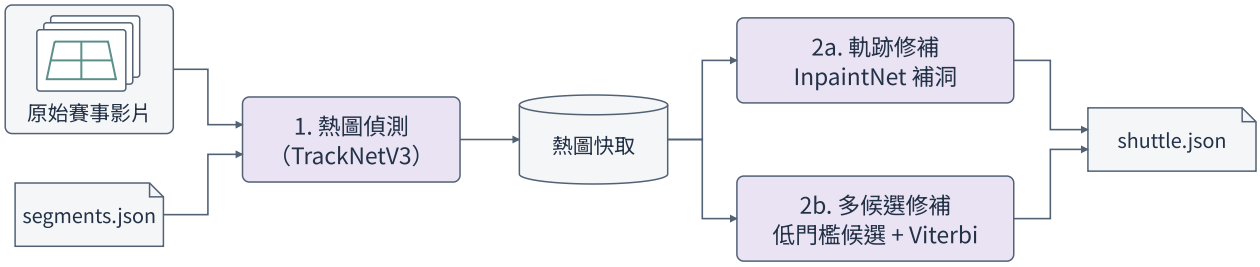
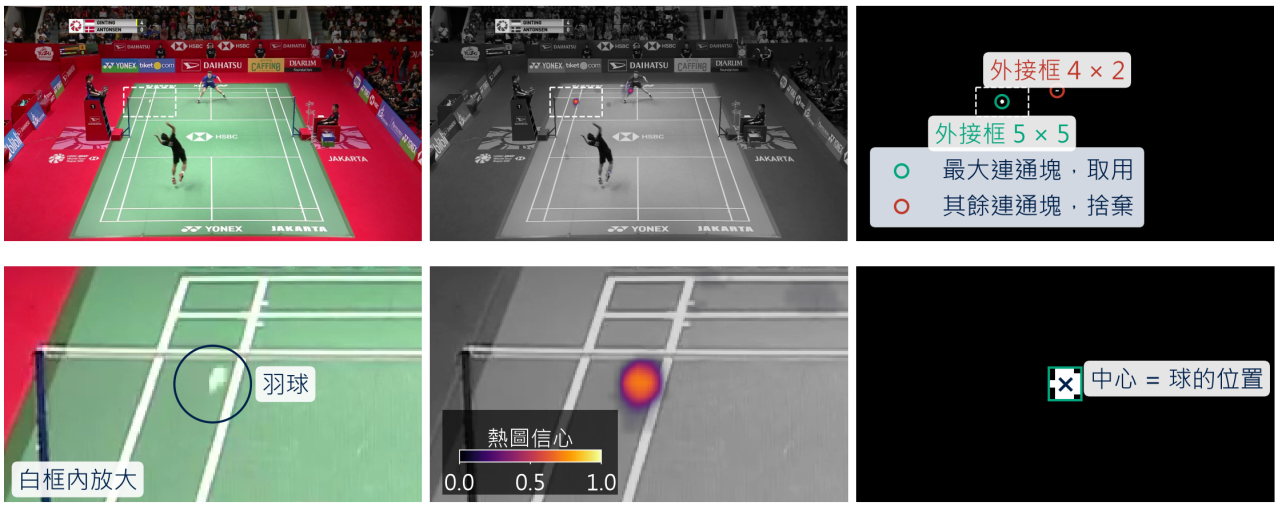


圖 3.34: 球體追蹤與軌跡重建處理流程

熱圖偵測與快取

第一階段要在極小又易與背景混淆的目標上給出可靠證據。系統使用 TrackNetV3 對連續影格進行推論，產生逐幀信心熱圖。熱圖能保留多個可能位置，使後續方法可利用時間連續性決定最合理的軌跡。由於熱圖推論是主要的 GPU 運算，系統將結果按片段快取，使 InpaintNet 與 Viterbi 能共用相同的偵測結果，並可在不重新推論的情況下反覆調整軌跡重建方法。

先以 largest-blob 讀出一條基準軌跡，用信心門檻二值化後取面積最大的連通區塊，以其中心為位置、區塊峰值為該幀信心（圖 3.35）。這條基準軌跡是修補法的輸入，而選擇法另從同一份熱圖以低門檻自取候選。

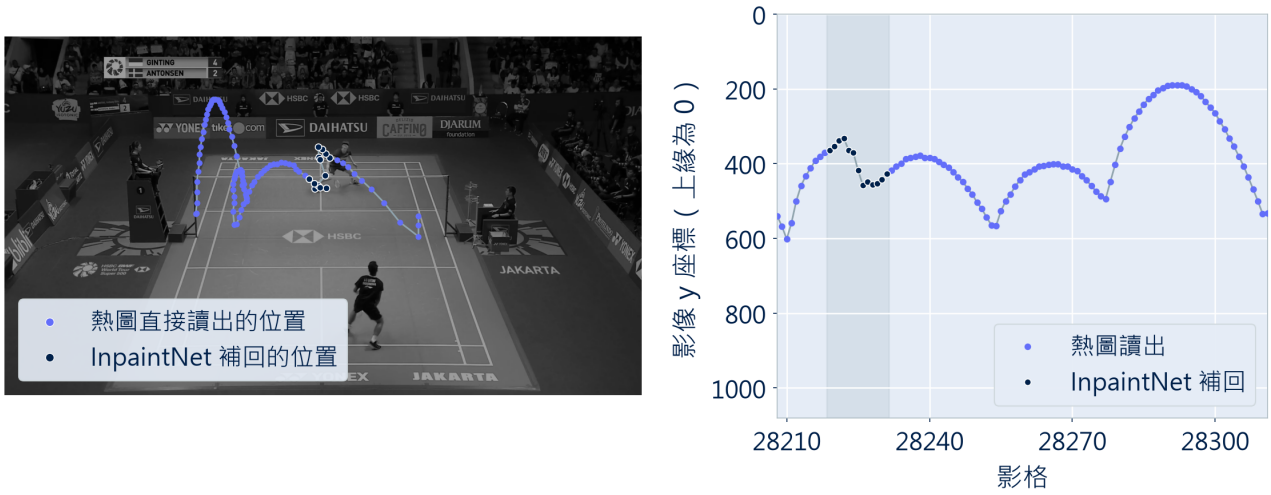


(a) 原始影格　(b) TrackNet 輸出的熱圖疊在影格上　(c) 以 0.5 二值化後取最大連通塊

圖 3.35: 從熱圖以 largest-blob 讀出基準軌跡

軌跡修補（InpaintNet）

第一種方法先將熱圖二值化，取每幀面積最大的連通區塊作為基準位置，再由補洞網路（InpaintNet [3]）補回遮擋造成的缺幀（圖 3.36）。補值點的信心標為 0，使下游能區分實際偵測與模型推論。

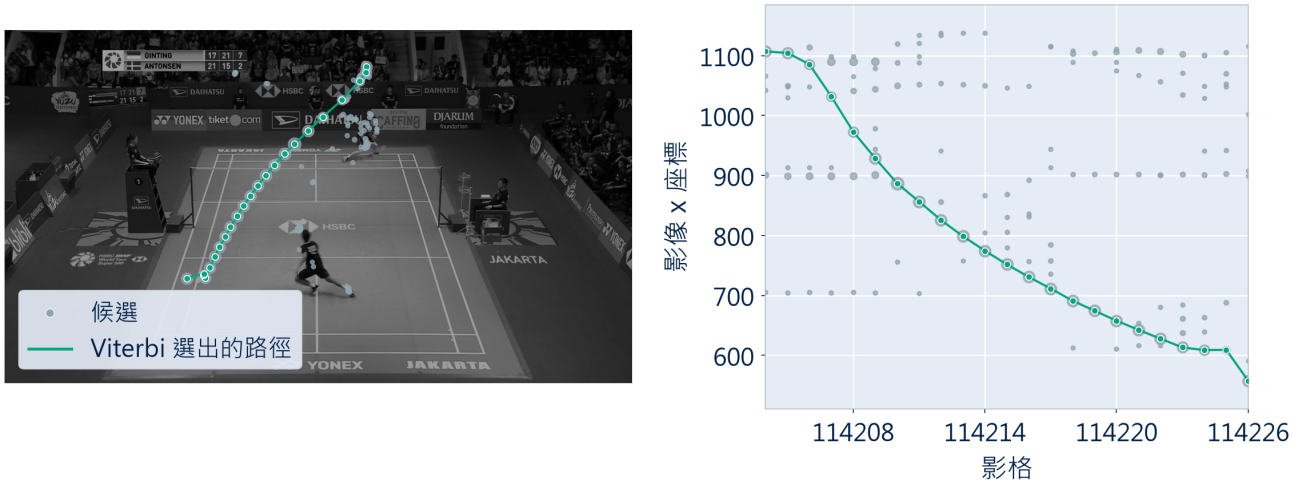


(a) 一段飛行的路徑　　(b) 逐影格的 y 座標（灰底為缺口）

圖 3.36: InpaintNet 補回球員遮擋造成的缺幀

多候選軌跡修補（Viterbi）

第二種方法改以全域路徑搜尋壓制孤立誤偵。它用極低門檻把每幀所有稍可能是球的區塊全留作候選，再找一條最像真實飛行的路徑穿過它們。把每幀候選當節點、相鄰候選間連邊，給節點獎勵（信心越高越好）、給邊代價（前後兩點推出的速度越不合物理越貴、跨幀越多越貴），以動態規劃取總分最高的路徑。如此微弱但連貫的偵測能勝過強烈但孤立的偵測（圖 3.37）。當某幀球員鞋子或場線在信心上壓過真球，只要接不上前後飛行就不會入選。最後才對剩餘缺口依長度與兩端運動狀態決定補或不補，並以卡爾曼濾波填值。



(a) 一記高遠球飛行期間各影格的所有候選　　(b) 同一批候選以影格為橫軸

圖 3.37: 低門檻多候選與 Viterbi 選出的最佳路徑

雙軌跡並存

修補法與選擇法各有所長、在不同情境下互有勝負，模組因此不在內部挑出贏家，而是把兩條軌跡都算出、都寫進 shuttle.json，把選擇交給握有更多上下文的下游。

3.4.2 擊球時刻偵測

概述

問題與目標：本模組在每個片段內標記出每一擊球的影格，為下游球種分類提供窗口範圍。

設計重點：本模組的初始版本參考了開源專案 SoloShuttlePose 的擊球事件偵測（event_detect）[14]，其僅以單條球體軌跡的 Y 軸峰值與軌跡轉角判定擊球。現行版本在其基礎上重新設計，改以多源證據融合與結構性補漏取代單一軌跡幾何。一次擊球是軌跡上一瞬間的事件，只在球路上留下細微痕跡，任何單一訊號都不夠可靠。本模組因此同時融合多種訊號，並以召回優先，把不漏拍作為首要目標。主要有兩點做法：

1. **多源證據融合：**不押注單一訊號，同時利用兩條球體軌跡、球員骨架與逐幀球種模型三種來源。相對於只讀軌跡幾何的做法，任一來源在特定情境失效時仍能由其他來源補上。
2. **結構性補洞：**不靠放寬偵測門檻加球，而是從「結構上一定有一拍」的位置（交替違規、模型長時間鎖定、發球遮擋）反推漏拍位置。

在 8 場未參與開發的比賽上，本模組在 ±5 影格容差下召回率 90.90%、精確率 95.49%（見 4.2.3 節）。

處理流程總覽：如圖 3.38。

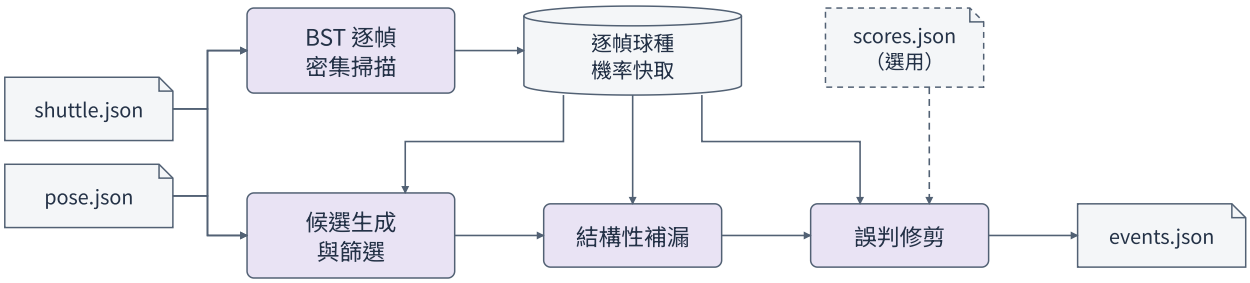


圖 3.38: 擊球時刻偵測處理流程

逐幀球種證據

本模組先對每個片段逐影格掃描，將每個影格前後各半秒的窗口，輸入球種分類模型 BST [4]，得到該影格的 25 類球種機率分布。

BST 原本設計的窗口是「兩次擊球之間」，需要以擊球時刻作為窗口邊界。但在本模組，擊球時刻正是需要偵測的目標。因此本模組將所有影格以固定窗口輸入模型，並將輸出反過來作為是否存在擊球的判斷依據之一。BST 的輸出提供三種輔助證據（圖 3.39）：

球種信心：若模型對某個影格的窗口給出高信心的球種判斷（例如 0.9 的殺球），代表這個窗口的球路與骨架很接近某個真實擊球的樣態。相較於另一個最高信心僅 0.3 的窗口，前者更可能對應到一次真實擊球。

擊球者判定：即使窗口沒對齊擊球，模型對「擊球者位於畫面上方或下方」的判斷仍相當可靠，比單純比較羽球與兩名球員的距離更為準確，因此在後續分邊時作為優先證據。

連續球種區間：若把某個真實擊球影格前後半秒作為窗口輸入模型會得到球種 a，那麼對它鄰近的前後一兩影格做同樣的事，因窗口內容高度重疊，也極可能得到相同的球種 a，只是信心略低。反過來說，只要觀察到連續多個窗口被判為同一球種，就代表該區間附近很可能存在一次擊球。

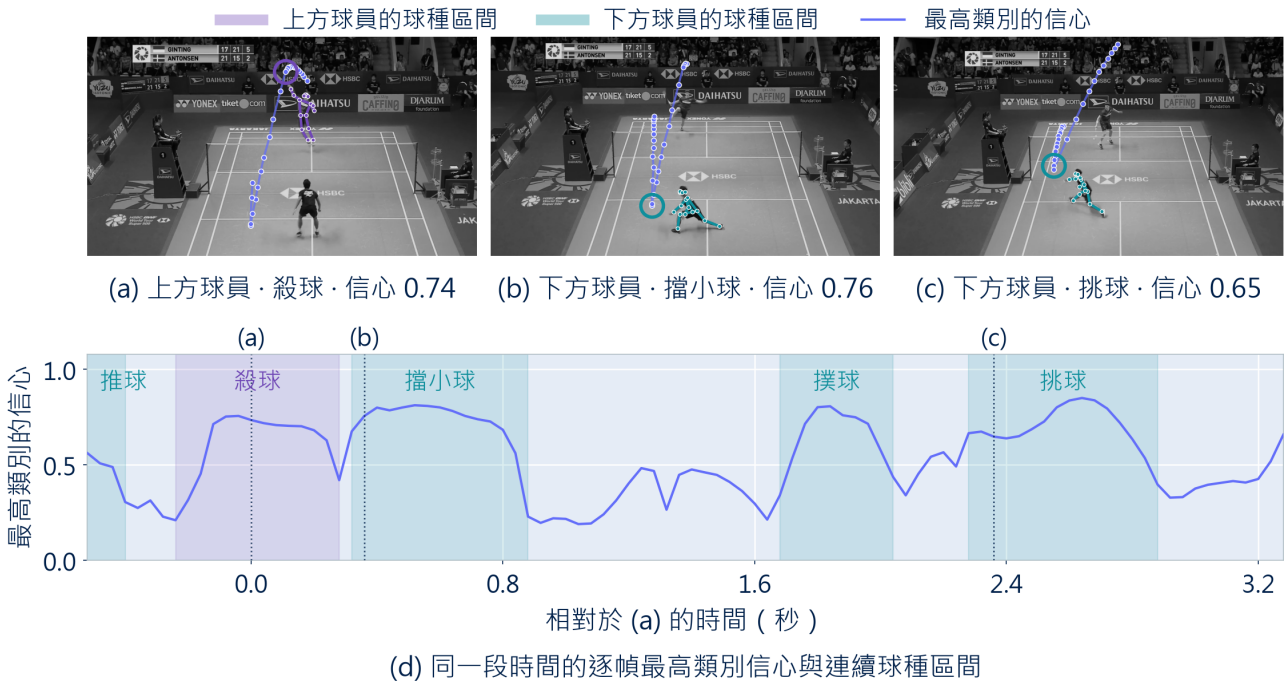


圖 3.39: 逐幀滑動窗口產生的球種、信心與擊球者證據

候選生成

本步驟分析軌跡中實際存在的有效點。由於不同球種在軌跡上的差異很大，不同位置擊球也會造成軌跡產生不同變化，因此單一訊號無法抓出所有擊球。本步驟尋找下列互補的擊球特徵（圖 3.40）：

垂直轉折：影像座標的 y 值向下增加，因此羽球下降至球員附近、擊球後重新上升時，會在軌跡上形成局部最大值。

方向與速率突變：比較候選前後的速度向量，若方向明顯轉折且速率比例突然改變，較可能是球拍接觸，而非重力造成的平順轉彎。

由慢轉快：平抽或快速推球未必形成明顯垂直轉折，因此另以擊球前後的速率比及水平速度變化尋找候選。

發球錨點：額外偵測回合開頭的陡峭垂直變化與第一段持續上升軌跡，作為發球候選。

本步驟刻意偏向敏感、高召回，先保留不確定候選，再交由後續規則排除（圖 3.41）。擊球時刻要用來為下一階段的球種模型切分析窗口，一旦漏掉一拍，窗口就會整段錯位，後續的比賽脈絡也會出現難以復原的斷點。

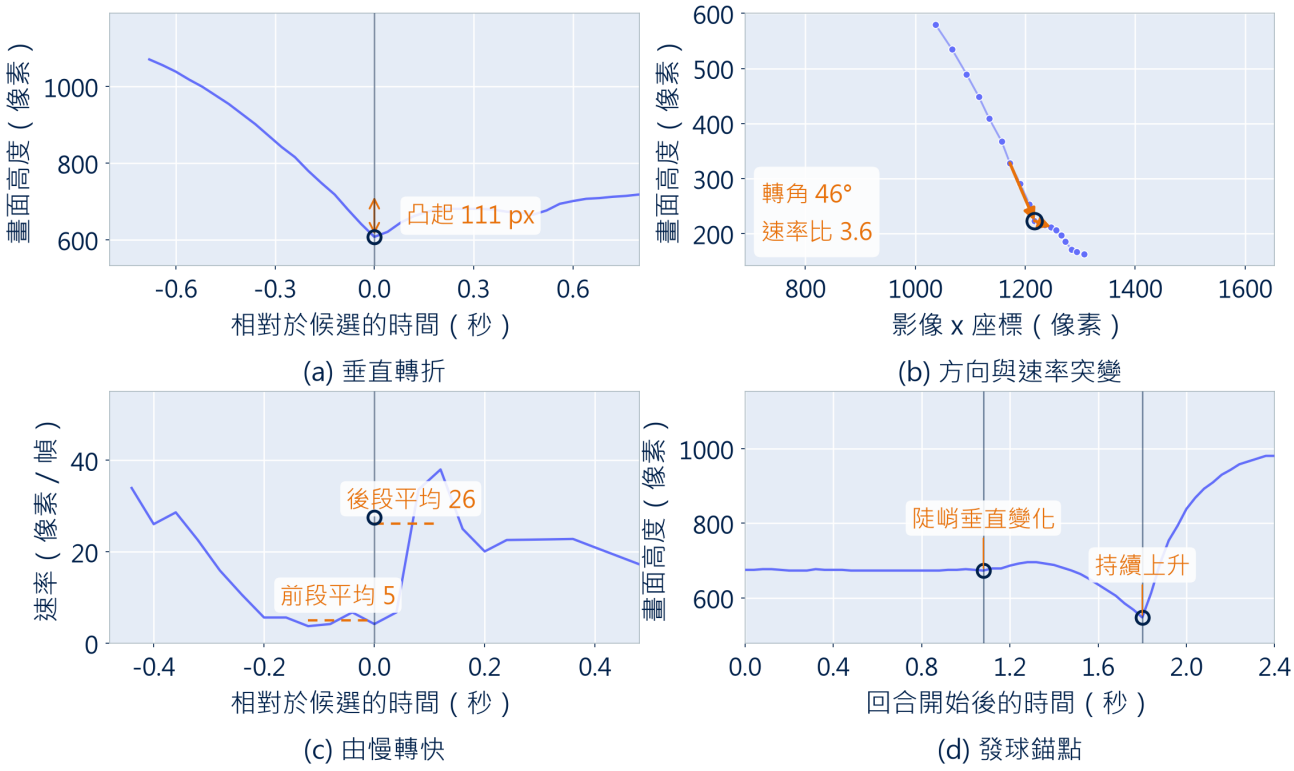


圖 3.40: 四種互補的擊球候選訊號

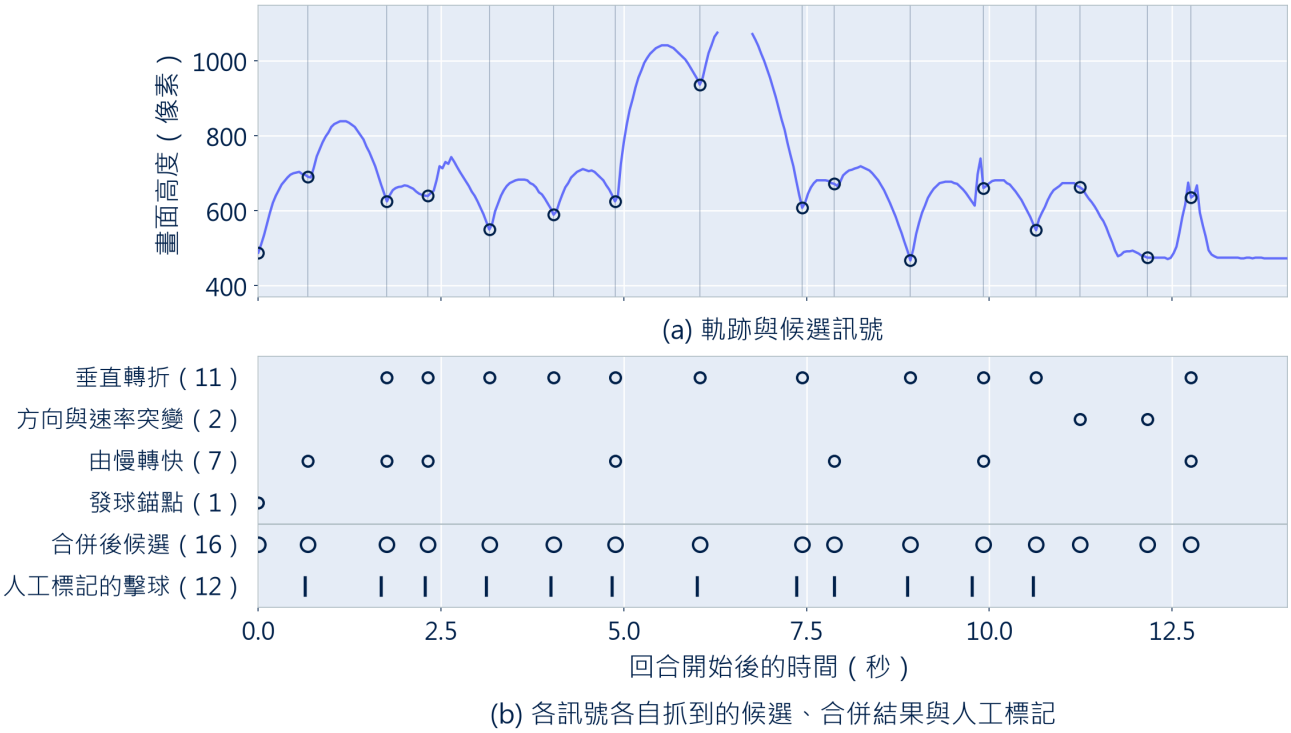


圖 3.41: 各訊號在完整回合中的候選分布與合併結果

候選篩選

本步驟將候選經過三道篩選（圖 3.42）：

位移振幅：計算候選附近時間窗內的水平與垂直位移範圍，只要其中一軸具有足夠位移便予以保留，以排除球落地後的微小追蹤抖動。

同球員去重：依靠 BST 的輸出判定擊球者，若時間相近的連續候選被判為同球員擊球，通常代表一次擊球同時觸發多種軌跡訊號。系統只保留水平與垂直位移振幅總和較大的候選。

回合脈絡：已知擊球側的候選附近應能找到對側球員的其他擊球候選。目前規則要求在指定時間窗內至少存在兩個對側候選，否則將其視為落地彈跳、追蹤脫線或其他孤立訊號。

結構性補漏

單靠放寬候選門檻會同時增加大量誤判，因此本步驟只在回合結構或多源證據顯示「此處很可能漏了一拍」時加入擊球。補漏刻意安排在候選篩選之後，序列先經篩選、去除雜訊後較為乾淨，回合脈絡才判斷得準，也才能據此救回一些訊號微弱、難以直接偵測的球。四條補漏規則依序執行，且新加入的擊球不得與既有擊球過近。

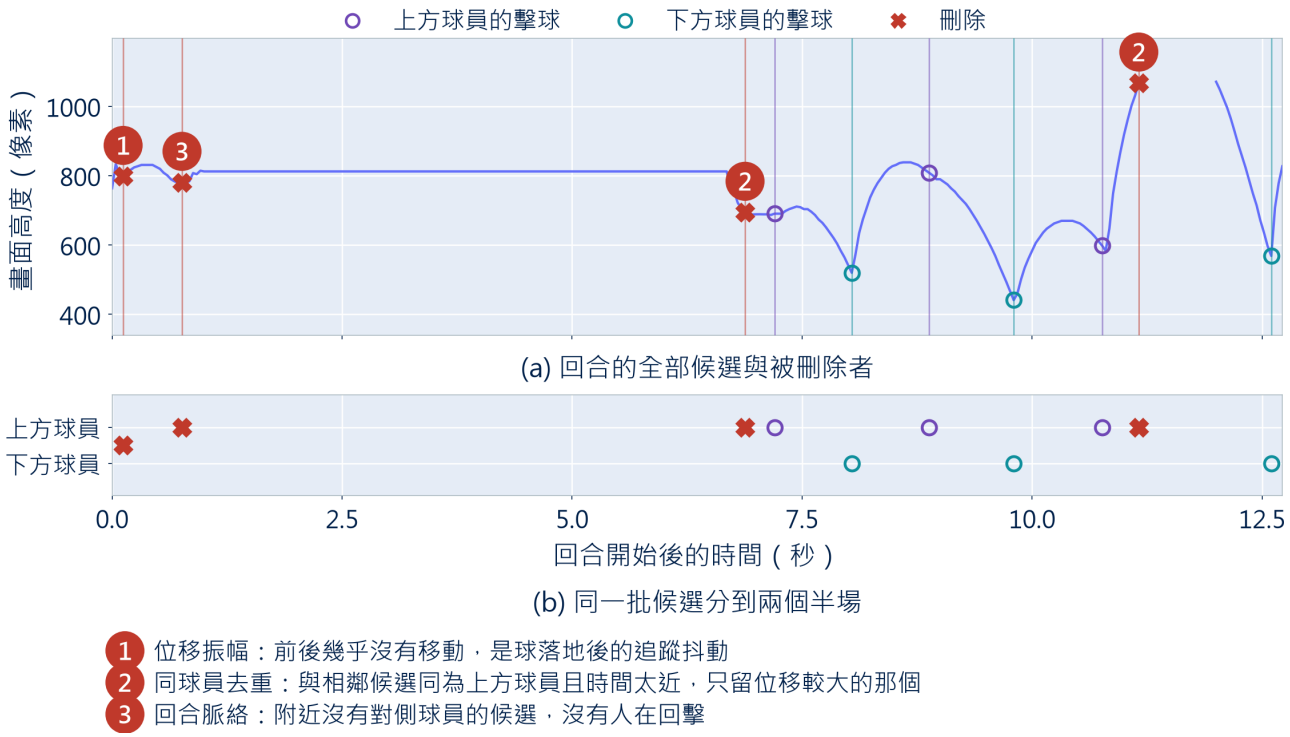


圖 3.42: 三道候選篩選規則及其保留／刪除結果

交替違規補漏：若序列中相鄰兩拍均由同球員擊出，代表兩者之間可能漏掉一拍對側擊球。系統依序搜尋主軌跡上較弱的垂直或水平轉折，以及 BST 的對側高信心連續區間，找到足夠證據後才補一拍。

未被認領的鎖定區間：若 BST 長時間、高信心地維持同一球種，但附近沒有任何擊球，系統會在該區間內尋找通過振幅條件的弱垂直轉折（圖 3.43）。BST 在此只指出可能漏拍的區域，不單獨決定擊球影格。

輔助軌跡救援：球體追蹤模組除了主軌跡，另輸出一條 Viterbi 軌跡（見 3.4.1 節），能看到主軌跡因遮擋或過度平滑而漏掉的球。若這條輔助軌跡在某處偵測到候選，且該候選接近 BST 球種訊號的起點、具有足夠信心，便將它加入。

近端發球補漏：畫面下方球員發球時，部分軌跡可能被身體遮擋。若持續上升錨點附近尚無擊球，而 BST 同時提供下方球員的發球證據，系統便在附近被篩掉的候選中選擇最接近者（圖 3.44）。

誤判修剪

本步驟清理回合邊緣不屬於實際對打的少數雜訊。僅針對三種很特定、平時不常觸發的情況：

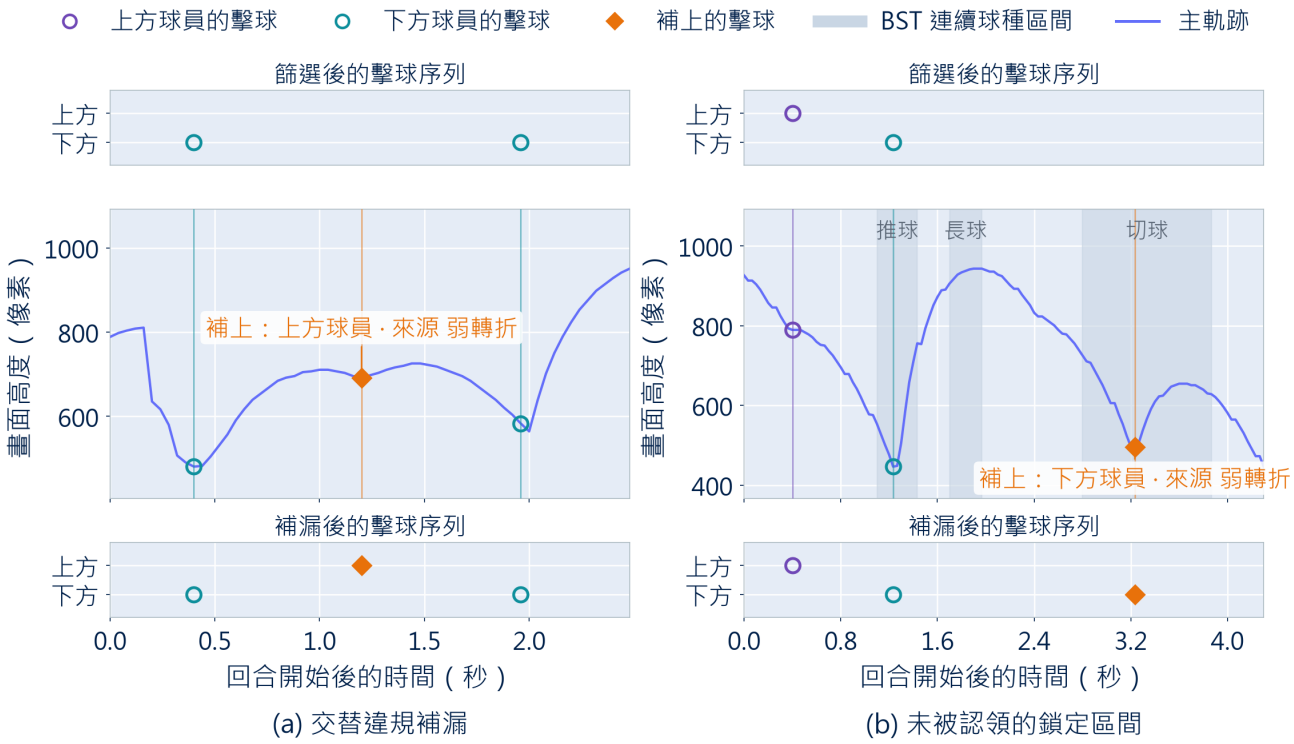


圖 3.43: 由擊球序列與連續球種證據反推漏拍位置

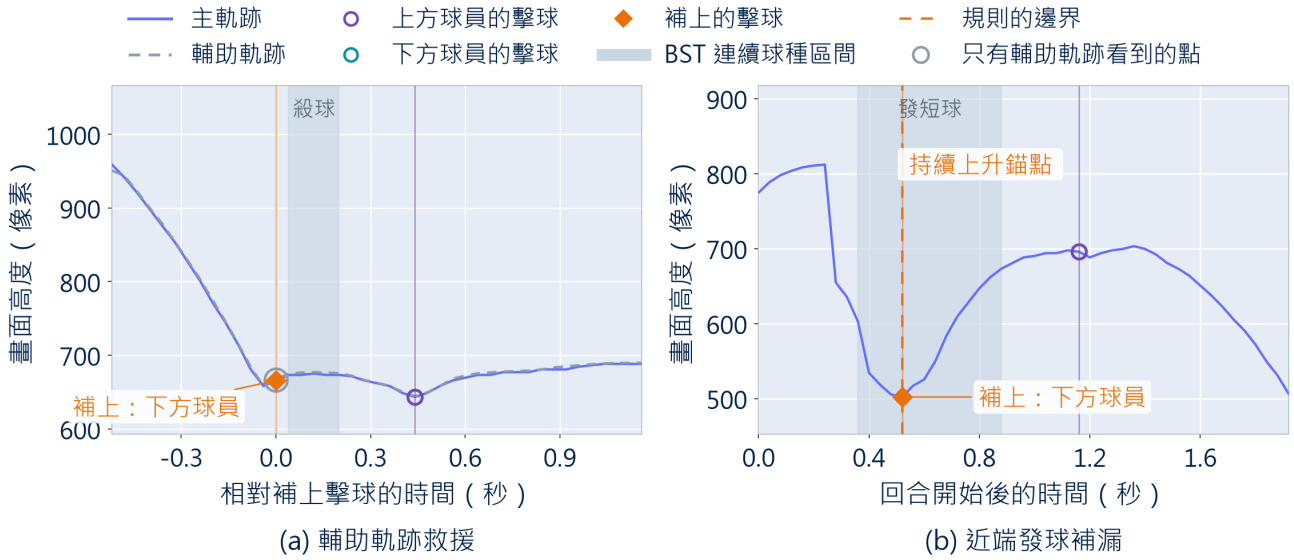


圖 3.44: 由輔助軌跡與發球錨點救回漏拍

回合有效區間：發球前場上還沒有球在移動，球體追蹤可能把反光亮點之類誤認成球，生出不存在的球路，需剔除。

尾段落地球：回合結束後，羽球的彈跳與滾動仍可能產生軌跡轉折，需剔除（圖 3.45）。

計分板死時間：特定休息比分下的計分板死時間（如開局 0:0、局中技術暫停，圖 3.46）。

(a) 開頭：發球之前的訊號　　(b) 尾段：得分之後的彈跳與滾動

圖 3.45: 利用回合有效邊界清除開頭誤偵與尾段落地球

規則：比分連續好幾段都沒變（0:0 或局中 11:x）時，
最後一個「BST 有發球證據」的片段才是真回合，它之前的全部清空

片段	比分	BST 發球證據	判定
第 0 段	0:0	有	死時間，本來就沒擊球
第 1 段	0:0	無	死時間，本來就沒擊球
第 2 段	0:0	有	死時間，本來就沒擊球
第 3 段	0:0	有	死時間，清空 5 拍
第 4 段	0:0	有	真回合，保留 9 拍

這一串裡最後一個有發球證據的片段

(a) 開賽前的 0:0

片段	比分	BST 發球證據	判定
第 21 段	11:6	有	死時間，本來就沒擊球
第 22 段	11:6	有	真回合，保留 5 拍

這一串裡最後一個有發球證據的片段

(b) 局中技術暫停 11:6

圖 3.46: 結合比分與發球證據判定計分板死時間

3.4.3 球種分類

概述

問題與目標：本階段根據每一次擊球前後的球員姿態、場上位置與羽球軌跡，判斷擊球者及球種。

設計重點：球種分類器採用預訓練的 Badminton Stroke-type Transformer（BST）[4]，本專題不重新訓練或修改其模型架構。BST 解決的是「已知擊球時刻及各項特徵後，如何判斷球種」。本專題在球種分類上的重點在於，利用前述各模組，自動從原始比賽影片產生 BST 所需的全部輸入。

處理流程總覽：如圖 3.47。

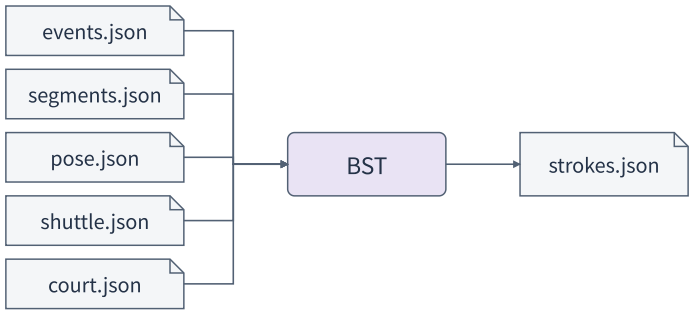


圖 3.47: 球種分類處理流程

分類流程

系統先將每個擊球時刻歸入對應回合，再依 BST 預訓練權重所使用的方式，以相鄰擊球時刻切出分析窗口。窗口內的球員骨架、場上位置與羽球軌跡會按照 BST 的資料格式進行正規化，並統一整理為 100 幀的輸入序列（圖 3.48）。

整理後的資料送入預訓練 BST，模型輸出 25 類機率（圖 3.49），將結果依賽評需求整理為 8 類：高遠球、發球、小球、平快球、殺球、切球、撲球與勾球。若模型輸出未知類別，系統保留為未知。

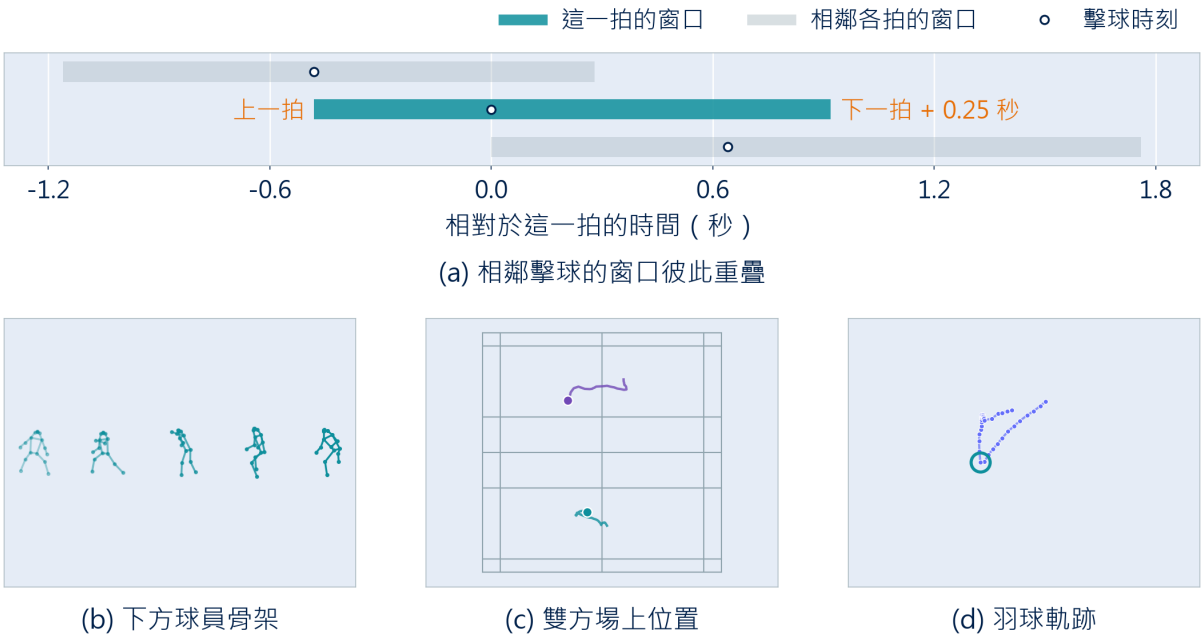


圖 3.48: 依相鄰擊球時刻建立 BST 的 100 幀輸入窗口

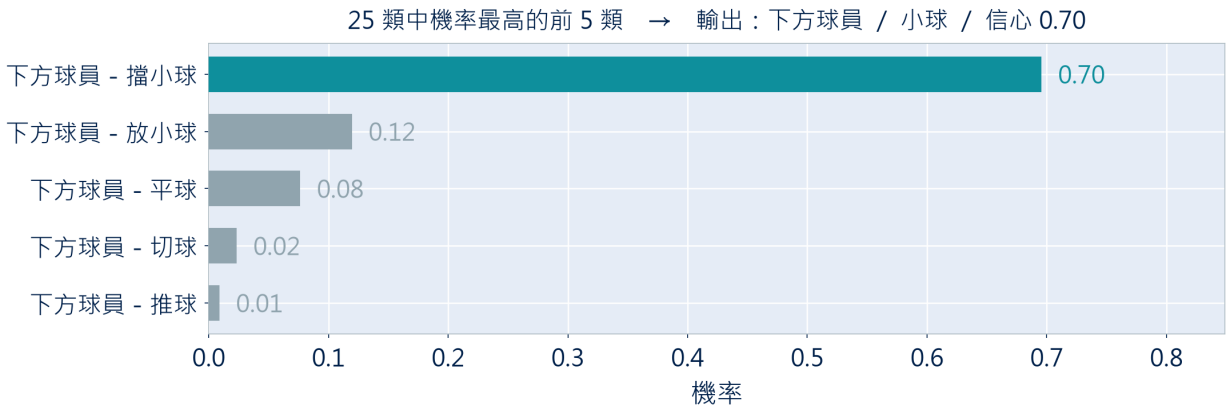


圖 3.49: BST 的 25 類預測

3.5 音訊與精彩度分析

3.5.1 歡呼音訊量測

概述

問題與目標：視覺模組能說明每一拍發生了什麼，卻無法判斷哪個回合最讓現場有反應。觀眾歡呼是轉播中現成的精彩度訊號。本模組以分段模組產生的每個片段為範圍，量測片段結束前後是否出現歡呼及其相對強度，作為精彩片段排序的依據。

設計重點：本模組不從頭訓練音訊模型，而是以凍結的 YAMNet [17] 擷取通用音訊特徵，只在其上訓練一個輕量的邏輯迴歸分類器，因此 400 個人工標註窗口就足以完成訓練。歡呼多在回合結束的瞬間爆發，所以讀取範圍會延伸到片段結束後 3 秒。分類與強度分開估計：分類器判斷窗口裡有沒有歡呼，響度則衡量歡呼有多大聲，兩者再各自聚合為每個片段一筆量測。音訊只是視覺分析的補充，不建立或覆寫擊球、球種、比分與球員身分等競賽事實，缺少音訊時，其餘模組仍照常運作。以 4 場比賽逐場交叉驗證，窗口層級歡呼偵測的 macro F1 為 0.827、ROC-AUC 為 0.957（見 4.2.5 節）。

處理流程總覽：如圖 3.50。

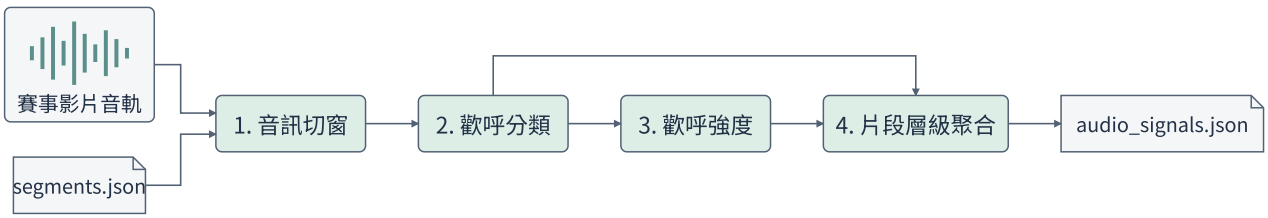


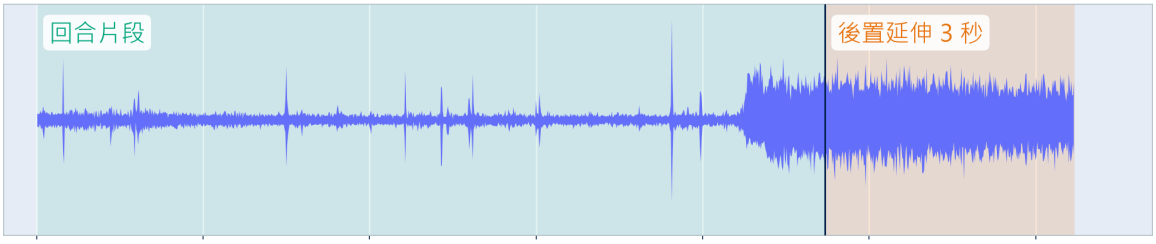
圖 3.50: 歡呼音訊量測處理流程

音訊切窗

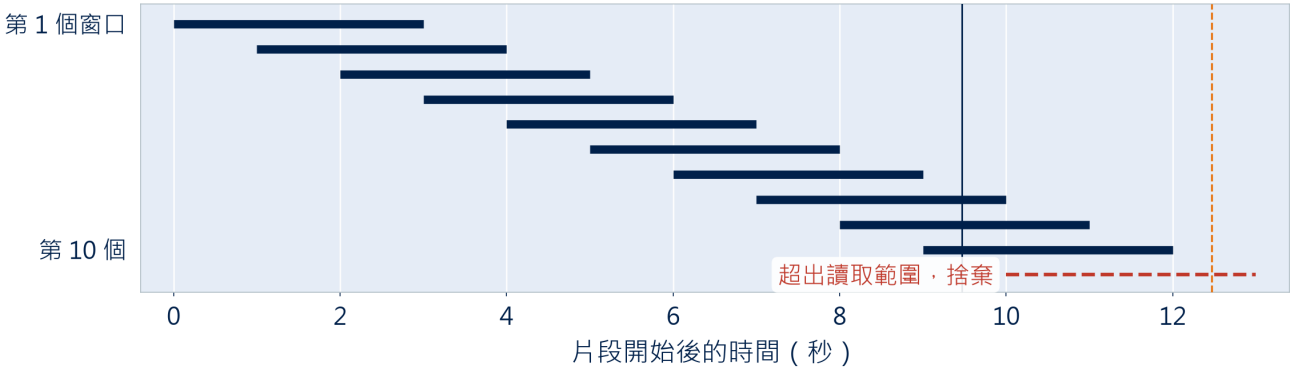
系統先將音訊統一轉為 16 kHz 單聲道。對片段 s，分析起點等於片段起點，終點則延伸到片段結束後 3 秒，並以影片長度為上限：

$$A_s^{start} = S_s^{start}, \qquad A_s^{end} = \min(S_s^{end} + 3,\ T_{media}) \tag{3.9}$$

延伸的原因是歡呼往往在回合結束時才開始，並在片段結束後持續數秒。分析範圍以 3 秒窗口切分，相鄰窗口的 hop 為 1 秒，只保留完整落在範圍內的窗口，尾端不足 3 秒的窗口直接捨棄（圖 3.51）。相鄰片段各自切窗，因此兩者的窗口可能重疊。



(a) 片段的音訊波形，讀取範圍延伸到片段結束後 3 秒



(b) 每 1 秒取一個 3 秒窗口，尾端不足 3 秒的窗口不保留

圖 3.51: 片段音訊的讀取範圍與切窗

歡呼分類

每個 3 秒窗口輸入凍結的 YAMNet [17]。YAMNet 以 AudioSet [18] 音訊事件資料預訓練，對一個窗口輸出多個時間切片的 1024 維特徵向量（patch embedding），系統取其平均作為窗口的固定長度表示：

$$\mathbf{e}_i = \frac{1}{K_i} \sum_{k=1}^{K_i} \mathbf{z}_{i,k}, \qquad \mathbf{e}_i \in \mathbb{R}^{1024} \tag{3.10}$$

其中 $\mathbf{z}_{i,k}$ 為第 i 個窗口的第 k 個時間切片特徵，K_i 為切片數。平均向量經標準化後送入邏輯迴歸，輸出窗口的歡呼分數 $p_i \in [0, 1]$；$p_i \geq 0.5$ 的窗口視為正例窗口。

本系統不直接使用 YAMNet 原生的 Cheering 類別分數。在相同的逐場交叉驗證下，原生分數的 ROC-AUC 只有 0.839，以特徵向量訓練的分類器則為 0.957（見 4.2.5 節）。歡呼分數是分類器的輸出，未經機率校準，只用於門檻判斷與排序，不應解讀為歡呼發生的機率。

歡呼強度估計

同樣被判為歡呼，全場起立的喝采與零星掌聲在響度上可能差很多。因此系統另以窗口波形 $x_1, \ldots, x_N$ 的均方根（RMS）衡量響度：

$$RMS_i = \sqrt{\frac{1}{N}\sum_{n=1}^{N} x_n^2}, \qquad L_i = 20\log_{10}(RMS_i + 10^{-12}) \tag{3.11}$$

不同轉播的錄音音量差異很大，絕對響度無法跨比賽比較，因此系統只在單場比賽內比較。將全場 M 個正例窗口的 L_i 由小到大排名（從 0 起算，同值取平均排名），再除以 $M-1$ 映射到 $[0,1]$，得到窗口的歡呼強度 r_i。全場只有一個正例窗口時，強度定為 0.5。負例窗口不計算強度。r_i 表示這個窗口在本場所有歡呼中的響度排名，而不是實際音壓。

片段層級聚合

每個片段約有十多個窗口，需要聚合為一筆量測。歡呼通常只占片段的最後幾秒，取平均會被前段大量的安靜窗口稀釋，取最大值則容易被單一誤判窗口左右。系統因此折衷採用線性內插的第 95 百分位數（p95）。令 W_s 為片段 s 的全部窗口、W_s^+ 為其中的正例窗口：

$$C_s = Q_{0.95}^{linear}(\{p_i \mid i \in W_s\}), \qquad N_s = |W_s^+| \tag{3.12}$$

$$I_s = \begin{cases} Q_{0.95}^{linear}(\{r_i \mid i \in W_s^+\}), & |W_s^+| > 0 \\ \text{null}, & |W_s^+| = 0 \end{cases} \tag{3.13}$$

C_s 為片段歡呼分數，聚合片段內全部窗口；I_s 為片段歡呼強度，只聚合正例窗口，沒有正例時為空值；N_s 為正例窗口數。由於窗口彼此重疊，N_s 不等於獨立的歡呼次數。各聚合方法的比較見 4.2.5 節。

3.5.2 精彩片段排序

概述

問題與目標：本模組把每個片段的歡呼量測合成一個精彩度分數，將整場比賽的片段由精彩到平淡排序，供回看介面標示重要回合，也作為賽評生成的選用背景訊號。

設計重點：目前沒有人工標註的精彩片段真值可供訓練，因此精彩度分數採用透明的固定公式，而不是學習得到的模型。沒有歡呼的片段，強度以 0 計：

$$H_s = 0.5\, C_s + 0.5\, I_s^{effective}, \qquad I_s^{effective} = \begin{cases} I_s, & I_s \neq \text{null} \\ 0, & I_s = \text{null} \end{cases} \tag{3.14}$$

分數相同時，依片段歡呼分數、再依片段先後排序。0.5 ／ 0.5 的權重只表示等量結合「有沒有歡呼」與「歡呼多大聲」，並非最佳化的結果。由於強度是單場內的相對量，精彩度分數也只能在同一場比賽內比較。

處理流程總覽：如圖 3.52。

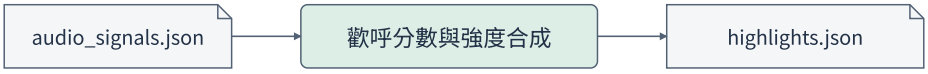


圖 3.52: 精彩片段排序處理流程

3.6 賽事理解

3.6.1 球員身分對應

概述

問題與目標：球種分類只知道擊球者位在畫面上方或下方，賽評卻需要指名選手。本階段判斷在每兩次換邊之間，上下方球員各是計分板上的哪一位選手。

設計重點：本階段不需要人工標註，主要也不依賴球員外觀，而是由既有的兩項結果推得。比分變化指出計分板上的哪一位選手發球，回合第一拍的擊球者指出發球者在上方或下方，兩者結合即為一次對應。換邊的時間點由羽球規則決定（第一局結束、第二局結束、決勝局領先方達 11 分），只需從比分推算，不必從畫面偵測。

處理流程總覽：如圖 3.53。

判定流程

誰發球：落地得分制下，贏得回合的一方發下一球。`scores.json` 記錄的是回合開始前的比分，所以比較相鄰兩個片段，比分剛好加一分的選手就是下一個片段的發球者。

發球者在哪一側：回合第一拍的擊球者所在的半場，就是發球者的半場。但若擊球時刻偵測漏掉發球，第一拍其實是接發球，半場會整個判反。因此只採用 BST 判為發球的第一拍。

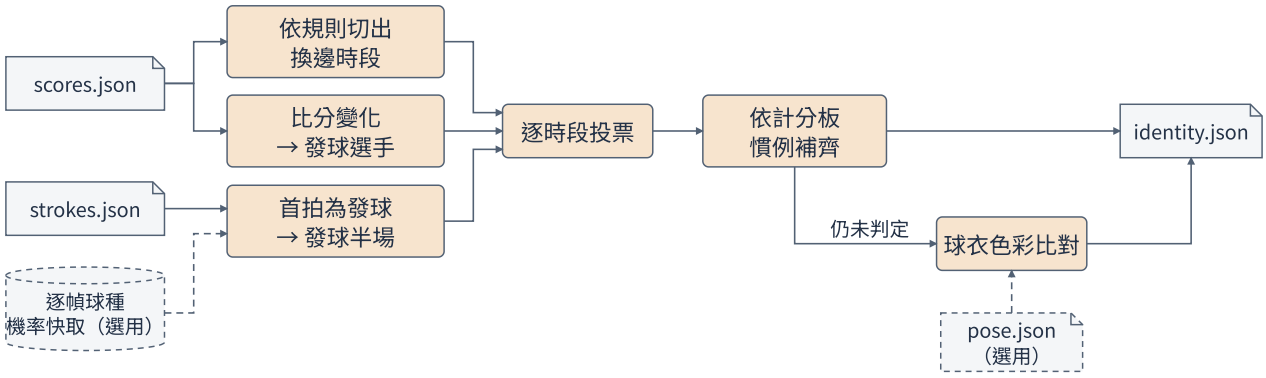


圖 3.53: 球員身分對應處理流程

其餘回合再從擊球時刻偵測的逐幀球種機率中找回發球，且只在發球類別與擊球者都明顯佔優時採用。

投票與補齊：同時知道發球選手與發球半場的回合各投一票，同一換邊時段內取多數。票數不足 5 票的時段，沿用最近已判定時段的結果，並依計分板慣例調整：列序固定的計分板每次換邊對調一次，隨場地換列的則不變。採用哪種慣例，由已判定時段之間是否對調推得。仍無法判定時，才比對兩側球員上衣與褲子的 HSV 色彩；連外觀也分不出來就留空，不做猜測。每個時段的判定依據（投票、慣例或外觀）都寫入 `identity.json`。

3.6.2 賽評生成

概述

問題與目標：本階段把一個回合的逐拍結果寫成繁體中文賽評，包含每一拍一句的逐拍賽評與一句回合摘要，並讓每一句都能追溯回管線的分析結果。

設計重點：大型語言模型擅長措辭，卻容易補上看似合理但資料中沒有的內容，例如誰贏得這一分、球速多快或球員跑向何處。因此本階段將事實與措辭分開：擊球順序、擊球者、球種與球場區域都由程式從上游產物組成，模型（Gemini 3.8 Flash）只負責提出戰術觀察與撰寫文字，輸出格式也不允許它填寫任何事實欄位。文字寫好後再經模型審查與程式規則兩層檢查，未通過的句子改用只陳述事實的模板句。

每個回合約需 3–4 次 Gemini 請求，為全場生成的成本不低。因此分析流程中的賽評階段只示範 3 個回合：有精彩片段排序時取排名最前的回合，否則取最長的片段。其他回合由使用者在回看介面按需生成。

處理流程總覽：如圖 3.54。

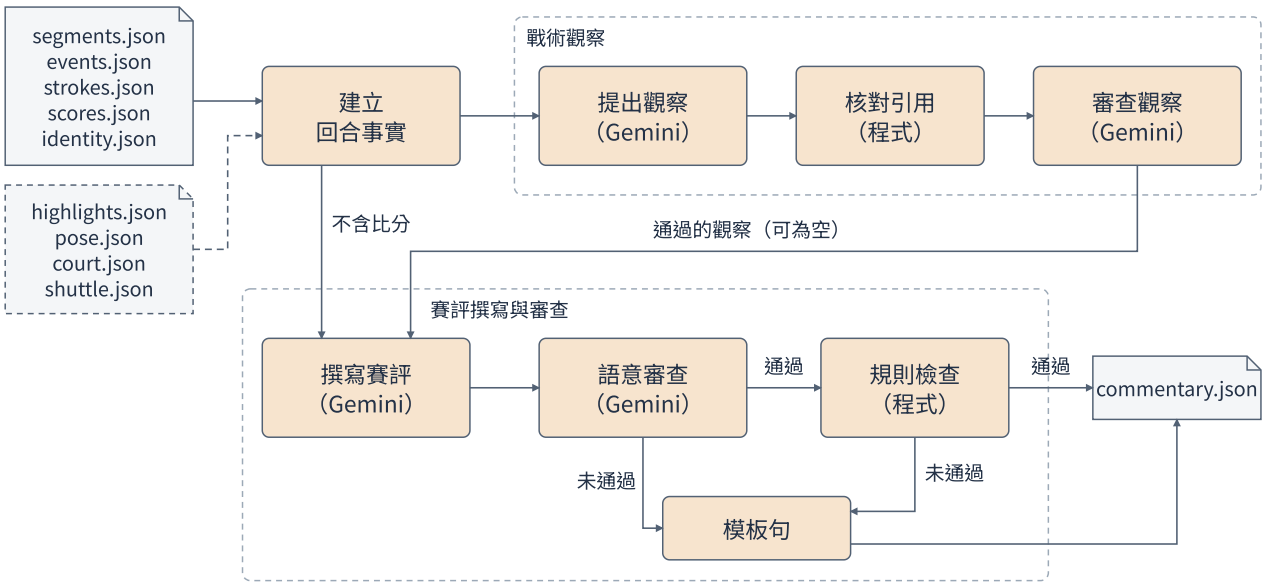


圖 3.54: 賽評生成處理流程

建立回合事實

程式先依片段範圍取出擊球並接上球種，再以該片段所屬換邊時段的身分對應，把畫面上方或下方換成選手 a 或 b。各項資訊送給模型的方式如表 3.3。

表 3.3: 賽評生成各項資訊的來源與處理方式

資訊	來源	處理方式
擊球順序、擊球者、球種與信心	`events.json` `strokes.json` `identity.json`	戰術觀察與賽評的共同事實，模型不得更改
球場區域（前中後、左中右）	`pose.json` `court.json`	只提供可靠的結果，可作為戰術觀察的依據
骨架與球路方向	`pose.json` `shuttle.json`	只作為提出戰術觀察時的背景，不可引用為依據，也不送入賽評撰寫
精彩度分數	`highlights.json`	只作為賽評撰寫時的背景，不得當作事件的證據
比分與發球方	`scores.json`	不送入模型，避免模型由此推測勝負或得分原因

信心分級：球種信心達 0.70 視為可靠，0.50–0.70 為謹慎，低於 0.50 為低信心。只有可靠的球種能作為戰術觀察的依據。謹慎與低信心的拍仍要輸出，但句子必須帶有「可能」、「似乎」等不確定語氣。

球場區域：以擊球時球員雙腳腳踝的中點，透過單應矩陣投影到球場平面，依該球員所在半場分為前、中、後場，依場寬分為左、中、右。只有雙腳腳踝都可用時，結果才視為可靠。系統目前無法自行察覺場地偵測的錯誤，因此球場必須先經使用者在介面確認，否則一律不提供區域，賽評也就不會提到位置。區域只表示擊球當下的位置，不代表跑動路徑。

無法處理的情況：擊球者不明的拍會保留在時間序中供模型參考，但不輸出賽評。賽評以回合為單位，若比分辨識判定片段內含多個回合，或該片段所屬時段的身分未判定，整個

片段都不生成。在分析流程的示範中，這類片段的名額由下一名遞補。這些檢查都在呼叫模型之前完成。

戰術觀察

逐拍賽評只描述單一拍，因此模型先看過整個回合，提出跨越數拍的觀察，供撰寫賽評時參考。觀察最多 5 則，不預設戰術類別，也可以一則都不提。程式另以規則找出常見的球路模式（例如高遠球後接殺球）作為選用提示，模型可以不採用。

每則觀察都必須引用至少兩拍作為依據，而且只能以編號引用可靠的球種或球場區域，引用內容、擊球者與影格再由程式依編號回填。程式接著逐則核對，編號不存在、引用不允許的欄位、來源不夠可靠或不足兩拍的觀察都會被丟棄。

通過核對的觀察會再送交模型審查。審查時可看到引用範圍內的每一拍與前後各一拍，包含其中未被引用的擊球，藉此抓出把不相連的擊球說成連續等錯誤。沒有觀察通過核對時，略過這次審查，所以一個回合的請求數是 3 或 4 次。引用核對只能確認觀察所依據的事實屬實，無法證明戰術解讀本身正確，因此觀察始終標記為模型詮釋。

撰寫賽評

程式把逐拍事實、可靠的球場區域與通過審查的觀察整理成一份撰寫計畫。模型以一次請求寫完整個回合，輸出只有「擊球編號與句子」以及回合摘要。影格、時間與擊球者由程式依編號接回，模型的輸出也必須剛好涵蓋計畫要求的每一拍。

提示詞限制每則賽評只有一句話，並禁止提到勝負、比分、發球方、得分原因、心理狀態、正反手、球速、落點與連續跑動。球種只能維持 8 類的粒度，例如高遠球不得寫成挑球或長球，因為合併後已沒有這項資訊。另一方面，若禁止一切詮釋，文字會只剩下球種的流水帳，所以「抓準機會」、「順勢」、「變換節奏」等不主張具體事實的轉播用語是允許的。

審查與模板句

生成的文字要通過兩層檢查，只有兩層都通過的句子才會保留。

- **語意審查：**以一次請求審查全部句子。每一句都對照該拍事實、前後各一拍與重疊的戰術觀察，摘要則對照整個回合。模型只判定通過、拒絕或不確定並標出違規類別，不改寫句子。這一層處理字面規則抓不到的問題，例如沒有區域資料卻寫「在網前」。
- **規則檢查：**通過語意審查的句子再由程式檢查是否含有禁用詞（如得分、失誤、正手、球速、落點、跑動、心態）、是否只有一句、謹慎與低信心的拍是否帶不確定語氣，以及

是否出現數字比分或被合併的細分類球種。審查模型也可能誤判，這一層的結果則是確定且可重現的。

未通過的句子改用模板句，例如「球員 b 以小球回擊。」或「球員 a 這拍可能於中場中央以小球回擊。」模板句只讀取事實欄位，不再呼叫模型，也要通過同一套規則檢查。由於已有模板句可以替換，系統不要求模型修改後重試，一句被換掉也不影響同回合的其他句子。摘要沒有模板句，未通過語意審查就直接省略。

表 3.4 是 MK_vs_CT_2019 片段 10 的實際輸出。第 1 拍的信心未達 0.70，模型因此使用「似乎」。這場比賽的球場未經使用者確認，模型沒有球場區域可用，第 3 拍的原句與回合摘要都因描述了沒有依據的位置而被審查拒絕，前者改用模板句，後者則省略。被拒絕的原句不會保留。

表 3.4: 賽評生成的實際輸出範例

拍	擊球者	球種（信心）	最終賽評	來源
1	b	平快球（0.68）	球員 b 似乎擊出了一記平快球。	模型
2	a	切球（0.84）	球員 a 順勢回以切球。	模型
3	b	小球（0.75）	球員 b 以小球回擊。	模板句（審查拒絕）
4	a	小球（0.90）	球員 a 接續放小球還擊。	模型
5	b	小球（0.89）	球員 b 再次輕推小球應對。	模型
6	a	高遠球（0.77）	球員 a 變換節奏送出高遠球。	模型

每一句都記錄所依據的擊球編號。逐拍賽評對應該拍，摘要對應該回合所有已輸出的拍，回看介面以此連回影片。審查只比對管線的結果，不看影片，所以上游判錯時賽評也會跟著錯。例如 ASG_vs_AA_2020 片段 38 的開頭四拍被球種分類判為同一人連續發球，賽評就寫成「球員 b 這裡看來再次送出發球」，只有不確定語氣透露該結果的信心不高。

失敗處理與輸出

文字內容的問題可以如上逐句替換，結構上的問題則讓整個回合失敗，包括 Gemini 請求失敗或逾時（上限 30 秒，不重試）、回應不完整、不符合格式、未剛好涵蓋要求的每一拍，或模板句、已通過審查的摘要未通過規則檢查。此時不寫入部分結果，只留下記錄階段與錯誤原因的診斷檔，其中不含 API 金鑰與提示詞內容。

賽評階段在全部回合完成後才一次寫入 commentary.json。回看介面按需生成的回合則另存為該片段自己的檔案，不更動 commentary.json。同一片段兩者都有時，回看介面顯示按需生成的版本。

3.7 使用者介面

3.7.1 概述

問題與目標：前述各模組的輸出是十多份 JSON 檔，記錄每個回合的起訖、比分、球員骨架、羽球軌跡等。這些資料必須對照影片才看得懂，直接閱讀數字既無法判斷分析是否正確。本系統的使用者介面讓使用者不必操作指令列，就能完成「匯入影片、執行分析、對照影片回看結果」的完整流程，並把逐拍資訊疊回轉播畫面，讓分析結果可以被直接檢視。

使用流程總覽：如圖 3.55。

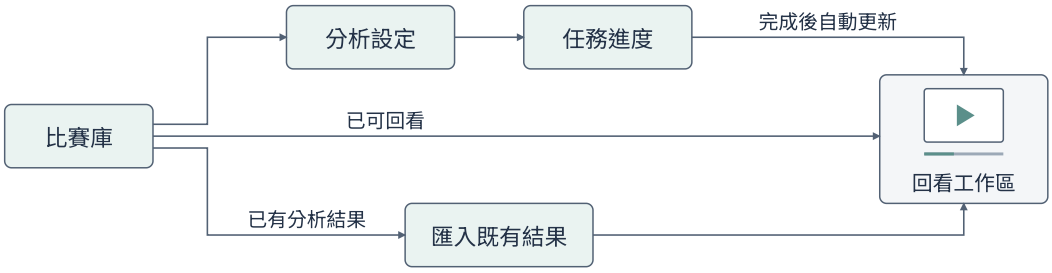


圖 3.55: 使用者介面操作流程

比賽庫列出比賽資料夾中的每一場比賽及其分析狀態（圖 3.56）。尚未分析的比賽從「分析設定」選擇要執行的項目（圖 3.57），分析在背景執行，完成後回看資料自動更新。已在其他環境完成分析的比賽，則可直接匯入既有結果，不需重新計算。

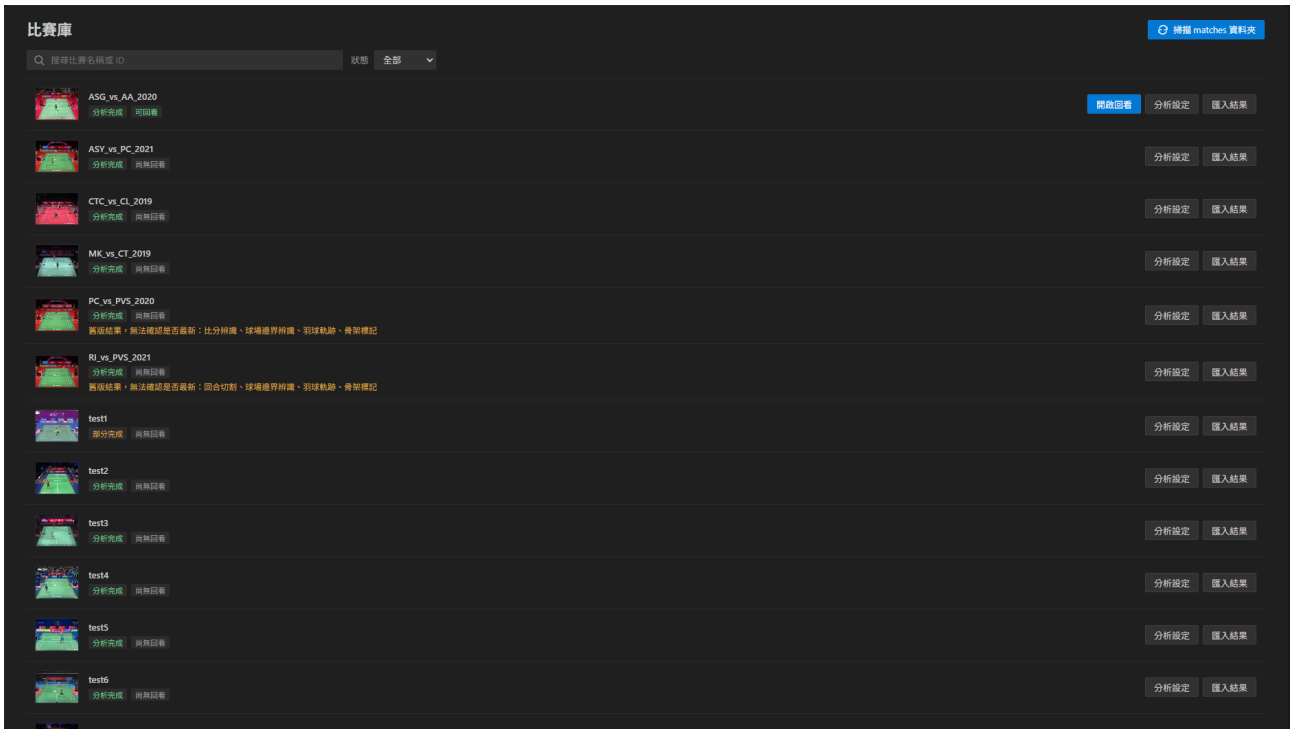


圖 3.56: 比賽庫，每場比賽標示分析狀態與是否可回看

圖 3.57: 分析設定，每個分析項目標示已完成、未執行或已過期

3.7.2 回看工作區

回看工作區是介面的核心，由播放器、回合分析欄與時間軸三個區域組成（圖 3.58）。三者共用同一個影片時間，任一區域的操作都會同步反映到另外兩區。

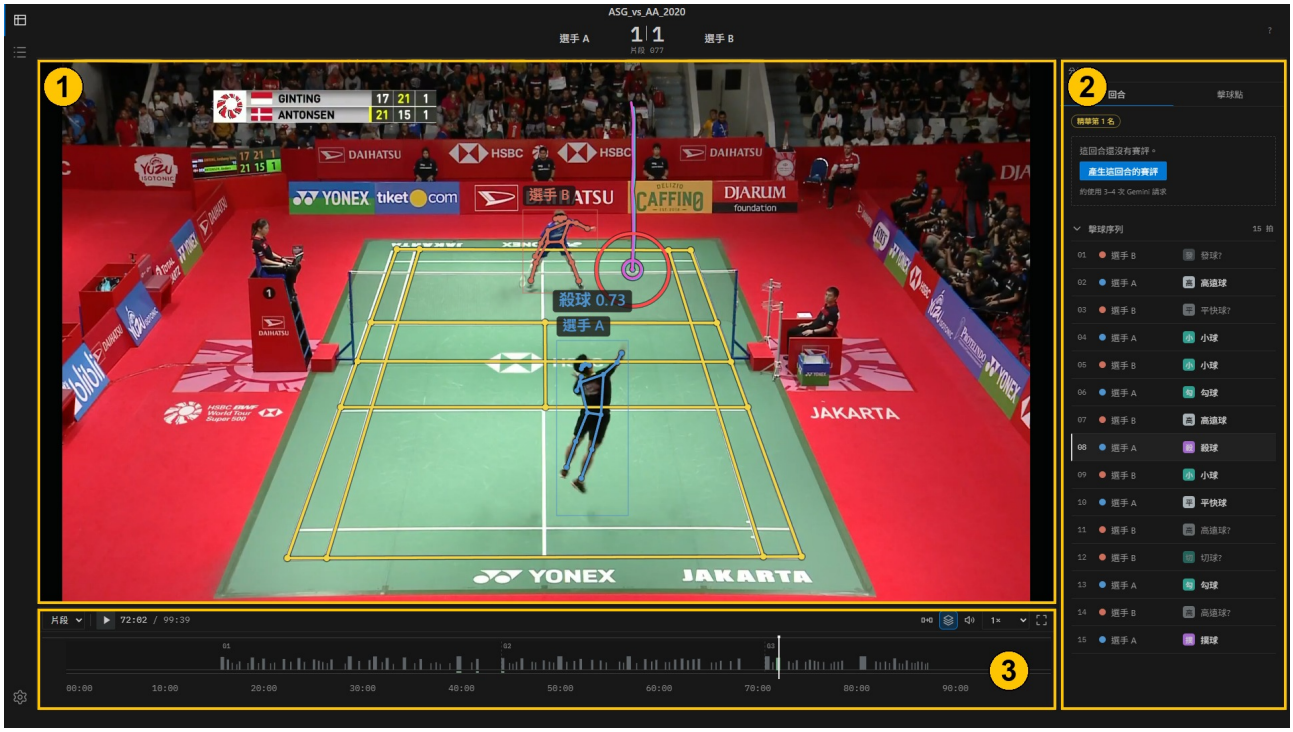


圖 3.58: 回看工作區：① 播放器與疊圖 ② 回合分析欄 ③ 時間軸

播放器與疊圖

播放器在轉播畫面上疊加各模組的分析結果，共 7 個圖層：球場線、骨架、選手標籤、羽球、球路拖尾、擊球閃圈與球種標籤（圖 3.59）。各圖層可以單獨開關，讓使用者直接用眼睛檢查各模組的結果，例如球場線是否貼合場地、骨架是否選對球員、軌跡是否跟著球走。

完整轉播中，回合只佔播出時間的約三成。「只播片段」模式會自動跳過回合之間的重播、休息與觀眾畫面，只播放分段模組切出的比賽片段。

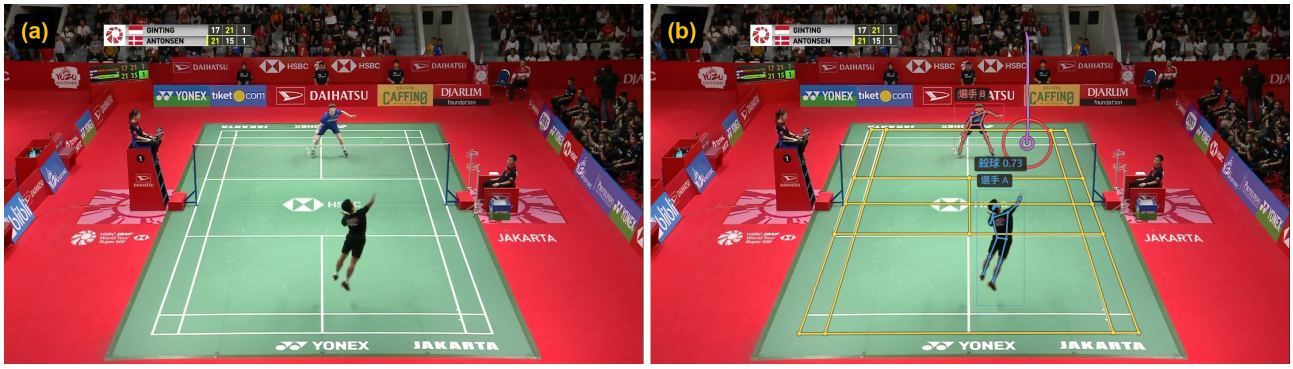


圖 3.59: 疊圖開啟前後：(a) 原始轉播畫面 (b) 開啟全部圖層

回合分析欄

分析欄顯示目前回合的逐拍資訊，分為「回合」與「擊球點」兩個分頁（圖 3.60）。

- **擊球序列**：依序列出每一拍的擊球者與球種，信心度低於 0.5 的球種以淡色加問號標示。點選任一拍，影片即跳到該拍的擊球時刻。
- **精華標記**：精彩度排名在全場前 10% 的回合，會在欄位頂端顯示排名徽章。
- **單回合賽評**：分析流程中的賽評階段只示範生成精彩排名前 3 的回合。其他回合由使用者在想看時按下「產生這回合的賽評」，每次約使用 3–4 次 Gemini 請求。產生後，回合摘要下方以「依據 #1、#2…」列出摘要所引用的擊球，點選即跳到該拍。逐拍賽評則附在擊球序列的每一拍下方，讓使用者能逐句對照影片確認賽評是否有所本。
- **擊球點**：以俯視球場標出本回合每一拍的擊球位置，依擊球順序連線，顏色區分兩名球員，可一眼看出回合中雙方的跑動與落點分布。

時間軸

時間軸提供四種模式，各自呈現一種隨時間變化的分析結果（圖 3.61）。

圖 3.60: 回合分析欄：(a) 擊球序列與精華排名 (b) 產生賽評後 (c) 擊球點分頁

- **片段**：每個片段一根長條，高度為回合長度，並以 G1–G3 標示局數。
- **擊球**：依每片段的球種組成著色。
- **比分**：呈現雙方比分差距隨時間的變化，位於中線上方表示選手 A 領先、下方表示選手 B 領先。
- **歡呼**：顯示各片段的歡呼訊號，並以三角形與數字標示精彩度排名。

時間軸可在「全場」與「單片段」兩種尺度間切換（圖 3.62）。全場尺度用於掌握整場比賽的分布與走勢。雙擊任一片段即放大到該回合，此時每一拍成為一個點並標上球種縮寫，可以看出來回的節奏。

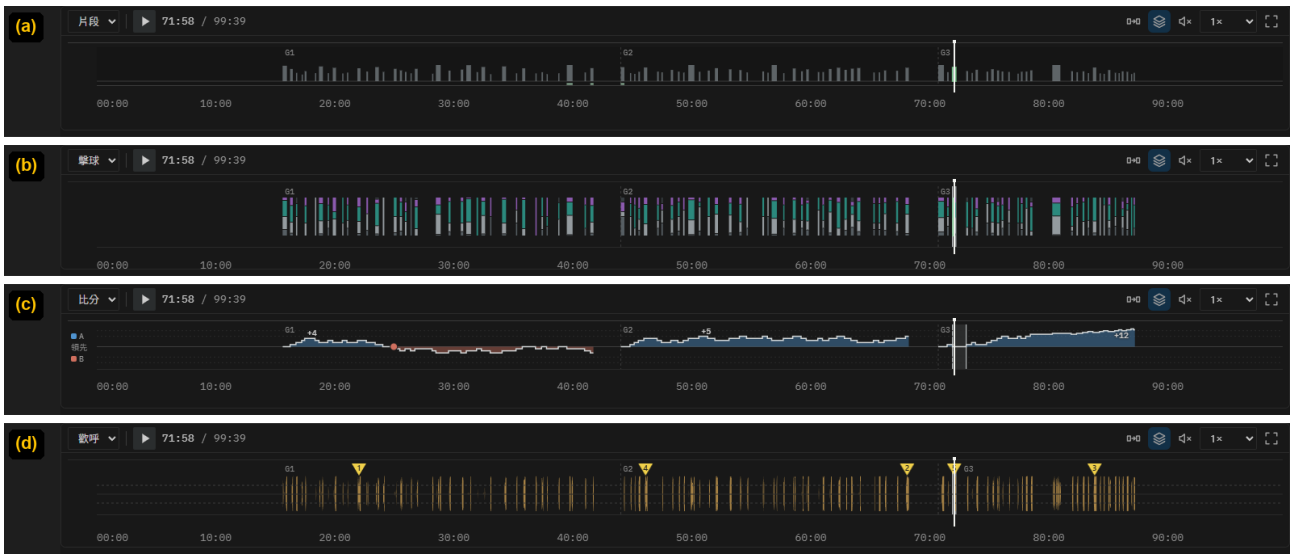


圖 3.61: 時間軸的四種模式（全場尺度）：(a) 片段 (b) 擊球 (c) 比分 (d) 歡呼

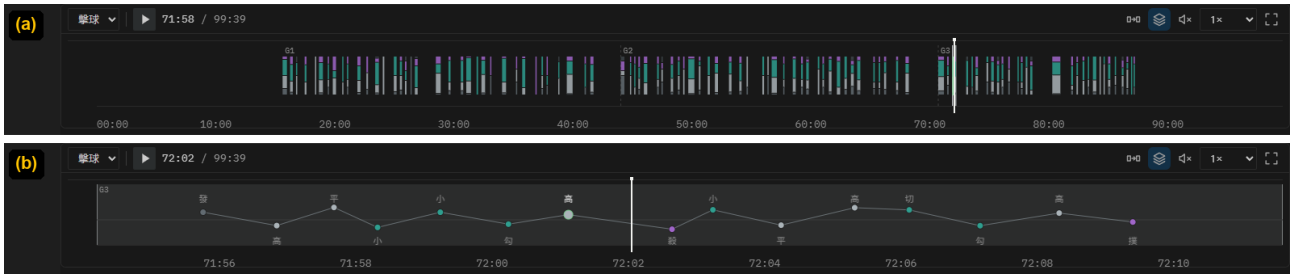


圖 3.62: 擊球模式在兩種尺度下的呈現：(a) 全場 (b) 單回合

3.7.3 分析任務與場地校正

分析計畫預覽：使用者勾選分析項目時，介面即時列出執行計畫：哪些階段會執行、哪些沿用既有結果及其原因，以及計畫是否會呼叫 Gemini API（圖 3.63）。若某些未勾選階段的上游結果會因這次分析而改變，介面也會列出這些將會過期的階段，回看時不會顯示過期的資料。

任務進度：分析在獨立的背景程序中執行。任務頁顯示整體進度、各階段耗時與詳細記錄，使用者也可以隨時取消，取消時已完成的階段會保留，下次分析只需從中斷處繼續（圖 3.64）。

場地校正：場地偵測是後續選人與站位換算的基礎，因此任務會先執行場地偵測，並把需要讀取球場的階段排到最後。球場一偵測完，介面即提示使用者檢查，分析設定頁自動展開場地預覽，其餘階段照常進行。若角點明顯偏移，使用者可直接在球場畫面上拖曳四個外角點，並以方向鍵逐像素微調（圖 3.65）。保存後，尚未開始的階段直接使用新球場，已讀過舊球場的階段則自動重跑。

圖 3.63: 分析計畫預覽

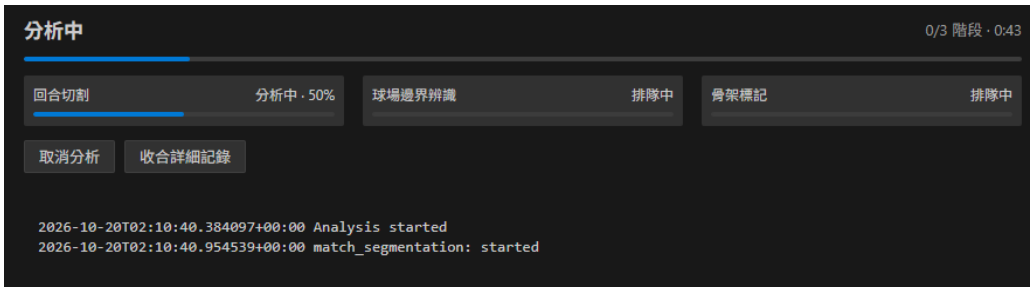


圖 3.64: 任務進度，顯示整體與各階段的進度及詳細記錄

3.7.4 實作概要

介面以 Vue 3 與 TypeScript 開發，並以 Electron 包裝為 Windows 桌面程式。分析本身仍由本機的 Python 服務執行，介面只負責送出分析任務與顯示進度。分析完成後，匯入程式會先驗證各階段的 JSON 是否完整、是否與影片相符，再整合成一份回看資料，介面只讀取這一份資料，不直接讀取各階段的原始輸出（圖 3.66）。骨架、軌跡等資料量較大的疊圖內容則按回合分開儲存，只在使用者打開疊圖時才讀取目前與下一個回合，避免開啟比賽時長時間等待。

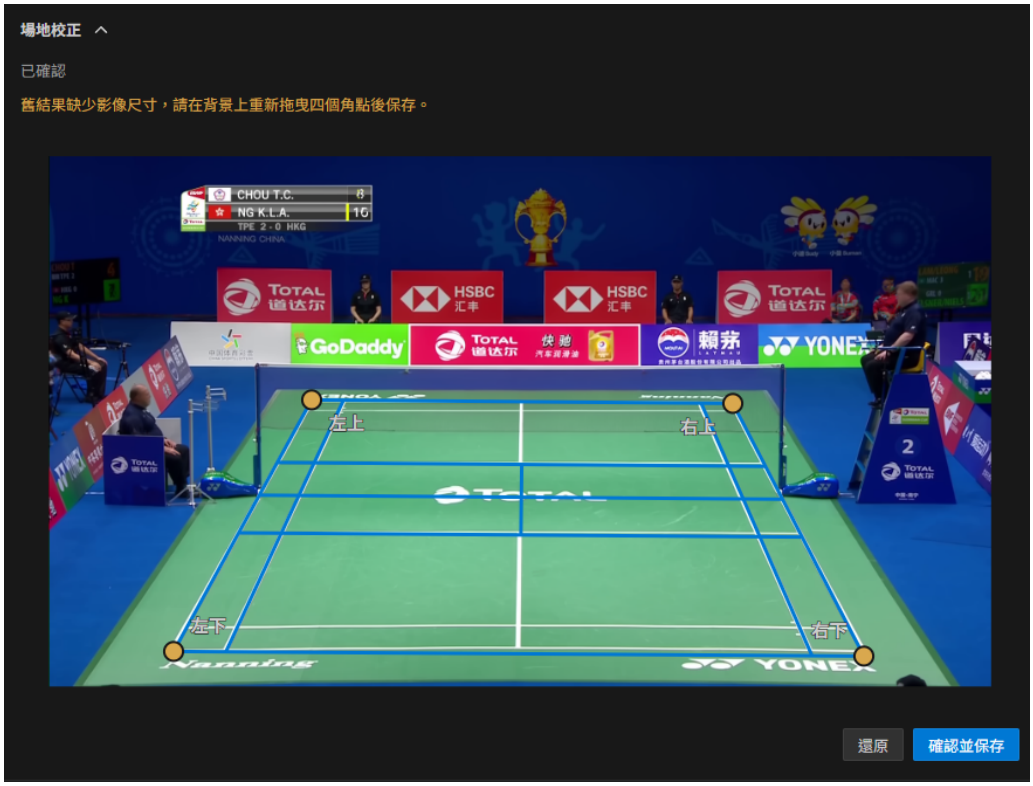


圖 3.65: 場地校正，拖曳四個外角點對齊球場邊線

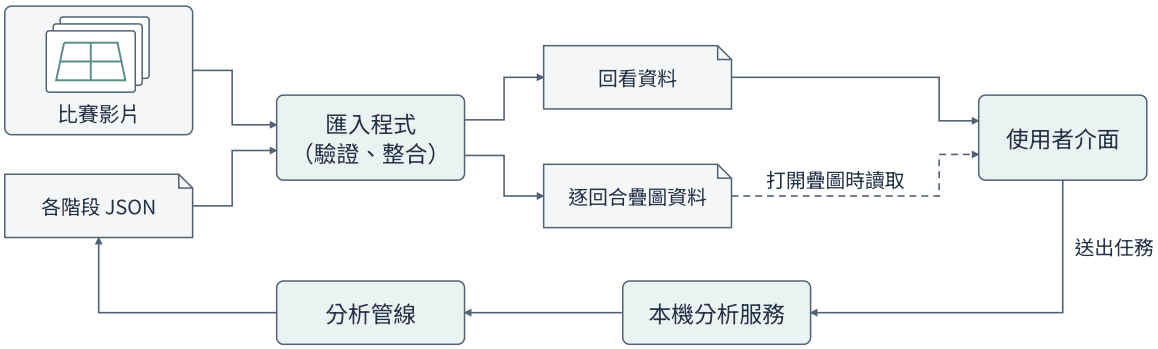


圖 3.66: 使用者介面實作架構

Chapter 4

實驗結果

4.1 資料集與重現方式

本章視覺模組的實驗共用下述影片庫。音訊模型另使用四場開發比賽與一場偵測器外部驗證，資料角色與結果另列於 4.2.5 節。資料集由 16 支 YouTube 羽球單打完整轉播構成，涵蓋 2018 至 2023 年的國際賽事。影片皆為 1920 × 1080，幀率為 25 或 30 fps，編碼格式包含 H.264、AV1 與 VP9。全部影片合計約 18.09 小時。為避免同一場比賽的資料同時出現在開發與評估中，資料以影片為單位切分為 7 場開發集與 9 場測試集（表 4.1）。開發集用於設計與調整規則，測試集不參與開發，僅在完成後用於報告測試結果。由於其中 15 支影片來自 BWF 官方製作，跨轉播單位的泛化能力仍待後續驗證。

表 4.1: 資料集切分

資料分割	影片數	影格數
開發集	7	852,297
測試集	9	851,704
合計	16	1,704,001

為使影格編號可被他人重現，`experiments/dataset/videos.json` 與 `videos.md` 記錄每支影片的 YouTube 來源、解析度、幀率、影格數、編碼、SHA-256 雜湊及影格指紋。下載影片後，應先放至 `matches/<代號>/input/`，並執行下列指令確認檔案與標記時所用影片一致：

```
uv sync
uv run python experiments/dataset/verify_video.py --all
```

驗證程式依序檢查檔案雜湊、影格數／幀率／長度，以及固定影格的畫面指紋。即使檔案經重新壓縮，只要影格序列相同仍可通過指紋檢查。

本章各實驗的真值、評分程式與逐場結果皆置於本系統儲存庫 badminton-analysis-system（`https://github.com/2026bmtproject/badminton-analysis-system`）的 `experiments/` 下，依模組分為子目錄。音訊研究的程式另見 badminton-audio-highlight 儲存庫（`https://github.com/2026bmtproject/badminton-audio-highlight`），重現說明見 `experiments/audio_highlight/README.md`。

4.2 各模組結果

4.2.1 賽事影片分段

在 9 場未參與開發的比賽、670 個真值片段上，本模組在 IoU ≥ 0.9 下達到 Precision 98.37%、Recall 98.96%，邊界誤差中位數為 0 影格。16 場、1354 個片段的整體 F1 為 99.01%，其中 11 場零錯誤。分段結果會直接決定下游分析範圍，因此本節分別驗證片段偵測、邊界精度，並分析所有失配案例。

本模組的真值定義為「固定主機位之連續比賽畫面」的起訖影格，起訖兩端均包含。所有真值由同一位標記者完成。

偵測準確率

配對規則為 IoU ≥ 0.9、一對一，未配對的輸出段計為 FP、未配對的真值段計為 FN（表 4.2）。

表 4.2: 賽事影片分段的偵測準確率（%）

範圍	真值	輸出	配對	FP	FN	Precision	Recall	F1
開發集	684	687	681	6	3	99.13	99.56	99.34
測試集	670	674	663	11	7	98.37	98.96	98.66
合計	1354	1361	1344	17	10	98.75	99.26	99.01

16 場中有 11 場零錯誤，全部 27 筆失配集中在 5 支影片上。

邊界精度

誤差定義為「輸出 − 真值」，單位為影格，只計算配對成功的段。起點為正代表系統比真值晚開始（切掉了開頭），終點為負代表系統比真值早結束（切掉了結尾）。

1344 個配對段中沒有任何一段的邊界誤差超過 10 影格（表 4.3）。只要一個回合被完整切出來，它的邊界幾乎一定是準的。

表 4.3: 賽事影片分段的邊界誤差（影格）

範圍	起點中位數	起點 MAE	起點最大偏移	終點中位數	終點 MAE	終點最大偏移
開發集	+0.0	0.31	+6	+0.0	0.01	−3
測試集	+0.0	0.19	+4	+0.0	0.05	−10
合計	+0.0	0.26	+6	+0.0	0.03	−10

失敗案例與限制

27 筆失配可歸為三類，且各自集中於特定影片（表 4.4）。其中漏掉表示回合未被完整切出，多切則表示產生額外片段。

表 4.4: 賽事影片分段的失配成因

成因	多切	漏掉	影響的比賽
一、播報圖卡動畫把回合切成碎片	11	6	TTY_vs_ASY_2023、test1、test2、test3
二、全域亮度變化使跨片段篩選失效	1	2	test4
三、門檻邊緣的高強度回合被切斷	5	2	test6
合計	17	10	

一、播報圖卡動畫：轉播在回合開始或結束時播放球員名條、比分卡等動畫，使 MAD 超過門檻，將回合切成碎片。動畫持續 3–6 秒，遠超過邊界精修可橋接的 12 影格，因此無法重新連接。

二、全域亮度變化：test4 開賽後前兩個回合整段消失。MAD 中位數與正常回合相近，表示門檻判斷本身沒有問題。真正失效的是跨片段一致性篩選。由於開賽初期場館燈光逐漸變亮，兩段畫面與其他回合雖然構圖相同，整體像素差卻明顯較高，因而被誤判為非比賽畫面。此類錯誤表現為整段消失，而非邊界偏移。

三、門檻邊緣的高強度回合：test6 有兩個高強度回合被切成數段，其中一個 11.4 秒的回合被切成三段。切斷處與保留區段的 MAD 均接近門檻，顯示球員劇烈移動使畫面差異在門檻附近波動。由於目前採用全片固定門檻，無法適應此類時間上的動態變化。

已知但未在本資料集上觸發的限制：若影片包含大量且相似的非比賽畫面，跨片段一致性篩選可能失效。系統亦假設存在穩定主機位，不適用於手持或多機快切影片。此外，16 支影片中有 15 支來自 BWF 官方製作，因此跨轉播單位的泛化能力尚未驗證。

4.2.2 場地偵測

在 9 場未參與開發的比賽、36 個球場外角上，本模組的角點定位誤差中位數為 2.79 像素，命中率 72.22%，9 場中有 6 場四角全中。16 場、64 個外角的整體誤差中位數為 3.05 像素，命中率 76.56%，11 場四角全中。

真值定義：球場外緣是有寬度的白線，邊線與底線在外角交疊成一個小四邊形，本節稱為「角區」。標記時不直接點選角點，而是在合成圖上標出外框四條線各自的外緣與內緣，共 8 條邊緣，每個角區由相交的兩條底線邊緣與兩條邊線邊緣的 4 個交點構成。所有真值由同一位標記者完成。

定位誤差

誤差定義為偵測角點到角區中心的歐氏距離，單位為像素（表 4.5）。

表 4.5: 場地偵測的角點定位誤差（像素）

範圍	角數	遠端中位數	近端中位數	全部中位數	全部平均	全部最大
開發集	28	0.97	3.59	3.34	31.95	395.72
測試集	36	1.00	4.83	2.79	33.71	440.27
合計	64	1.00	3.62	3.05	32.94	440.27

中位數與平均值相差十倍，顯示誤差分布呈兩極，多數角點的誤差只有數像素，少數完全失敗的角則偏離數百像素。近端的像素誤差大於遠端，推測是鏡頭畸變使部分影片中近端底線彎曲造成。

命中率

命中定義為偵測角點落在角區內，或在角區外緣 2 像素容差內（表 4.6）。

表 4.6: 場地偵測的命中率

範圍	場數	遠端命中	近端命中	全部命中	命中率（%）	四角全中場數
開發集	7	11/14	12/14	23/28	82.14	5/7
測試集	9	14/18	12/18	26/36	72.22	6/9
合計	16	25/32	24/32	49/64	76.56	11/16

若不給容差、只計嚴格落在線寬內的角點，命中為 33/64（51.56%）。有 16 個角點落在角區外緣 2 像素內，這類偏移已小於一條白線的寬度，在後續換算球場座標時可視為已對準。

失敗案例與限制

15 個未命中的角中，14 個來自同一根因，底線旁的地面廣告字樣在線段分群時與底線合併，導致代表線偏移（表 4.7）。

底線被廣告字樣污染：轉播場館常在場地墊印製贊助商字樣，距底線僅約 10–40 像素。由於字樣位於球場遮罩內，白色字母會被視為線像素，並經霍夫轉換產生大量與底線平行的短線段，其方向正確，因此無法由消失點過濾排除。

表 4.7: 場地偵測的未命中成因

成因	未命中角數	到角區中心距離	影響的比賽
底線被廣告字樣污染，收縮至內部矩形	8	17–440 px	CTC_vs_CL_2019、test5
底線被廣告字樣污染，外擴至字樣邊緣	6	17–70 px	test8、test6
遠端角輕微偏移	1	10 px	ASG_vs_AA_2020
合計	15		

將人工標記的真值四角直接代入同一套吻合度評分，test5 與 CTC_vs_CL_2019 分別得到 0.825 與 0.922，明顯高於實際錯誤解的約 0.45，顯示評分本身能辨識正確解，問題主要發生在候選生成階段。

進一步僅修改線段分群方式並重跑 16 場，將合併距離由 14 像素縮為 6 像素，或改以最長線段為錨點並依線段長度加權擬合，命中數皆由 49/64 提升至 61/64，且上述四場全部恢復命中。由於這些修正是在觀察測試集失敗後才提出，因此不能將修正後結果視為未參與開發的測試結果，修正列為後續改進方向。

系統無法察覺自身失敗：三場四角全錯的比賽中，模組皆正常輸出四角與單應性，未標記為偵測失敗。原因是吻合度目前僅用於候選間比較，沒有絕對門檻判斷最佳解是否可信，因此錯誤結果仍會傳入下游模組。

已知但未觸發的限制：本模組假設機位固定，且球場大致位於畫面中央偏下，並從該區域取樣參考色。若機位偏移使取樣區域落在場外，顏色分割可能整體失效。

4.2.3 擊球時刻偵測

在 8 場未參與開發的比賽、6990 拍上，本模組在 ±5 幀容差下達到 Precision 95.49%、Recall 90.90%，配對成功的擊球時間誤差中位數為 −1 幀。14 場、12163 拍的整體 F1 為 92.10%。

真值來源：本模組的真值取自公開資料集 ShuttleSet [1]（MIT 授權），以原檔形式收錄。16 場中有 14 場在 ShuttleSet 內，因此本節的開發集為 6 場、測試集為 8 場。答案幀原樣使用，唯一的調整是 `test5` 與 `test8`，我們取得的影片版本開頭多出一段賽前畫面，整體差了整數分鐘（+15000 與 +12000 幀），先予補回。

評分範圍：真值只涵蓋 ShuttleSet 收錄的回合，系統輸出也只存在於分段模組切出的片段內。因此只在兩者重疊的時間評分，以每個回合的第一拍到最後一拍（前後各放寬容差 tol）與片段取交集。全真值中有 97.9% 落在評分範圍內。

偵測準確率

配對規則為偵測幀與答案幀相差 ≤ 5 幀、一對一，由相差最小者開始配對。未配對的偵測計為 FP，未配對的答案計為 FN（表 4.8）。

表 4.8: 擊球時刻偵測的準確率（%，容差 ±5 影格）

範圍	真值	輸出	配對	FP	FN	Precision	Recall	F1
開發集	5173	4981	4605	376	568	92.45	89.02	90.70
測試集	6990	6654	6354	300	636	95.49	90.90	93.14
合計	12163	11635	10959	676	1204	94.19	90.10	92.10

容差放寬或收緊時的變化如表 4.9。

表 4.9: 不同容差下的擊球時刻偵測準確率（%，全部 14 場）

容差（影格）	Precision	Recall	F1
3	89.07	84.16	86.54
5	94.19	90.10	92.10
10	95.68	92.65	94.14

時間誤差

誤差定義為「偵測 − 答案」，單位為影格，只計算配對成功的擊球（表 4.10）。

表 4.10: 擊球時刻偵測的時間誤差（影格）

範圍	中位數	平均	MAE
開發集	−1	−1.46	1.80
測試集	−1	−1.15	1.55
合計	−1	−1.28	1.66

以毫秒計，整體誤差中位數為 −40 ms，MAE 為 64.6 ms。多數擊球被系統抓在真值前 1 至 2 影格，推測是因為本模組偵測的是軌跡轉折點，而人工標記的是球拍接觸的畫面。

失敗案例與限制

以下分析測試集的 636 個 FN 與 300 個 FP，根據失配前後 ±5 影格的模組內部紀錄，將失配依成因分類（表 4.11）。

最大宗的失配為時間偏移，FN 的 30%、FP 的 49% 屬於系統已在正確位置附近偵測到，但與真值相差 6–10 影格，因此同一拍同時形成 FN 與 FP。將容差由 5 影格放寬至 10 影格後，F1 由 92.10% 提升至 94.14%，顯示部分誤差來自擊球時間定位差異。

表 4.11: 擊球時刻偵測的失配成因（測試集）

成因	FN	FP
抓到但時間差 6–10 影格	192	148
遠端擊球在影像中沒有轉折	159	—
篩選與修剪規則誤刪	182	—
發球未產生候選	63	—
同一拍重複偵測	—	74
其他	40	78
合計	636	300

FN 主要來自兩類。其一是遠端擊球缺乏軌跡轉折，部分遠端球員擊球時，球在影像中的運動方向未改變，僅速度微小改變，因此無法觸發以軌跡轉折為核心的候選訊號。其二是候選遭後續規則刪除，主要涉及同側去重、回合脈絡判斷與尾段修剪。

FP 則主要來自同一拍的重複偵測。同一擊球可能產生多個候選，若其擊球側判斷不同，現有去重規則無法合併。其餘補漏規則誤觸發與球體追蹤雜訊合計僅占少數。

本實驗的評分範圍只延伸至回合最後一拍後 5 影格，因此無法完整評估回合結束後落地彈跳造成的誤判，以及尾段修剪規則的實際效果。

4.2.4 球種分類

在 3 場 BST 未曾看過的比賽、1601 拍上，本模組的 8 類球種準確率為 87.38%，Macro-F1 為 84.71%，擊球者（上方／下方）準確率為 97.88%。若把擊球時刻偵測漏掉的拍也算錯，端到端準確率為 78.99%。

真值來源：本節與擊球時刻偵測使用同一份 ShuttleSet 真值，評分範圍與配對規則也完全相同。ShuttleSet 的 18 種球種先依 BST 訓練時的方式併為 12 類，再併為系統輸出的 8 類。

BST 看過的比賽：本模組使用的 BST 權重即以 ShuttleSet 訓練 [4]。依 BST 公開的切分，14 場中有 11 場屬於其訓練集，在這些比賽上量得的準確率可能偏高，只有 3 場未參與 BST 訓練。因此本節以這 3 場為主要結果，其餘 11 場僅作對照。

分類準確率

表 4.12: 球種分類的準確率（%）

範圍	計分拍數	球種 Acc	Macro-F1	端到端 Acc	擊球者 Acc
BST 沒看過（3 場）	**1601**	**87.38**	**84.71**	**78.99**	**97.88**
BST 看過（11 場）	9333	88.32	86.36	80.06	98.02
合計（14 場）	10934	88.18	86.22	79.90	98.00

BST 看過與沒看過的兩組只相差約 1 個百分點（表 4.12），可見在 BST 看過的比賽上量得的結果沒有明顯偏高。推測原因是 BST 訓練時的骨架與球軌跡由其原作者的流程產生，本系統則由自己的姿態與球體追蹤模組產生。

沒看過的 3 場各球種表現如表 4.13。

表 4.13: 各球種的分類表現（BST 沒看過的 3 場，%）

球種	真值拍數	Precision	Recall	F1
高遠球	531	92.28	90.02	91.13
發球	93	98.81	89.25	93.79
小球	387	95.03	88.89	91.86
平快球	136	59.63	70.59	64.65
殺球	174	87.85	91.38	89.58
切球	189	85.71	88.89	87.27
撲球	19	93.33	73.68	82.35
勾球	72	75.00	79.17	77.03

上游影響

為區分錯誤來自擊球時刻還是 BST 本身，另將擊球幀換成真值的擊球幀重跑一次球種分類，其他輸入（骨架、球軌跡、場地、權重）不變（表 4.14）。

表 4.14: 以系統偵測或真值擊球幀進行球種分類的比較（%）

擊球幀	計分拍數	球種 Acc	Macro-F1	擊球者 Acc
系統偵測	1601	87.38	84.71	97.88
真值	1771	88.54	84.98	98.54

即使擊球時刻完全正確，準確率也只提升 1.16 個百分點。

失敗案例與限制

沒看過的 3 場中共有 202 拍判錯，依重跑結果分類如表 4.15。

表 4.15: 球種分類的錯誤成因（BST 沒看過的 3 場）

成因	拍數
BST 本身判錯（換成真值擊球幀仍錯）	146
擊球時刻所致（換成真值擊球幀即正確）	48
判成未知球種	8
合計	202

BST 本身的錯誤集中在軌跡相近的球種之間。最多的是高遠球與平快球互判（49 + 26 拍），其次是小球與勾球（19 + 10 拍）和殺球與切球（14 + 10 拍）。

4.2.5 歡呼偵測與精彩度分析

歡呼偵測資料集與評估方式

歡呼偵測任務以 3 秒窗口為單位，資料來自四場開發比賽。每場以固定 seed 42 的 segment-diverse deterministic sampling 選取 100 個窗口，再由人工進行 blind labeling（表 4.16）。標註允許 ambiguous；二元分類資料會排除 ambiguous，本次四場凍結資料沒有 ambiguous 樣本。

表 4.16: 歡呼偵測的標註資料

比賽	歡呼	非歡呼	合計
match_001	33	67	100
match_002	31	69	100
match_003	55	45	100
match_004	31	69	100
合計	150	250	400

評估採 Leave-One-Match-Out（LOMO）：每一 fold 以三場共 300 個窗口訓練，剩餘一場的 100 個窗口作為測試資料。StandardScaler 只在該 fold 的訓練資料上 fitting，再套用到 held-out match。相較隨機拆分窗口，此方法以 match ID 隔離訓練與測試資料，避免同場相鄰窗口分散到兩邊；目前缺少完整賽事來源對照，尚不能據此宣稱跨轉播商的獨立性，但四場比賽仍不足以證明模型可普遍泛化。

跨比賽歡呼偵測結果

表 4.17: 歡呼偵測的逐場交叉驗證結果（LOMO）

測試比賽	Accuracy	Precision	Recall	F1	ROC-AUC	AP	[[TN,FP],[FN,TP]]
match_001	0.940	0.966	0.848	0.903	0.990	0.983	[[66,1],[5,28]]
match_002	0.870	1.000	0.581	0.735	0.934	0.904	[[69,0],[13,18]]
match_003	0.870	0.850	0.927	0.887	0.958	0.968	[[36,9],[4,51]]
match_004	0.850	0.711	0.871	0.783	0.947	0.907	[[58,11],[4,27]]
Macro 平均	0.883	0.882	0.807	0.827	0.957	0.940	—

四場 LOMO 的 macro Accuracy 為 0.883、macro F1 為 0.827、macro ROC-AUC 為 0.957（表 4.17）。0.957 是 ROC-AUC，不是 95.7% accuracy。各場 Brier score、Log Loss 與 ECE 顯示 score calibration behavior 存在差異；本研究沒有進行 recalibration，門檻仍固定為 0.5，因此分類器輸出不視為已校準機率。

校準診斷採 10 個等寬 bins，每場 100 個窗口；結果只用於檢查分數分布，沒有重新擬合校準器（表 4.18）。

另以未參與上述四場訓練的 match_005 進行偵測器外部驗證。100 個窗口包含 34 個正例與 66 個負例，Accuracy 為 0.840，Precision、Recall 與 F1 均為 0.765，ROC-AUC 為 0.930，Av-

表 4.18: 歡呼分類器的校準診斷

比賽	Brier score	Log Loss	ECE
match_001	0.038	0.125	0.037
match_002	0.112	0.480	0.131
match_003	0.101	0.340	0.096
match_004	0.121	0.528	0.132

erage Precision(AP)為 0.862。match_005 只對偵測器是 untouched external validation；它後續又參與強度與 segment 聚合方法的開發，因此不能用來證明後兩者的 external generalization。

為比較特徵路線，研究亦評估 RMS、YAMNet 原生 Cheering 分數、YAMNet crowd-combination 分數與 YAMNet embedding + Logistic Regression（表 4.19）：

表 4.19: 不同歡呼特徵路線的比較（LOMO）

方法	LOMO ROC-AUC	LOMO AP
RMS	0.920	0.876
YAMNet Cheering 分數	0.839	0.750
YAMNet crowd-combination	0.857	0.757
YAMNet embedding + Logistic Regression	**0.957**	**0.940**

此比較支持 production 採用 embedding + Logistic Regression，而非直接使用 YAMNet 原生類別分數。這些數字是描述性比較，未執行 threshold tuning 或統計顯著性檢定。

最終部署模型再用四場的全部 400 個窗口 fitting scaler 與分類器；此 final fit 不是新的測試結果。LOMO 的 scaler 則各自只用當 fold 的 300 個訓練窗口。

歡呼強度方法選擇

強度實驗使用 match_005 中 34 個已確認的歡呼窗口。單一主要標註者在不查看候選特徵值的情況下完成 60 組配對比較，其中 39 組有明確較強者、21 組平手、0 組無法判斷(表 4.20)。

表 4.20: 歡呼強度候選方法與人工配對比較的一致性

候選方法	Strict concordance	Coverage	Tie-adjusted concordance
log_rms_db	0.692	1.000	0.692
cheer_probability	0.667	1.000	0.667
temporal_occupancy	0.457	0.897	0.462
relative_loudness_db	0.436	1.000	0.436

log_rms_db 的描述性 concordance 最高且 coverage 完整，也能將分類器信心與聲學強度分開，因此選為凍結的 intensity feature。然而，它與 cheer_probability 的差距不大，實驗沒有顯著性檢定，而且同一窗口會出現在多組配對中，60 組觀察值並不獨立。因此，此結果不能解讀為 log_rms_db 在其他資料上必然優於分類器分數。

Segment 聚合方法選擇

Segment 聚合實驗使用 `match_005` 的 106 個 segments、1142 個重疊窗口與 420 個 detector-positive 窗口。人工 segment detection 標註包含 75 個 cheer 與 31 個 no-cheer segments；尖峰強度實驗則由 60 組人工配對構成，其中 45 組有明確結果、15 組平手（表 4.21）。

表 4.21: 片段層級聚合方法的比較

方法	偵測 ROC-AUC	偵測 AP	Peak-pair concordance	移除單一窗口的最大分數差異，中位數 / Q95（偵測）
mean	0.624	0.749	0.600	0.066 / 0.144
max	0.710	0.862	0.711	0.005 / 0.489
p95 linear	0.718	0.863	0.711	0.021 / 0.391
top-2 mean	0.719	0.867	0.733	0.026 / 0.430
top-3 mean	0.717	0.862	0.733	0.134 / 0.333

Max 與 p95 是預先指定的主要候選，mean 是參考方法，top-2 與 top-3 則是 secondary diagnostics。雖然 top-2 與 top-3 的部分 point estimates 較高，但不能在看到結果後將 secondary diagnostics 改稱勝出方法；各候選也沒有統計上的 winner claim。

最終選擇 linear p95 是工程折衷，而非實驗證明它是最佳聚合器。P95 保留對局部尖峰的敏感度，其人工 detection 與 peak-pair 表現和 max 接近；在移除單一窗口的診斷中，p95 的極端尾端敏感度又低於 max。因此，正式系統以 p95 同時聚合窗口機率與非空值強度。

本節數字逐項核對既有研究 JSON，未重跑模型或重新標註。Intensity strict concordance 以特徵可區分的人工明確配對為分母，coverage 表示明確配對中可區分的比例；人工平手不納入該分母。Segment peak-pair 比較的 45 組明確配對皆可由各候選區分。

4.3 端到端結果

本節時間表沿用既有七個視覺階段的量測，未計入本次新增說明的音訊量測、精彩度排序、身分對應與賽評生成成本，不能用作完整十一階段流程的耗時。

在 5 支影片中，回合總長 1.63 小時，整條管線從空快取開始執行共耗時 4.42 小時。主要計算只作用在回合片段，因此處理時間以回合總長衡量，E2E 時間為回合總長的 2.71 倍，扣除比分辨識等待 Gemini API 的時間後為 1.96 倍。

量測方式：每場另建只含原始影片的資料夾，所有 stage 都從空快取開始實際執行。E2E 時間由外層計時包住 runner 取得，各 stage 時間取自各自的狀態紀錄。比分辨識的耗時主要取決於 API 速率上限而非本機效能，因此另列扣除該 stage 後的「本機時間」。賽評不在預設管線內，不列入。測試環境為 Windows 11 與 RTX 3060 Ti（8 GB）。

4.3.1 處理時間

表 4.22: 端到端處理時間

比賽	編碼／幀率	片長（分）	回合總長（分）	E2E（分）	本機（分）	E2E ÷ 回合總長	本機 ÷ 回合總長
test1	VP9 ／ 25	8.2	5.3	19.5	10.8	3.65	2.02
test5	H.264 ／ 25	57.3	15.0	38.0	26.8	2.53	1.79
test4	VP9 ／ 25	71.9	18.3	42.5	32.9	2.32	1.80
test2	H.264 ／ 25	100.9	31.4	74.9	57.2	2.38	1.82
TTY_vs_ASY_2023	AV1 ／ 30	94.0	27.8	90.1	63.8	3.24	2.29
合計		332.3	97.8	264.9	191.5	2.71	1.96

以回合總長正規化後，本機時間在 3 場 25 fps 的完整比賽間相當一致（表 4.22）。TTY_vs_ASY_2023 為 30 fps，同樣長度的回合多出 20% 影格，加上 AV1 解碼較重，比例較其他場高。

4.3.2 時間分布

表 4.23: 各階段佔總處理時間的比例

stage	佔比
`shuttle_tracking`	34.6%
`score_recognition`	27.8%
`pose`	27.6%
`match_segmentation`	7.0%
其餘 6 個 stage	3.0%

前三個 stage 合計約 90%（表 4.23）。球體追蹤與骨架追蹤的耗時與回合影格數成正比，在 25 fps 的影片上分別約為每影格 33 ms 與 27–31 ms。

4.3.3 限制

- **偶發崩潰：**TTY_vs_ASY_2023 第一次執行時，骨架追蹤在開始約 11 秒後發生原生程式崩潰，重跑後正常完成，表中為重跑結果。原因尚未查明。
- **量測範圍：**每場只執行一次，未估計量測變異，也只在單一硬體上測試。

4.3.4 應用於延遲直播的可能性

以上結果顯示，系統稍加調整後，或許能用於延遲一段時間播出的比賽直播。本機計算時間約為回合長度的 1.8–2.3 倍，而完整比賽中回合只佔播出時間的 25–31%，平均而言處理

速度跟得上播出，因此若改為逐回合處理，計算量不會隨比賽進行而累積。以本機時間約為回合長度的 1.8–2.3 倍估算，最長的 55.7 秒回合約需 2 分鐘以上為依本次量測推估的結果，尚未實作驗證。

Chapter 5

討論與結論

5.1 系統優點與限制

優點：系統的設計特點已於 1.4 節說明，第四章的實驗進一步證實了自動化的上游步驟在未參與開發的比賽上依然有效。賽事影片分段與擊球時刻偵測在測試集上的 F1 分別為 98.66% 與 93.14%，與開發集的 99.34% 與 90.70% 相當。

限制：

1. **依賴對轉播形式的假設：**多數規則建立在 BWF 轉播的慣例上，固定俯拍主機位、球場大致位於畫面中央偏下，且只處理單打。在符合假設的影片上準確率高，但假設一旦不成立，模組可能直接失效而非逐漸退化，例如手持或多機快切的影片無法分段，機位偏移可能使場地顏色分割整體失效。資料集 16 支影片中有 15 支來自 BWF，這類情況在實驗中幾乎沒有機會出現，此部分泛化能力尚未驗證。

2. **規則有過擬合的跡象：**分段、場地偵測與擊球偵測都包含許多依開發集調整的門檻與規則。分段與擊球偵測在測試集上的表現與開發集相近，但場地偵測的命中率由開發集的 82.14% 降至測試集的 72.22%，失敗幾乎都來自開發時未遇過的底線廣告字樣。規則愈多，遇到新場館或新轉播風格時愈可能出現未預期的交互作用。本系統以規則取代標記訓練，避開了第二章指出的標記成本，但依開發集調出的門檻同樣可能隨轉播形式失準。在資料集以 BWF 轉播為主的情況下，尚無法證明此路線比學習式方法更能泛化。

3. **無法察覺自身失敗：**場地偵測在三場四角全錯的比賽中仍正常輸出，分段也可能整段漏掉回合而不留任何警示。系統缺乏信心度判斷，目前只能靠使用者回看時發現並手動修正。

4. **外部依賴與處理速度：**比分辨識與賽評依賴 Gemini API，前者受速率上限影響，約占處理時間的 28%。處理時間約需回合總長的 2.7 倍，且未含音訊、身分對應與賽評。若要用於延遲直播，仍需改為逐回合處理，並降低對 API 的依賴。

5. **音訊支線的證據有限：**歡呼偵測只以四場開發比賽與一場外部驗證評估，不足以涵蓋不同轉播商、場館、觀眾規模與混音方式。該外部驗證場之後又參與了強度與聚合方法的開發，而這兩項的人工比較樣本少、只有單一標註者，因此只足以支持第一版的方法選擇。此外，歡呼強度是單場內的相對排名，不能跨比賽比較。歡呼也不等同回合的技術品質，可能同時受比分、球星或爭議影響，加上精彩度分數的 0.5 ／ 0.5 權重沒有人工排序真值支持，只能作為補充的排序訊號。

6. **缺乏使用者評估：**介面與賽評目前只經開發者自行使用，沒有對教練、球員等目標使用者進行可用性測試，本報告也未量化評估生成賽評的品質，系統實際能減少多少人工逐拍標記的工作仍待驗證。

5.2 結論

本專題建立了一套全自動的羽球單打轉播分析系統。系統輸出每個主機位比賽片段的起訖時間與比分、每一拍的擊球影格、擊球者與球種，以及雙方球員的骨架、球場範圍與羽球軌跡，並據此產生精彩片段排序與賽評。既有方法所依賴的比賽片段起訖、球場範圍與擊球時刻，以及既有系統未曾直接讀取的計分板比分，在本系統中都已自動取得。在未參與開發的比賽上，賽事影片分段與擊球時刻偵測的 F1 分別達 98.66% 與 93.14%，管線已能在真實轉播影片上從頭到尾自動完成。最弱的一環是場地偵測，測試集的命中率為 72.22%，未命中的角幾乎都源於底線旁的廣告字樣。

綜合各模組的實驗結果，上游步驟自動化之後，球種資料的瓶頸落在兩處。一是 BST 本身，將擊球幀換成真值後準確率只提升 1.16 個百分點，未看過的 3 場中 202 拍錯誤有 146 拍來自 BST。二是擊球時刻偵測的召回率，漏掉的拍無從分類，使端到端準確率比單看分類低約 8 個百分點。此外，系統無法察覺自身的失敗，場地偵測在三場四角全錯的比賽中仍正常輸出，錯誤會靜默傳入下游，目前只能靠人工回看發現。因此，除了提升單一模組的準確率，讓系統能判斷自身輸出是否可信同樣重要。

未來工作將針對上述限制展開。首先，蒐集非 BWF 轉播與更多場館的影片，驗證並改善泛化能力，同時為各模組加入信心度估計，在輸出可疑時提示使用者回看。其次，降低對外部 API 的依賴並改為逐回合處理，實現延遲直播的應用。此外，為比分辨識與球員身分對應建立真值並量化評估，擴充歡呼偵測資料，並建立精彩度的人工排序真值。最後，對教練與球員進行使用者研究，量化評估系統與生成賽評的實際效益。

人工智慧的使用

開發過程中的生成式 AI

使用的工具：主要使用 Claude Code（Claude Opus 4.8、5.5），讓模型直接在專案目錄中讀寫程式與文件；另以 ChatGPT（Terra 5.6）作為不同來源的第二個模型，用於討論與交叉檢查。

程式：系統的程式碼幾乎全由 AI 撰寫。組員負責決定架構與設計，包括模組切分、階段之間的資料契約、採用哪些既有模型、實驗的評估方式，以及介面的互動與視覺設計，實作則交給 AI。實際流程通常是先請 AI 提出方案而不動手，由組員取捨後再實作，完成後由 AI 列出需要人工確認的項目，組員實際操作系統檢查，確認無誤才提交版本。

研究：AI 用於搜尋文獻與導讀論文，例如整理 BST 在 ShuttleSet 上的資料切分方式，以確認哪些比賽未參與其訓練。也用於部分演算法的優化、以小型實驗驗證想法是否可行，以及把中間結果視覺化，例如將骨架、軌跡與擊球時刻疊回影片逐幀檢視。

報告與展示：AI 參與了討論章節的內容整理、全文順稿與排版，並撰寫報告圖片的生成程式。專題競賽的影片分鏡、模組介紹動畫的程式與海報版面規劃也由 AI 協助。報告中的數據皆來自實驗程式的輸出，並非由 AI 生成。

確保正確性的做法

AI 產出的程式與文字可能看似合理卻有錯，因此我們依影響程度採取不同的檢查方式：

1. **評分程式由人閱讀：**實驗的評分程式決定報告中所有準確率數字，因此要求 AI 以最直白的寫法撰寫，並由組員自行讀過確認邏輯。其他必要的程式也同樣自行閱讀。
2. **以不同模型交叉檢查：**重要的程式與文件會交由另一個模型審查。例如第二個模型在審查分段實驗時發現，標註工具輸出的 1354 筆真值，其秒數欄位全部少算一幀。

3. **視覺化檢查：**不只看程式回報的數字，而是把結果疊回影片或畫成圖，以人眼確認是否合理，許多錯誤是在這一步才發現的。

4. **標出依據：**要求 AI 在提出結論時附上依據，例如程式的檔案與行號、實驗輸出的檔案或原始論文的段落，方便回頭查證。

5. **重點驗證結果：**關鍵數字重新執行評分程式確認，修改模組後也以既有的評估結果作為回歸檢查，確認沒有退步。

AI 大幅縮短了實作所需的時間，讓小組能把心力放在設計與驗證上。代價是組員對部分實作細節並不熟悉，因此我們把人工閱讀集中在評分程式這類直接影響結論的部分。

參考文獻

[1] W.-Y. Wang, Y.-C. Huang, T.-U. Ik, and W.-C. Peng. "ShuttleSet: A human-annotated stroke-level singles dataset for badminton tactical analysis." arXiv: 2306.04948.

[2] Y.-H. Chien and F. Yu. "Automated hit-frame detection for badminton match analysis." arXiv: 2307.16000v2 [cs.CV].

[3] Y.-J. Chen and Y.-S. Wang, "TrackNetV3: Enhancing ShuttleCock tracking with augmentations and trajectory rectification," in *Proceedings of the 5th ACM International Conference on Multimedia in Asia (MMAsia '23)*, 2023. DOI: 10.1145/3595916.3626370

[4] J.-Y. Chang. "BST: Badminton stroke-type transformer for skeleton-based action recognition in racket sports." arXiv: 2502.21085.

[5] K. Guo et al., "AI-Powered badminton coach: Automated match analysis and key event annotation," in *2025 IEEE Region 10 Symposium (TENSYMP)*, 2025. DOI: 10.1109/TENSYMP63728.2025.11144996

[6] P. Liu and J.-H. Wang, "MonoTrack: Shuttle trajectory reconstruction from monocular badminton video," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2022, pp. 3512–3521. arXiv: 2204.01899.

[7] YeonSeong-Lee. "RallyLens: Badminton match video → player & shuttle tracking → court visualization → Korean-language LLM rally analysis report." commit 331c06b, GitHub repository, Accessed: Oct. 24, 2026. [Online]. Available: https://github.com/YeonSeong-Lee/rallylens

[8] S. Bosi, "Audio-video techniques for the analysis of players behaviour in badminton matches," Academic year 2020–2021, M.Sc. thesis, Politecnico di Milano, 2021.

[9] JWM0203. "RallyLive: 羽毛球比赛的实时 AI 分析系统." commit d60670e, GitHub repository, Accessed: Oct. 24, 2026. [Online]. Available: https://github.com/JWM0203/RallyLive

[10] A. Ghosh, S. Singh, and C. V. Jawahar, "Towards structured analysis of broadcast badminton videos," in *2018 IEEE Winter Conference on Applications of Computer Vision (WACV)*, 2018, pp. 296–304. arXiv: 1712.08714.

[11] amramer. "Badminton-VisionAI: AI-Powered badminton performance analysis system." commit a17e472, GitHub repository, Accessed: Oct. 24, 2026. [Online]. Available: https://github.com/amramer/Badminton-VisionAI

[12] K. A. K. Singha, M. A. Ali, T. Acharjee, P. Debnath, and S. Ahmadullah, "Badminton match analysis along with players and shuttlecock setection using hybrid CNN," in *2025 IEEE Silchar Subsection Conference (SILCON)*, 2025. DOI: 10.1109/SILCON67893.2025.11327275

[13] Y.-C. Huang, I.-N. Liao, C.-H. Chen, T.-U. İk, and W.-C. Peng, "TrackNet: A deep learning network for tracking high-speed and tiny objects in sports applications," in *2019 16th IEEE International Conference on Advanced Video and Signal Based Surveillance (AVSS)*, 2019. arXiv: 1907.03698.

[14] sunwuzhou03. "SoloShuttlePose: Lightweight singles badminton player and court detection," GitHub repository. [Online]. Available: https://github.com/sunwuzhou03/SoloShuttlePose

[15] A. Ghosh and C. V. Jawahar, "SmartTennisTV: Automatic indexing of tennis videos," in *National Conference on Computer Vision, Pattern Recognition, Image Processing and Graphics (NCVPRIPG)*, 2017. arXiv: 1801.01430.

[16] wulongyuan. "BadmintonCV: Badminton court and players detection using OpenCV," GitHub repository. [Online]. Available: https://github.com/wulongyuan/BadmintonCV

[17] M. Plakal and D. P. W. Ellis. "YAMNet," TensorFlow Models, official model documentation, Accessed: Oct. 14, 2026. [Online]. Available: https://github.com/tensorflow/models/tree/master/research/audioset/yamnet

[18] J. F. Gemmeke et al., "Audio set: An ontology and human-labeled dataset for audio events," in *Proc. IEEE ICASSP*, 2017. [Online]. Available: https://research.google/pubs/audio-set-an-ontology-and-human-labeled-dataset-for-audio-events/